ΕΛΛΗΝΙΚΟ ΑΝΟΙΚΤΟ ΠΑΝΕΠΙΣΤΗΜΙΟ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ ΚΑΙ ΤΕΧΝΟΛΟΓΙΑΣ
ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ «ΠΛΗΡΟΦΟΡΙΚΗ»
ΘΕΜΑΤΙΚΗ ΕΝΟΤΗΤΑ ΠΛΗ24 «ΣΧΕΔΙΑΣΜΟΣ ΛΟΓΙΣΜΙΚΟΥ»

ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ: 2025-2026

Αναφορά Έργου

Εφαρμογή WikiViewer

ΕΑΠ – ΠΛΗ 24

3η ΓΡΑΠΤΗ ΕΡΓΑΣΙΑ 2025-2026

No	Ονοματεπώνυμο	Α.Μ.
1	Αθανάσιος Αδαμάκης	
2	Νίκος Ζουμάκης	
3	Νίκος Ντινόπουλος	

ΣΧΟΛΙΑ ΠΡΟΣ ΤΟΝ ΚΑΘΗΓΗΤΗ	Θα θέλαμε να αναφέρουμε ότι για την αντιμετώπιση επιμέρους τεχνικών δυσκολιών και την αποσφαλμάτωση του κώδικα (debugging), αξιοποιήσαμε επικουρικά εργαλεία Τεχνητής Νοημοσύνης (ChatGPT, Gemini, Copilot). Η συμβολή τους λειτούργησε καθαρά συμβουλευτικά, διευκολύνοντας την κατανόηση των νέων τεχνολογιών που εισήχθησαν στο πλαίσιο της εργασίας.

Υπεύθυνη Δήλωση Φοιτητή: *Βεβαιώνω ότι είμαι συγγραφέας αυτής της εργασίας και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία αυτής της εργασίας, είναι πλήρως αναγνωρισμένη και αναφέρεται, είτε στο σημείο «Σχόλια προς καθηγητή», είτε μέσα στην εργασία. Επίσης, έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε αυτές αναφέρονται ακριβώς, είτε παραφρασμένες. Επίσης, βεβαιώνω ότι αυτή η εργασία προετοιμάστηκε από εμένα προσωπικά ειδικά για τη συγκεκριμένη Θεματική Ενότητα.*

☒ Συμφωνώ και αποδέχομαι την ανωτέρω δήλωση

☐ Δε συμφωνώ και δεν αποδέχομαι την ανωτέρω δήλωση (στην περίπτωση αυτή, ο Κ-Σ έχει δικαίωμα να μην αξιολογήσει την εργασία του φοιτητή)

Ημερομηνία ανακοίνωσης εργασίας:, 21/01/2026

Ημερομηνία παράδοσης εργασίας: 04/03/2026

ΔΕΝ ΘΑ ΔΟΘΟΥΝ ΠΑΡΑΤΑΣΕΙΣ

ΣΤΟΙΧΕΙΑ ΠΟΥ ΣΥΜΠΛΗΡΩΝΕΙ Ο ΚΑΘΗΓΗΤΗΣ

ΗΜΕΡΟΜΗΝΙΑ ΑΞΙΟΛΟΓΗΣΗΣ	
ΒΑΘΜΟΣ	

ΣΧΟΛΙΑ ΠΡΟΣ ΦΟΙΤΗΤΗ / ΦΟΙΤΗΤΡΙΑ

Αναλυτική Αξιολόγηση

Άσκηση	Περιγραφή	Ποσοστό
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 1	Διαχείριση του Έργου με έμφαση στη χρήση του εργαλείου συνεργασίας	15
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2.Α	Διάγραμμα ER, Διάγραμμα Κλάσεων και Υλοποίηση Κλάσεων σε Java (περιλαμβάνει χρήση API και επεξεργασία json)	15
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2.Β	Δημιουργία GUI Εφαρμογής	20
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2.Γ	Παρουσίαση Στατιστικών Στοιχείων	20
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2.Δ	Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής	15
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 3	Κριτικός Απολογισμός του Έργου	5
ΔΡΑΣΤΗΡΙΟΤΗΤΑ 4	Δημιουργία video	10
	Σύνολο	100

ΠΕΡΙΕΧΟΜΕΝΑ

1 ΕΙΣΑΓΩΓΗ 4

1.1 Παραδοχές 5

2 ΔΙΑΧΕΙΡΙΣΗ ΕΡΓΟΥ 6

2.1 Υπολογισμός της απαιτούμενης προσπάθειας ανά απαίτηση 6

2.2 Υπολογισμός των προτεραιοτήτων 9

2.3 Το product backlog 11

2.4 Οργάνωση ομάδος και αναθέσεις αρμοδιοτήτων 17

2.5 Παρακολούθηση της προσπάθειας κατά τη διάρκεια του έργου 21

2.6 Χρήση εργαλείου trello 26

3 ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2 - ΥΛΟΠΟΙΗΣΗ ΕΡΓΟΥ 32

3.1 Ερώτημα Α– Διάγραμμα Κλάσεων και Υλοποίηση Κλάσεων σε Java 32

3.2 Ερώτημα Β – Δημιουργία GUI Εφαρμογής 53

3.3 Ερώτημα Γ – Παρουσίαση στατιστικών 79

3.4 Ερώτημα Δ – Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής 85

4 ΚΡΙΤΙΚΟΣ ΑΠΟΛΟΓΙΣΜΟΣ ΤΟΥ ΕΡΓΟΥ 91

4.1 Προβλήματα που αντιμετωπίσαμε και τι θα αλλάζαμε. 91

4.2 Αποκλίσεις στις επαναλήψεις (Sprints) σε σχέση με τα προβλεπόμενα. 92

4.3 Χρόνος που απαιτήθηκε σε σχέση με τον προϋπολογισμένο. 92

4.4 Αξιολόγηση εργαλείου συνεργασίας Trello. 93

4.5 Κίνδυνοι που πραγματώθηκαν και τρόπος απόκρισης. 93

4.6 Τι θα αλλάζαμε σε ένα επόμενο αντίστοιχο έργο. 93

4.7 Θετικά σημεία που αποκομίσαμε από την εργασία. 94

5 VIDEO 96

6 ΑΝΑΦΟΡΕΣ 97

1 ΕΙΣΑΓΩΓΗ

Το σύστημα Wikiviewer επιτρέπει την άντληση και αποθήκευση πληροφοριών από την ελληνική διαδικτυακή εγκυκλοπαίδεια Wikipedia σε περιβάλλον desktop.

Οι λειτουργίες που επιτελεί είναι οι εξής:

- **Επικοινωνία με το Wiki API.**
  Αποστολή αιτήματος αναζήτησης, λήψη πληροφοριών άρθρων σε μορφή json και ανάθεσή τους σε πεδία τοπικών στιγμιοτύπων κλάσεων.

- **Προβολή πληροφοριών στο γραφικό περιβάλλον.**
  Ανάκτηση από τοπική λίστα στιγμιοτύπων άρθρων.

- **Αποθήκευση άρθρου και τροποποίηση.**
  Δυνατότητα εμπλουτισμού πληροφοριών με προσθήκη κατηγορίας, βαθμολογίας ενδιαφέροντος και σχολίων. Η αποθήκευση γίνεται σε τοπική βάση δεδομένων.

- **Προβολή αποθηκευμένων άρθρων.**
  Ανάκτηση από την τοπική βάση δεδομένων.

- **Διατήρηση στατιστικών στοιχείων χρήσης.**
  Προβολή στο γραφικό περιβάλλον και εξαγωγή σε αρχείο pdf.

Το σύστημα αναπτύχθηκε με τη γλώσσα προγραμματισμού Java και ακολουθήθηκε η αντικειμενοστραφής προσέγγιση για την υλοποίησή του, ενώ για τον προγραμματισμό και την παρακολούθηση του κύκλου ανάπτυξης έγινε χρήση ευέλικτων μεθόδων διαχείρισης έργου.

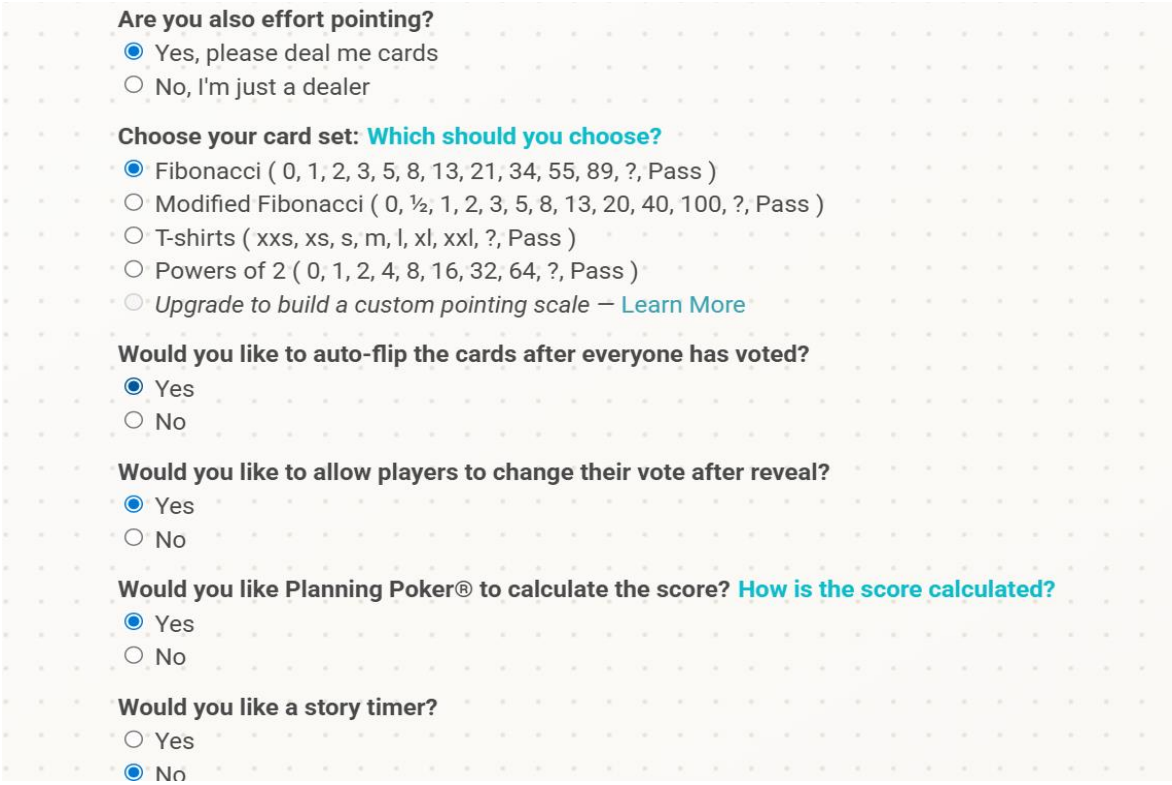
1.1 Παραδοχές

- Η γλώσσα του βασικού γραφικού περιβάλλοντος της εφαρμογής είναι η αγγλική.
- Αμυντικός προγραμματισμός εφαρμόζεται στην απάντηση που λαμβάνεται από το Wiki API και όχι στον όρο αναζήτησης που εισάγει ο χρήστης.
- Στις ανακτώμενες πληροφορίες κάθε άρθρου περιλαμβάνεται μορφοποιημένο και το πλήρες κείμενο.
- Γίνεται περιορισμός στο πλήθος των ανακτώμενων αποτελεσμάτων σε 10 για κάθε αναζήτηση που πραγματοποιείται.
- Γίνεται διαχωρισμός της λίστας αποτελεσμάτων αναζήτησης άρθρου live από το API και των τοπικά αποθηκευμένων (ενδεχομένως εμπλουτισμένων) άρθρων του χρήστη.
- Η κλάση **Dbmanager** όπως αποτυπώνεται στο διάγραμμα κλάσεων περιλαμβάνει μία επιπλέον μέθοδο ***editCategory,*** καθώς και μία επιπλέον παράμετρο *category* στη μέθοδο ***updateArticle,*** οι οποίες δεν έχουν υλοποιηθεί στον κώδικα του προγράμματος. Αφορούν τις λειτουργικές απαιτήσεις R7-R9 της εκφώνησης.

2 ΔΙΑΧΕΙΡΙΣΗ ΕΡΓΟΥ

2.1 Υπολογισμός της απαιτούμενης προσπάθειας ανά απαίτηση

Για την εκτίμηση της απαιτούμενης προσπάθειας υλοποίησης των απαιτήσεων του έργου, επιλέχθηκε η ευέλικτη (agile) μέθοδος Planning Poker. Η μέθοδος αυτή είναι αποτελεσματική γιατί βασίζεται στην αμερόληπτη άποψη κάθε μέλους της ομάδας. Οι εκτιμήσεις γίνονται ταυτόχρονα και ανεξάρτητα και ακολουθεί συζήτηση με επιχειρήματα και εν τέλει ομόφωνη απόφαση.



Η κλίμακα εκτίμησης που χρησιμοποιήθηκε βασίστηκε στην ακολουθία Fibonacci (1, 2, 3, 5, 8, 13, 21). Η επιλογή της συγκεκριμένης ακολουθίας δεν ήταν τυχαία. Καθώς οι αριθμοί μεγαλώνουν, η απόσταση μεταξύ τους αυξάνεται, γεγονός που διευκόλυνε την ομάδα να πάρει γρήγορες αποφάσεις, κυρίως στα πιο απαιτητικά κομμάτια του έργου, αναγκάζοντας τα μέλη να ταξινομήσουν την προσπάθεια σε κατηγορίες μεγέθους και όχι σε απόλυτα ακριβείς προβλέψεις.

Η διαδικασία υπολογισμού της προσπάθειας κατά τη διάρκεια του αρχικού Sprint Planning είχε ως εξής:

- Έπειτα από σύντομη προφορική συζήτηση καταγράφηκαν στο Trello board όλες οι απαιτήσεις και προδιαγραφές που απαιτούνταν από την εκφώνηση της εργασίας.

- Στη συνέχεια ο Scrum Master δημιούργησε το παιχνίδι στην πλατφόρμα https://www.planningpoker.com/ συμπεριλαμβάνοντας όλες τις εργασίες (κάρτες του Trello) που είχαν καταγραφεί.

- Με την έναρξη του παιχνιδιού και σε κάθε γύρο του, τα μέλη επέλεγαν κρυφά μία κάρτα με τον αριθμό Fibonacci που αντικατόπτριζε την πολυπλοκότητα και τον όγκο δουλειάς (Story Points) της συγκεκριμένης εργασίας κατά τη γνώμη τους.

- Οι εκτιμήσεις αποκαλύπτονταν αυτόματα ταυτόχρονα με την επιλογή του τελευταίου χρονικά μέλους. Στις περιπτώσεις όπου υπήρχε απόκλιση μεταξύ των μελών, τα άτομα με τη χαμηλότερη και την υψηλότερη εκτίμηση αιτιολογούσαν το σκεπτικό τους. Η διαδικασία επαναλαμβανόταν μέχρι να επιτευχθεί ομοφωνία.

Your Planning Poker® Game Summary

Effort

Deciding the effort of every backlog card

Story	Story Title	Score
1	User Story - Τροποποίηση άρθρου	3
2	User Story - Αναζήτηση λέξη-κλειδί	3
3	Προβολή στατιστικών	5
4	Παραγωγή Στατιστικών & Εξαγωγή σε PDF	5
5	Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής	13
6	Κριτικός Απολογισμός του Έργου	13
7	Δημιουργία Video Παρουσίασης	8
8	User Story - αποθηκευμένα άρθρα	3
9	Τεχνική απαίτηση GUI (main)	8
10	GUI (stats)	5
11	Τεχνική απαίτηση Διάγραμμα ER	5
12	Τεχνική απαίτηση Βάσης Δεδομένων	5
13	Τεχνική Απαίτηση - (Class Diagram)(1/2)	3
14	Τεχνική Απαίτηση - (Class Diagram)(2/2)	3
15	GUI (about)	3

Προκειμένου να καλυφθούν οι απαιτήσεις τεκμηρίωσης του έργου, τα Story Points που προέκυψαν μεταφράστηκαν σε ανθρωποημέρες, ορίζοντας εξαρχής μια αναλογία μετατροπής της τάξης: 1 Story Point = 1 ανθρωποημέρα.

Παραδοχές

Κατά την εφαρμογή της μεθόδου και την εξέλιξη του έργου, έγιναν οι εξής παραδοχές:

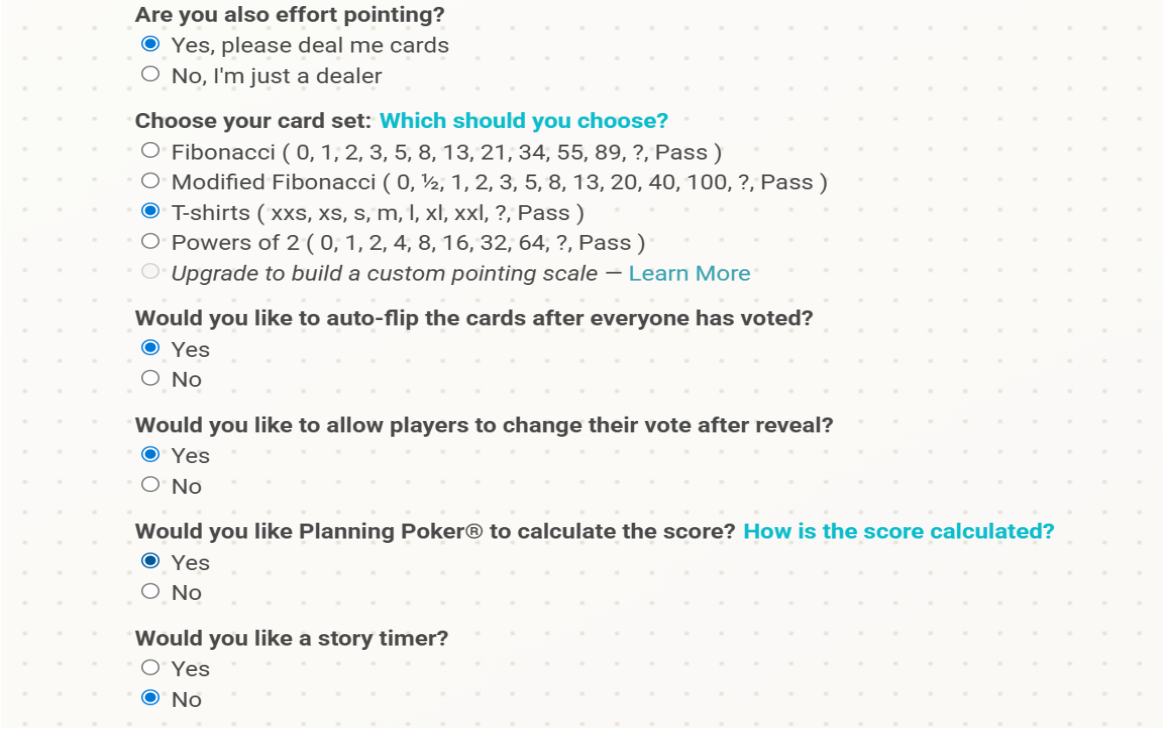
Ενώ ο αρχικός προγραμματισμός κάλυψε τις βασικές απαιτήσεις του συστήματος, στην πορεία υλοποίησης προέκυψε η ανάγκη προσθήκης νέων, υποστηρικτικών καρτών (5 εργασίες) για την εύρυθμη λειτουργία του λογισμικού αλλά και την καλύτερη αξιοποίηση των δυνατοτήτων κάθε μέλους.

Ο «Κριτικός Απολογισμός» αρχικά είχε θεωρηθεί ως εργασία της τελικής συγγραφής της εργασίας. Στην πορεία και έπειτα από τη μελέτη των πρακτικών Scrum κρίθηκε ως απαραίτητο στοιχείο για κάθε επιμέρους Sprint. Ως εκ τούτου, διασπάστηκε σε επιμέρους κάρτες και κατανεμήθηκε και στα τρία Sprints του έργου.

Για τις κάρτες που προστέθηκαν κατά τη διάρκεια των Sprints, δεν ακολουθήθηκε η τυπική διαδικασία του Planning Poker. Λόγω της εξοικείωσης που είχε ήδη αποκτήσει η ομάδα με την ταχύτητά της (velocity), η προσπάθεια αυτών των καρτών υπολογίστηκε προφορικά και ομόφωνα κατά τη διάρκεια των συναντήσεων (scrum ceremonies).

2.2 Υπολογισμός των προτεραιοτήτων

Για την ιεράρχηση των απαιτήσεων (Product Backlog), η ομάδα μας χρησιμοποίησε τη μέθοδο Planning Poker χρησιμοποιώντας αυτή τη φορά τις κάρτες «T-Shirt Sizes» με την παραδοχή ότι το XXS αντιστοιχεί στην μικρότερη προτεραιότητα και το XXL στη μεγαλύτερη.



Ο τρόπος που εργαστήκαμε ήταν όμοιος με τον υπολογισμό απαιτούμενης προσπάθειας που είχε προηγηθεί. Πριν από οποιαδήποτε συγγραφή κώδικα, θεωρήθηκε επιβεβλημένη η σαφής αποτύπωση της αρχιτεκτονικής του συστήματος. Για τον λόγο αυτό, δώσαμε τη μέγιστη προτεραιότητα (XXL) στις απαιτήσεις σχεδίασης, δηλαδή στη δημιουργία του Διαγράμματος Κλάσεων (Class Diagram) και του Διαγράμματος Οντοτήτων-Συσχετίσεων (ER Diagram). Αυτά τα διαγράμματα λειτούργησαν ως ο απαραίτητος οδηγός για την επακόλουθη ανάπτυξη της υποδομής, όπως η δημιουργία της βάσης δεδομένων Derby και η διασύνδεση με το API της Wikipedia, καθώς αποτελούν τον πυρήνα (backend) πάνω στον οποίο χτίζεται όλη η εφαρμογή. Αντίθετα, λειτουργίες όπως η Εξαγωγή σε PDF ή το γραφικό περιβάλλον (GUI), έλαβαν χαμηλότερη προτεραιότητα, καθώς εξαρτώνται άμεσα από την ολοκλήρωση των προηγούμενων σταδίων και αποτελούν προστιθέμενη αξία στο τελικό στάδιο του έργου.

Μέσω αυτής της διαδικασίας διασφαλίσαμε ότι στο Sprint 1 υλοποιήθηκαν τα απολύτως απαραίτητα θεμέλια του λογισμικού.

Your Planning Poker® Game Summary

Priority

Deciding the priority of every backlog card

Story	Story Title	Score
1	User Story - Τροποποίηση άρθρου	xl
2	User Story - Αναζήτηση λέξη-κλειδί	xxl
3	Προβολή στατιστικών	m
4	Παραγωγή Στατιστικών & Εξαγωγή σε PDF	m
5	Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής	s
6	Κριτικός Απολογισμός του Έργου	s
7	Δημιουργία Video Παρουσίασης	xs
8	User Story - αποθηκευμένα άρθρα	l
9	Τεχνική απαίτηση GUI (main)	m
10	GUI (stats)	s
11	Τεχνική απαίτηση Διάγραμμα ER	xxl
12	Τεχνική απαίτηση Βάσης Δεδομένων	xxl
13	Τεχνική Απαίτηση - (Class Diagram)(1/2)	xxl
14	Τεχνική Απαίτηση - (Class Diagram)(2/2)	xxl
15	GUI (about)	xxs

2.3 Το product backlog

Στον παρακάτω πίνακα παρουσιάζεται το Product Backlog του έργου, κατανεμημένο στα τρία διαδοχικά Sprints. Για κάθε απαιτούμενη λειτουργικότητα (User Story), καταγράφηκαν συμπληρωματικές πληροφορίες όπως η προτεραιότητα (Priority) και η εκτιμώμενη προσπάθεια (Story Points). Παράλληλα, για την αποφυγή αστοχιών κατά την υλοποίηση, ορίστηκαν αυστηρά Κριτήρια Αποδοχής (Acceptance Criteria) ανά κάρτα, τα οποία αποτέλεσαν τον οδηγό ελέγχου ποιότητας για την ομάδα. Το πλήρες και διαδραστικό Backlog, μαζί με τα γραφήματα παρακολούθησης (Burndown) και τον υπολογισμό της ταχύτητας (Velocity) της ομάδας, παραδίδεται ως ξεχωριστό επισυναπτόμενο αρχείο Excel (Product_Backlog.xlsx).

Sprint 1

Α/Α	Τίτλος	Περιγραφή	Effort	Priority	Sprint	Υπεύθυνος	Acceptance Criteria
1	User Story - αναζήτηση άρθρου	Ως χρήστης θέλω να κάνω αναζήτηση άρθρου με λέξεις κλειδιά και να μου επιστρέφει τα αποτελέσματα.	3	xxl	1	NZ	1. Καλεί επιτυχώς το API 2. Λήψη JSON response 3. Εμφάνιση στην κονσόλα 4. Καθαρισμός με JSOUP
2	Τεχνική απαίτηση GUI (main)	Δημιουργία GUI αρχικού παραθύρου	8	m	1	NN	1. Δημιουργία Κεντρικού παραθύρου 2. Να περιλαμβάνει βασικές λειτουργίες
3	Τεχνική απαίτηση Διάγραμμα ER	Διάγραμμα Entity-Relationship (ER)	5	xxl	1	AA	1. Visual paradigm project 2. Κατανοητό και περιληπτικό για τη σχεδίαση της βάσης 3. Να είναι σε pdf

4	Τεχνική απαίτηση Βάσης Δεδομένων	Σχεδιασμός & Υλοποίηση Τοπικής Βάσης Δεδομένων (Derby)	5	xxl	1	AA	1. Να είναι βάση με χρήση της Derby 2. Περιλαμβάνει όλες τις απαραίτητες στήλες 3. Να υπάρχει SQL script για τη δημιουργία πινάκων
5	Τεχνική Απαίτηση - (Class Diagram) 1/2	Συγκεντρωτικό Διάγραμμα Κλάσεων (UML)	3	xxl	1	NZ	1. Visual Paradigm 2. Να είναι πιστό στον κώδικα 3. Να έχει συσχετίσεις
6	Κριτικός Απολογισμός του Έργου	Απολογισμός sprint	5	l	1	S	1. Συζήτηση και καταγραφή του τι πήγε καλά και τι όχι στο τρέχον Sprint. 2. Καθορισμός ενέργειας βελτίωσης που θα εφαρμοστεί στο επόμενο Sprint. 3. Ενημέρωση του αρχείου Word της αναφοράς με τα συμπεράσματα.

Sprint 2

A/A	Τίτλος	Περιγραφή	Effort	Priority	Sprint	Υπεύθυνος	Acceptance Criteria
1	User Story - Τροποποίηση άρθρου	Ως χρήστης, θέλω να επεξεργάζομαι ένα αποθηκευμένο άρθρο προσθέτοντας κατηγορία, σχόλια και βαθμολογία.	3	xl	2	NZ	1.Φόρμα επεξεργασίας με πεδία: Κατηγορία (string), Σχόλια (string), Βαθμολογία (int). 2.Έλεγχος εγκυρότητας στη βαθμολογία (1-5) 3.Το κουμπί "Ενημέρωση" αποθηκεύει τις αλλαγές στη ΒΔ.
2	User Story - Αναζήτηση λέξη-κλειδί	Ως χρήστης θέλω να κάνω αναζήτηση άρθρων βάσει λέξης-κλειδιού (Keyword Search)	3	xxl	2	S	1.Πεδίο αναζήτησης που ψάχνει στον τίτλο ή/και στο κείμενο των αποθηκευμένων άρθρων. 2.Επιστροφή σχετικών αποτελεσμάτων από τη Derby.
3	(UserStory) - αποθήκευση άρθρου	Ως χρήστης, θέλω να επιλέγω ένα άρθρο από τη λίστα αναζήτησης και να το αποθηκεύω, ώστε να το έχω διαθέσιμο μόνιμα στη βάση δεδομένων μου.	3	1	2	S	1.Επιτυχής σύνδεση με βάση δεδομένων 2.Έλεγχος για υπαρξη των άρθρων 3.Υπάρχει κουμπί "Αποθήκευση" δίπλα ή κάτω από το επιλεγμένο άρθρο.

4	(UserStory) - Προβολή αποθηκευμένων άρθρων	Ως χρήστης θέλω να μπορώ να βλέπω τα αποθηκευμένα άρθρα ανά κατηγορία που έχω παλιότερα αποθηκεύσει.	3	1	2	NN	1.Εμφάνιση λίστας με τα άρθρα που υπάρχουν στη Derby. 2.Δίπλα σε κάθε άρθρο φαίνονται και Κατηγορία, Βαθμολογία. 3.Δυνατότητα επιλογής Κατηγορίας (Dropdown menu).
5	(User Story) Προβολή αποτελεσμάτων αναζήτησης	Ως χρήστης, θέλω να βλέπω τα αποτελέσματα της αναζήτησής μου σε λίστα στο περιβάλλον της εφαρμογής, ώστε να διαβάζω την περίληψη και να επιλέγω ποιο με ενδιαφέρει.	3	m	2	NN	1.Λίστα με τα αποτελέσματα που επιστρέφει το API. 2.Εμφανίζονται: Τίτλος, Snippet και Timestamp. 3.Αν η αναζήτηση δεν φέρει αποτελέσματα, εμφανίζεται μήνυμα "Δεν βρέθηκαν αποτελέσματα".
6	(User Story) προβολή στατισικών	Ως χρήστης, θέλω να βλέπω στατιστικά στοιχεία χρήσης.	5	m	2	NN	1.Υπολογισμός και προβολή των πιο συχνών λέξεων-κλειδιών αναζήτησης (Απαιτεί να καταγράφετε το ιστορικό αναζητήσεων στη ΒΔ). 2.Υπολογισμός και προβολή πλήθους άρθρων ανά Κατηγορία. 3.Η λέξη κάθε αναζήτησης αποθηκεύεται αυτόματα στο ιστορικό αναζητήσεων στη ΒΔ

7	Τεχνική Απαίτηση - (Class Diagram) 2/2	Τεχνική απαίτηση για τον σχεδιασμό των κλάσεων του Sprint 1.	3	xxl	2	NZ	1. Visual Paradigm 2. Να είναι πιστό στον κώδικα 3. Να έχει συσχετίσεις
8	Παραγωγή Στατιστικών & Εξαγωγή σε PDF	Ως χρήστης, θέλω να εξάγω σε αρχείο PDF τα στατιστικά χρήσης, ώστε να έχω μια αναφορά των ενδιαφερόντων μου.	5	m	2	NZ	1.Κουμπί "Εξαγωγή σε PDF". 2.Δημιουργία κλάσης που χρησιμοποιεί βιβλιοθήκη PDF 3.Το PDF περιέχει δύο πίνακες: Δημοφιλέστερες λέξεις-κλειδιά (από τον πίνακα ιστορικού) και Άρθρα ανά κατηγορία (από τον πίνακα άρθρων).
9	GUI about	Το παράθυρο με τα σχετικά στοιχεία της εφαρμογής.	3	xxs	2	NN	1.Περιλαμβάνει τους συμμετέχοντες 2.Γενικά στοιχεία εφαρμογής (έκδοση, έτος κτλ)
10	Τεχνική απαίτηση - Άντληση δεδομένων και αξιοποίησή τους από ΒΔ	Προσθήκη μεθόδων σε συνοριακή κλάση DBmanager για να αντλούνται αποθηκευμένα δεδομένα αλλά και να επεξεργάζονται πριν τη χρήση τους.	5	l	2	NN	1.JPQL queries για άντληση δεδομένων 2.Μέθοδοι για υπολογισμό και επεξεργασία αποθηκευμένων δεδομένων. 3. Ενσωμάτωση βάσης για ευελιξία.
11	Κριτικός Απολογισμός του Έργου	Απολογισμός sprint	3	l	2	NZ	1. Συζήτηση και καταγραφή του τι πήγε καλά και τι όχι στο τρέχον Sprint. 2. Καθορισμός ενέργειας βελτίωσης που θα εφαρμοστεί στο επόμενο Sprint. 3. Ενημέρωση του αρχείου Word της αναφοράς με τα συμπεράσματα.

Sprint 3

Α/Α	Τίτλος	Περιγραφή	Effort	Priority	Sprint	Υπεύθυνος	Acceptance Criteria
1	Κριτικός απολογισμός του έργου	Γενικός απολογισμός	5	s	3	NN	1. Συνολική αποτίμηση του έργου (τι πήγε καλά και τι όχι συνολικά). 2. Ολοκλήρωση του Κεφαλαίου 4 της τελικής αναφοράς.
2	Δημιουργία Video παρουσίασης	Παρουσίαση της εφαρμογής	8	xs	3	NZ	1. Το βίντεο παρουσιάζει καθαρά τη λειτουργία του GUI, τη διασύνδεση με τη ΒΔ και περιπτώσεις χρήσης. 2. Η διάρκεια είναι εντός των ορίων της εκφώνησης. 3. Το αρχείο ανέβηκε στο YouTube ή σε άλλη πλατφόρμα και το link δουλεύει..
3	Συνολικός έλεγχος και εκτέλεση εφαρμογής	Debugging	13	s	3	NZ	1. Η εφαρμογή κάνει build (Clean & Build) στο NetBeans χωρίς σφάλματα. 2. Εκτελούνται επιτυχώς οι λειτουργίες R1 έως R5 χωρίς να "σκάει" το πρόγραμμα (Exceptions). 3. Το παραγόμενο αρχείο PDF

							εξάγεται και ανοίγει κανονικά.
4	Συγγραφή εργασίας	Συγκέντρωση αρχείων για υποβολή	8	s	3	S, NZ, NN	1. Το κείμενο ακολουθεί το Πρότυπο Συγγραφής. 2. Περιλαμβάνονται όλα τα απαιτούμενα διαγράμματα (Burndown, Class, ER). 3. Το παραδοτέο αρχείο zip/rar έχει τη σωστή ονομασία.
5	Παραγωγή Javadoc	Σχολιασμός κώδικα	3	s	3	S	1. Προσθήκη σχολίων Javadoc στις βασικές κλάσεις και μεθόδους. 2. Επιτυχής παραγωγή των HTML αρχείων από το NetBeans.

2.4 Οργάνωση ομάδος και αναθέσεις αρμοδιοτήτων

Η οργάνωση της ομάδας κατά την υλοποίηση του έργου ήταν η ακόλουθη:

- Product Owner Δρ. Πάνος Φιτσιλής
- Scrum Master Αθανάσιος Αδαμάκης (“S” η συντομογραφία στο Backlog)
- Developer Νίκος Ζουμάκης (“NZ” η συντομογραφία στο Backlog)
- Developer Νίκος Ντινόπουλος (“NN” η συντομογραφία στο Backlog)

Η πλήρης και αναλυτική αντιστοίχιση των μελών με τις εκάστοτε απαιτήσεις αποτυπώνεται στη στήλη «Υπεύθυνος» του επισυναπτόμενου Product Backlog (αρχείο .xlsx). Η κατανομή των εργασιών γινόταν στην αρχή κάθε Sprint, τα μέλη αναλάμβαναν εργασίες από το Sprint Backlog στο Trello, λαμβάνοντας υπόψη την ατομική τους εξειδίκευση και τον διαθέσιμο χρόνο τους.

Κατά τη διάρκεια ανάπτυξης της εφαρμογής «WikiViewer», η ομάδα μας υιοθέτησε το ευέλικτο πλαίσιο (Agile framework) Scrum. Στόχος μας ήταν η σταδιακή και επαυξητική παράδοση του λογισμικού, η συνεχής προσαρμογή στις τεχνικές προκλήσεις και η διατήρηση υψηλού επιπέδου συνεργασίας. Οι βασικές πρακτικές που εφαρμόστηκαν είναι οι εξής:

Καθορισμός Ρόλων (Scrum Roles).

Η ομάδα οργανώθηκε με βάση τους τυπικούς ρόλους του πλαισίου. Ο ρόλος του Scrum Master, είχε ως κύρια ευθύνη την τήρηση των διαδικασιών, την καθοδήγηση της ομάδας στη χρήση των εργαλείων (Trello, GitHub) και την άρση των εμποδίων (όπως επίλυση merge conflicts στο github κτλ.). Τα υπόλοιπα μέλη (μαζί με τον Scrum Master) αποτέλεσαν την Ομάδα Ανάπτυξης (Development Team), ενώ ο ρόλος του Product Owner (υπεύθυνος για τις απαιτήσεις) καλύφθηκε από τον υπεύθυνο καθηγητή του τμήματος, ο οποίος καθόρισε τις απαιτήσεις και τις προδιαγραφές του έργου μέσα από την εκφώνηση της εργασίας αλλά και τις προσωπικές του επισημάνσεις.

Τεχνουργήματα (Scrum Artifacts).

Καταγράψαμε όλες τις απαιτήσεις του συστήματος (R1-R5, διαγράμματα, αναφορά) σε ένα κεντρικό Product Backlog στο Excel. Σε κάθε sprint, μεταφέραμε τις επιλεγμένες κάρτες από τη λίστα Product Backlog στην τρέχουσα λίστα του Sprint.

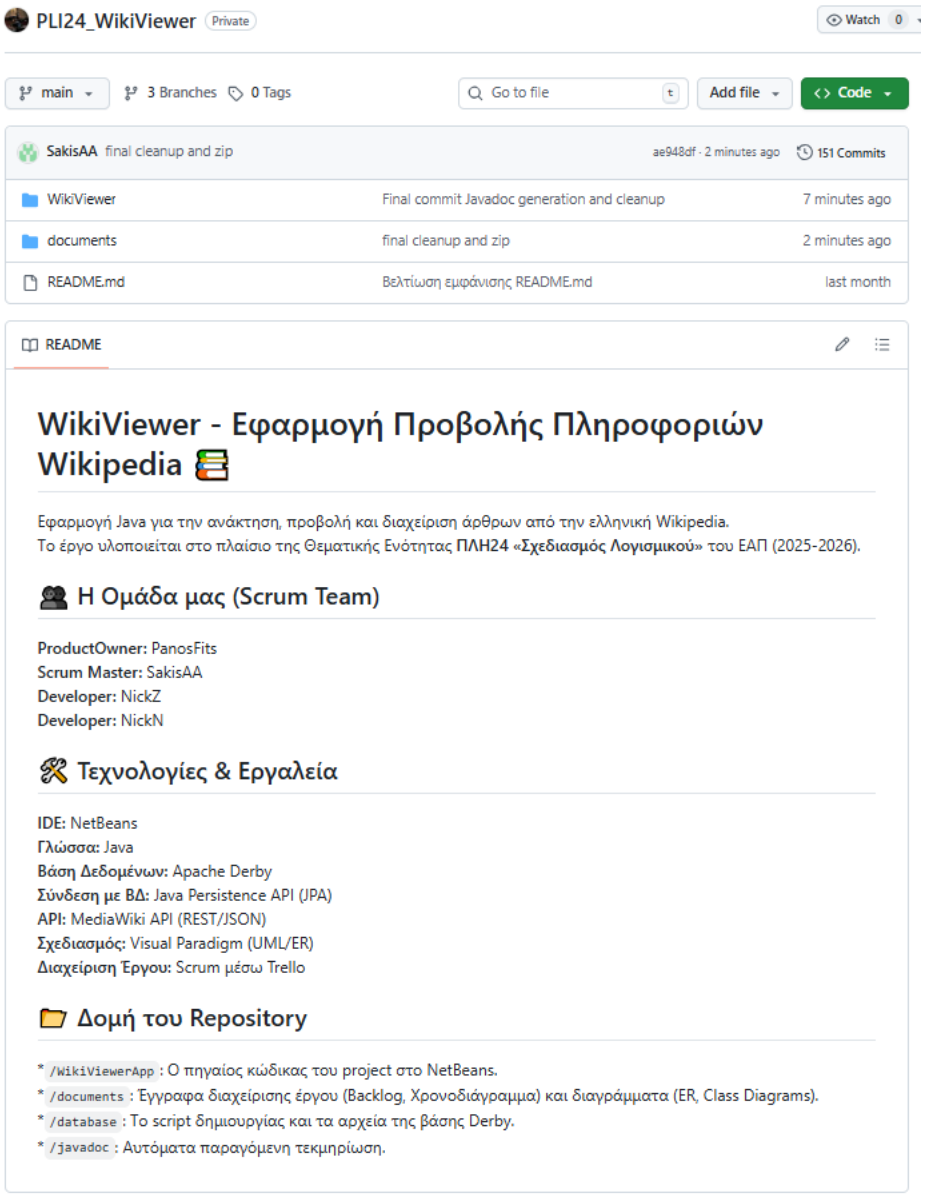
Για κάθε κάρτα του Trello (π.χ. αναζήτηση άρθρου), ορίσαμε κριτήρια αποδοχής. Μια κάρτα θεωρούνταν «Ολοκληρωμένη» (Done) μόνο όταν πληρούνταν όλα τα κριτήρια.

Καταγράφαμε την υπολειπόμενη προσπάθεια (effort) σε καθημερινή βάση, δημιουργώντας Burndown Charts για κάθε Sprint, ώστε να ελέγχουμε οπτικά αν βρισκόμαστε εντός χρονοδιαγράμματος.

Διαχείριση Πηγαίου Κώδικα (Version Control).

Για την ασφαλή, ταυτόχρονη και απρόσκοπτη ανάπτυξη του λογισμικού από τα μέλη της ομάδας, βασιστήκαμε στο σύστημα ελέγχου εκδόσεων Git και την πλατφόρμα του GitHub. Υιοθετήσαμε την πρακτική των μικρών και συχνών υποβολών. Με τον τρόπο αυτό, κάθε επιμέρους λειτουργία ή διόρθωση σφάλματος αποθηκευόταν άμεσα στο αποθετήριο,

διευκολύνοντας τον έλεγχο, την επαναφορά σε προηγούμενες καταστάσεις αν χρειαζόταν, και μειώνοντας τα πιθανά merge conflicts. Το αποθετήριο (repository) περιέχει ένα αναλυτικό αρχείο τεκμηρίωσης (README.md) ώστε να είναι πιο φιλικό και κατανοητό ως προς το περιεχόμενό του.



Τεχνικές Εκτίμησης & Ιεράρχησης.

Πριν την έναρξη του έργου, όλα τα μέλη ψήφισαν για την πολυπλοκότητα και τον απαιτούμενο κόπο κάθε απαίτησης χρησιμοποιώντας πόντους (effort points), επιτυγχάνοντας έτσι μια ρεαλιστική εκτίμηση.

Ιεραρχήσαμε τις απαιτήσεις λαμβάνοντας υπόψη τις τεχνικές εξαρτήσεις. Για παράδειγμα, η βάση δεδομένων και η διασύνδεση με το API της Wikipedia πήραν τη μέγιστη προτεραιότητα, καθώς αποτελούσαν θεμέλιο για όλο το έργο (Sprint 1).

Τελετές (Scrum Ceremonies).

Στην αρχή κάθε Sprint, η ομάδα συναντιόταν για να αποφασίσει ποιες απαιτήσεις θα υλοποιηθούν (Sprint plan), λαμβάνοντας υπόψη το διαθέσιμο χρόνο της ομάδας. Το έργο χωρίστηκε σε 3 Sprints (τα δύο πρώτα για ανάπτυξη κώδικα και το τρίτο για ελέγχους και τεκμηρίωση).

Λόγω της φύσης της εξ αποστάσεως συνεργασίας, προσαρμόσαμε τα Daily Scrums σε τακτικές συνεδρίες μέσω Google Meet. Κάθε μέλος απαντούσε στα τρία βασικά ερωτήματα: Τι έκανα, τι θα κάνω, και τι με εμποδίζει. Σε καθημερινή βάση διατηρούσαμε επικοινωνία μέσω Viber messages (Viber Group).

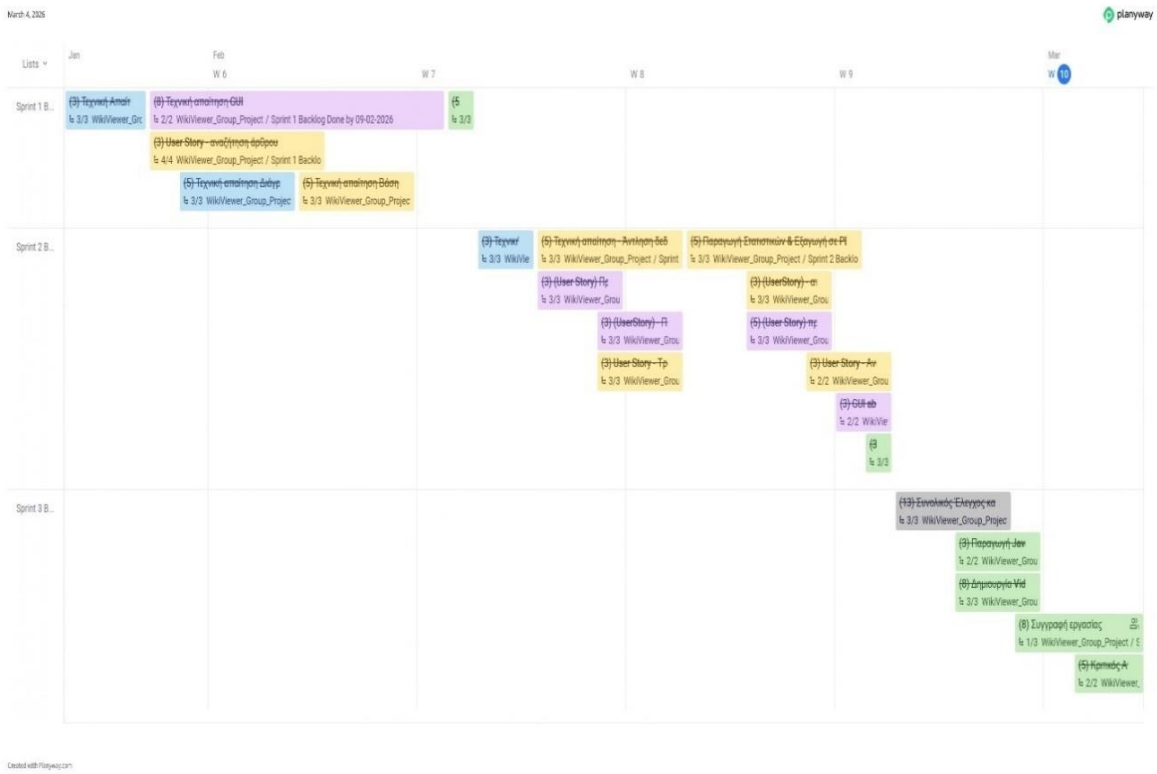
Στο τέλος κάθε Sprint, εφαρμόσαμε την πιο κρίσιμη πρακτική, τον «Κριτικό Απολογισμό». Αναλύαμε τι πήγε καλά και τι όχι αλλά κυρίως τι θα κάνουμε για να λύσουμε το όποιο πρόβλημα συναντήσαμε. Μέσω αυτής της διαδικασίας εντοπίσαμε, για παράδειγμα, δυσλειτουργίες με τη βάση δεδομένων και αποφασίσαμε την ενσωμάτωση της στο project στο επόμενο Sprint.

Όλες οι παραπάνω πρακτικές υποστηρίχθηκαν από το Agile Board στο Trello. Εκεί διαμορφώσαμε τις λίστες της ροής εργασίας (Backlog, To be Done, Done), χρησιμοποιήσαμε χρωματικές ετικέτες (Labels) για να διαχωρίσουμε τα tasks (Frontend, Backend, Documentation) και συνδέσαμε τον πίνακα απευθείας με το GitHub, ώστε κάθε "Done" κάρτα να συνοδεύεται από το αντίστοιχο commit του κώδικα, αξιοποιώντας τις δυνατότητες των powerups του Trello.

Κατά τη διάρκεια του έργου, ήταν αναμενόμενο να προκύψουν διαφορετικές τεχνικές απόψεις, όπως για παράδειγμα η αρχιτεκτονική της βάσης δεδομένων ή η εμφάνιση (GUI) της κύριας οθόνης. Η διαχείριση αυτών των διαφορών έγινε με γνώμονα τη διαφάνεια και τον σεβασμό. Όταν υπήρχε διαφωνία, διοργανωνόταν μια έκτακτη συνάντηση όπου τα μέλη παρέθεταν τεχνικά επιχειρήματα. Έτσι προέκυπτε ομόφωνη απόφαση σε κάθε ζήτημα που μας απασχολούσε. Σε περιπτώσεις που οι τεχνικές διαφορές απειλούσαν το χρονοδιάγραμμα του Sprint, ο Scrum Master παρενέβαινε για να διευκολύνει τη συζήτηση και να διασφαλίσει ότι η ομάδα θα καταλήξει άμεσα στην πιο εφικτή λύση, διατηρώντας το κλίμα συνεργασίας.

2.5 Παρακολούθηση της προσπάθειας κατά τη διάρκεια του έργου

Το Διάγραμμα Gantt που ακολουθεί (αρχείο Gantt.pdf) δημιουργήθηκε μέσω του εργαλείου Planyway του Trello και αποτυπώνει το συνολικό χρονοδιάγραμμα καθώς και τις εξαρτήσεις μεταξύ των εργασιών,. Στο διάγραμμα αποτυπώνεται ο κάθετος άξονας με τις ομαδοποιημένες εργασίες ανά Sprint και ο οριζόντιος άξονας του χρόνου (σε εβδομάδες). Η σειριακή και παράλληλη διάταξη των ράβδων δείχνει τη λογική ροή της υλοποίησης, όπου ο σχεδιασμός προηγήθηκε της ανάπτυξης κώδικα και ο έλεγχος (testing) ακολούθησε στο τελικό στάδιο.



Κατανομή και Χρόνος Υλοποίησης ανά Επανάληψη (Sprint)

Η παρακολούθηση της εξέλιξης του έργου βασίστηκε στην εκτιμώμενη προσπάθεια (Story Points) που είχε καθοριστεί κατά τη διάρκεια του Sprint Planning. Η ομάδα κατάφερε να διατηρήσει τον ρυθμό υλοποίησης πολύ κοντά στις αρχικές εκτιμήσεις, με ελάχιστες αποκλίσεις. Παρακάτω παρουσιάζεται ο χρόνος υλοποίησης και η εκτιμώμενη προσπάθεια ανά απαίτηση, κατανεμημένη στις τρεις επαναλήψεις του έργου.

Sprint 1 (26/01/2026 - 09/02/2026) – Θεμελίωση & Βασικό GUI

Στο πρώτο Sprint δόθηκε έμφαση στη σχεδίαση και τις βασικές υποδομές (ER & Class Diagrams, Api, Database) αλλά και στο αρχικό γραφικό περιβάλλον.

List	Title	Story Points	Start	Due	Πραγματικές ημέρες
Sprint 1	User Story - αναζήτηση άρθρου	3	30/1/2026	4/2/2026	6
Sprint 1	Κριτικός Απολογισμός του Έργου	5	9/2/2026	9/2/2026	1
Sprint 1	Τεχνική απαίτηση Διάγραμμα ER	5	31/1/2026	3/2/2026	4
Sprint 1	Τεχνική απαίτηση GUI	8	30/1/2026	8/2/2026	10
Sprint 1	Τεχνική απαίτηση Βάσης Δεδομένων	5	4/2/2026	7/2/2026	4
Sprint 1	Τεχνική Απαίτηση - (Class Diagram)	3	27/1/2026	29/1/2026	3
ΣΥΝΟΛΟ		**29**			**28**

Στο πρώτο Sprint, ενώ το συνολικό άθροισμα εκτιμώμενης και πραγματικής προσπάθειας ήταν ισορροπημένο, παρατηρήθηκαν σημαντικές μεμονωμένες αποκλίσεις. Συγκεκριμένα, η «**Αναζήτηση Άρθρου**» απαίτησε διπλάσιο χρόνο (6 ανθρωποημέρες αντί για 3) λόγω της απαιτούμενης εκμάθησης για τη σωστή διασύνδεση με το API της Wikipedia. Αντίστοιχα, το «**Αρχικό GUI**» υποεκτιμήθηκε (πήρε 10 αντί για 8), καθώς ο συντονισμός των γραφικών στοιχείων απαίτησε περισσότερες δοκιμές. Αντίθετα, ο «**Κριτικός Απολογισμός**» υπερεκτιμήθηκε σημαντικά, καθώς η ομάδα είχε θέσει συντηρητικούς στόχους τους οποίους και πέτυχε. Το φαινόμενο αυτό είναι απολύτως φυσιολογικό στο αρχικό στάδιο (forming stage) μιας Agile ομάδας.

Sprint 2 (09/02/2026 - 23/02/2026) – Λειτουργικότητα & Διασυνδέσεις

Το δεύτερο Sprint ήταν το πιο απαιτητικό, καθώς υλοποιήθηκε ο κύριος όγκος των λειτουργιών και της διασύνδεσης Frontend-Backend.

List	Title	Story Points	Start	Due	Πραγματικές ημέρες
Sprint 2	User Story - Τροποποίηση άρθρου	3	14/2/2026	16/2/2026	3
Sprint 2	User Story - Αναζήτηση λέξη-κλειδί	3	21/2/2026	23/2/2026	3
Sprint 2	(UserStory) - αποθήκευση άρθρου	3	19/2/2026	21/2/2026	3
Sprint 2	(UserStory) - Προβολή αποθηκευμένων άρθρων	3	14/2/2026	16/2/2026	3
Sprint 2	(User Story) Προβολή αποτελεσμάτων αναζήτησης	3	12/2/2026	14/2/2026	3
Sprint 2	(User Story) προβολή στατισικών	5	19/2/2026	21/2/2026	3
Sprint 2	Τεχνική Απαίτηση - (Class Diagram)	3	10/2/2026	11/2/2026	2
Sprint 2	Παραγωγή Στατιστικών & Εξαγωγή σε PDF	5	17/2/2026	22/2/2026	6
Sprint 2	GUI about	3	22/2/2026	23/2/2026	2

Sprint 2	Τεχνική απαίτηση - Άντληση δεδομένων και αξιοποίησή τους από ΒΔ	5	12/2/2026	16/2/2026	5
Sprint 2	Κριτικός Απολογισμός του Έργου	3	23/2/2026	23/2/2026	1
ΣΥΝΟΛΟ		**39**			**34**

Στο δεύτερο Sprint, η ομάδα βρήκε τον ρυθμό της. Οι εκτιμήσεις (Effort) ταυτίστηκαν σχεδόν απόλυτα με τις πραγματικές ανθρωποημέρες, αποδεικνύοντας ότι η εξοικείωση με τις βιβλιοθήκες και τα υπόλοιπα εργαλεία που χρησιμοποιήθηκαν βελτίωσε την ικανότητα πρόβλεψης. Η μοναδική καθυστέρηση παρατηρήθηκε στην «**Εξαγωγή σε PDF»** (+1 ημέρα), η οποία δικαιολογείται από την πολυπλοκότητα ενσωμάτωσης βιβλιοθηκών (libraries) για τη δημιουργία του αρχείου.

Sprint 3 (23/02/2026 - 04/03/2026) – Έλεγχος, Τεκμηρίωση & Παράδοση

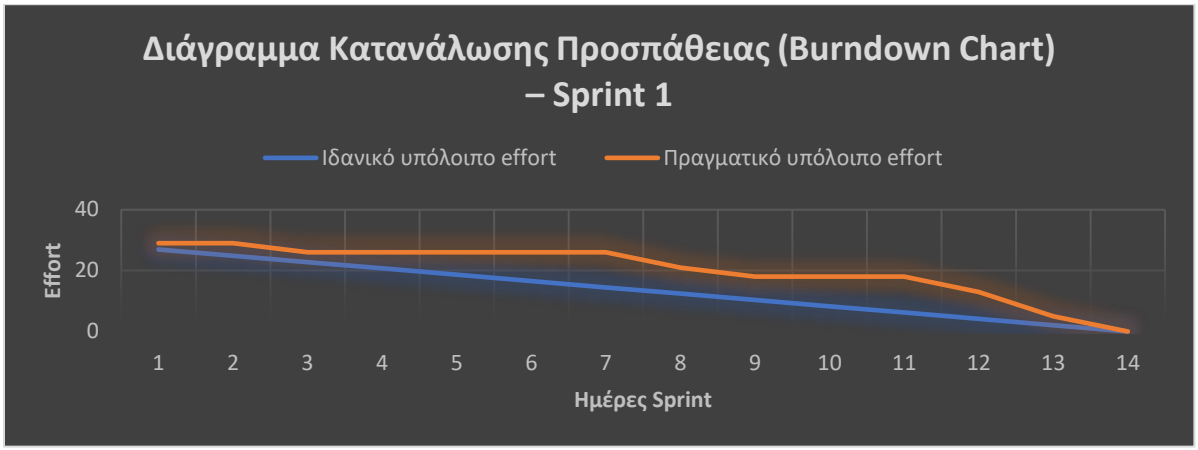
Το τελευταίο Sprint αφιερώθηκε στο testing του συστήματος και στη δημιουργία των τελικών παραδοτέων.

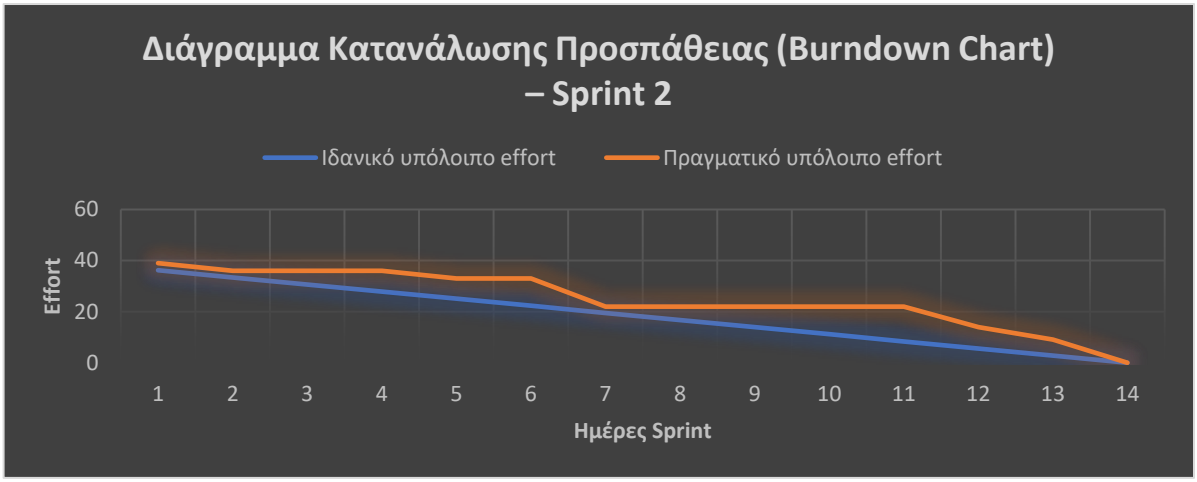
List	Title	Story Points	Start	Due	Πραγματικές ημέρες
Sprint 3	Κριτικός Απολογισμός του Έργου	5	2/3/2026	2/3/2026	3
Sprint 3	Δημιουργία Video Παρουσίασης	8	26/2/2026	28/2/2026	3
Sprint 3	Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής	13	24/2/2026	27/2/2026	4
Sprint 3	Συγγραφή εργασίας	8	28/2/2026	4/3/2026	5
Sprint 3	Παραγωγή Javadoc	3	26/2/2026	28/2/2026	3
ΣΥΝΟΛΟ		**37**			**18**

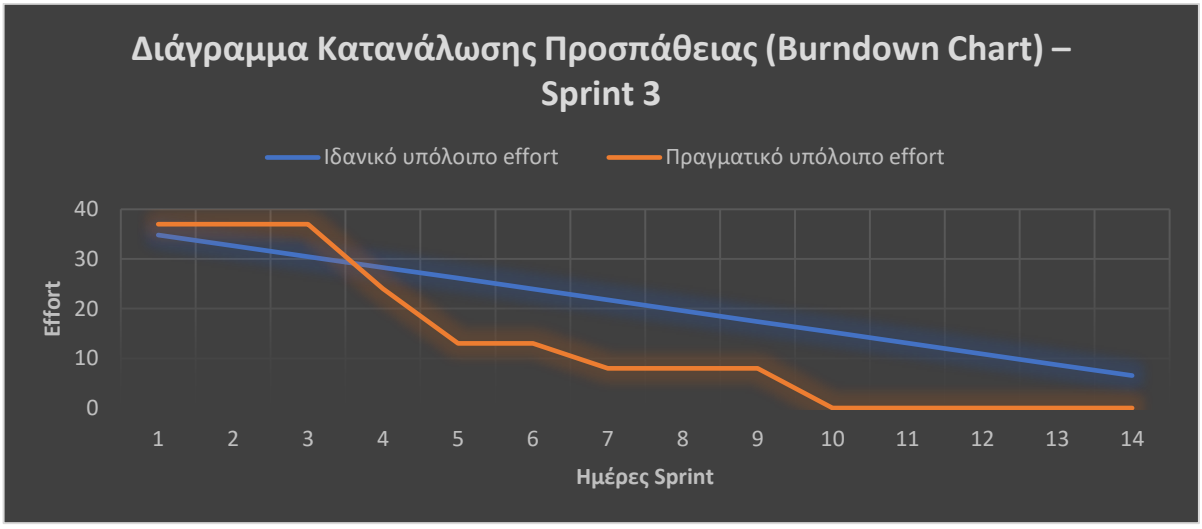
Το τρίτο Sprint χαρακτηρίζεται από μια υπερεκτίμηση του απαιτούμενου χρόνου. Η ομάδα είχε δεσμεύσει 13 μονάδες για τον «**Συνολικό Έλεγχο»** (Testing), ωστόσο ολοκληρώθηκε μόλις σε 4. Αυτό δεν αποτελεί αστοχία, αλλά επιτυχία της μεθοδολογίας. Επειδή η ομάδα έκανε συνεχείς ελέγχους (testing) και διόρθωνε τα σφάλματα (debugging) κατά τα Sprint 1 και 2, το τελικό σύστημα ήταν ήδη εξαιρετικά σταθερό. Επίσης, εργασίες τεκμηρίωσης όπως το «**Video Παρουσίασης**» ολοκληρώθηκαν ταχύτερα χάρη στη χρήση σύγχρονων και εύχρηστων εργαλείων καταγραφής και επεξεργασίας. Ο τελικός Κριτικός Απολογισμός του Sprint 3 διήρκεσε περισσότερο από τους προηγούμενους, καθώς δεν αφορούσε μόνο την τελευταία επανάληψη, αλλά αποτέλεσε μια συνολική ανασκόπηση (Project Retrospective) ολόκληρου του έργου, των πρακτικών μας και της συνεργασίας μας.

Διαγράμματα Κατανάλωσης Προσπάθειας (Burndown Charts)

Για την οπτική παρακολούθηση της προόδου, δημιουργήσαμε Burndown Charts για κάθε επανάληψη. Ο κατακόρυφος άξονας αναπαριστά την υπολειπόμενη προσπάθεια σε Story Points, ενώ ο οριζόντιος τον χρόνο (ημέρες). Όπως φαίνεται στα διαγράμματα, η πραγματική γραμμή προσπάθειας ακολούθησε σε ικανοποιητικό βαθμό την ιδεατή γραμμή, επιβεβαιώνοντας την ορθότητα των αρχικών μας εκτιμήσεων.


Διάγραμμα Κατανάλωσης Προσπάθειας (Burndown Chart) – Sprint 1
Ιδανικό υπόλοιπο effort
Πραγματικό υπόλοιπο effort
Effort
40
20
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14
Ημέρες Sprint


Διάγραμμα Κατανάλωσης Προσπάθειας (Burndown Chart) – Sprint 2
Ιδανικό υπόλοιπο effort
Πραγματικό υπόλοιπο effort
Effort
60
40
20
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14
Ημέρες Sprint

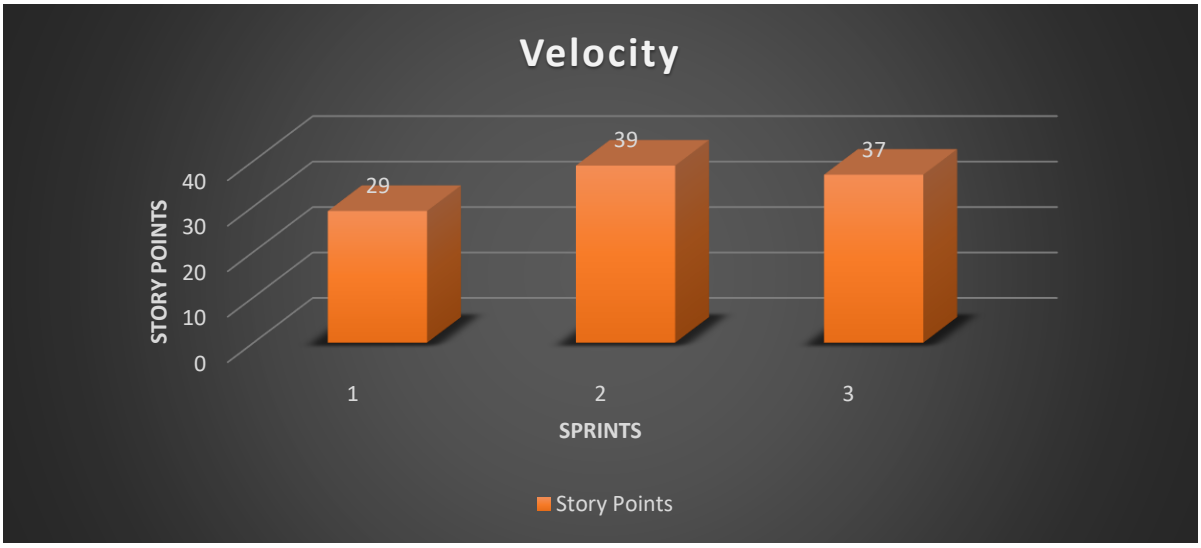

Διάγραμμα Κατανάλωσης Προσπάθειας (Burndown Chart) – Sprint 3
Ιδανικό υπόλοιπο effort
Πραγματικό υπόλοιπο effort
Effort
40
30
20
10
0
1 2 3 4 5 6 7 8 9 10 11 12 13 14
Ημέρες Sprint

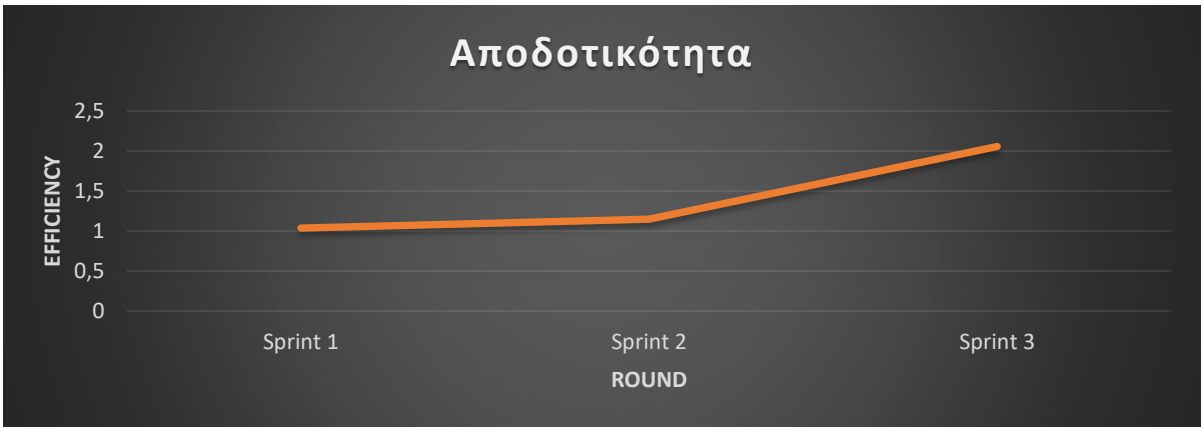
Ταχύτητα Υλοποίησης (Velocity)

Η ταχύτητα της ομάδας υπολογίστηκε προσθέτοντας τα Story Points των ολοκληρωμένων απαιτήσεων σε κάθε Sprint. Η διακύμανση της ταχύτητας παρουσιάζει την εξής εικόνα:

- Sprint 1 Velocity: 29 SP
- Sprint 2 Velocity: 39 SP
- Sprint 3 Velocity: 37 SP

Η μέση ταχύτητα (Average Velocity) της ομάδας διαμορφώθηκε στα 35 Story Points ανά Sprint. Η χαμηλότερη ταχύτητα στο Sprint 1 δικαιολογείται από τον αρχικό χρόνο που απαιτήθηκε για το στήσιμο του περιβάλλοντος και την εξοικείωση της ομάδας με τα νέα εργαλεία. Στα Sprint 2 και 3, η ομάδα «ανέβασε ρυθμό», αυξάνοντας αισθητά την παραγωγικότητά της.





Όπως αποτυπώνεται στο Διάγραμμα Αποδοτικότητας, η ομάδα αύξησε κατακόρυφα την ταχύτητα παραγωγής της (Story Points ανά ανθρωποημέρα) κατά το τελευταίο Sprint. Αυτό αποδεικνύει την επιτυχή ενσωμάτωση των πρακτικών Agile, καθώς η εξοικείωση με τον κώδικα και τα εργαλεία ελαχιστοποίησε τους νεκρούς χρόνους.

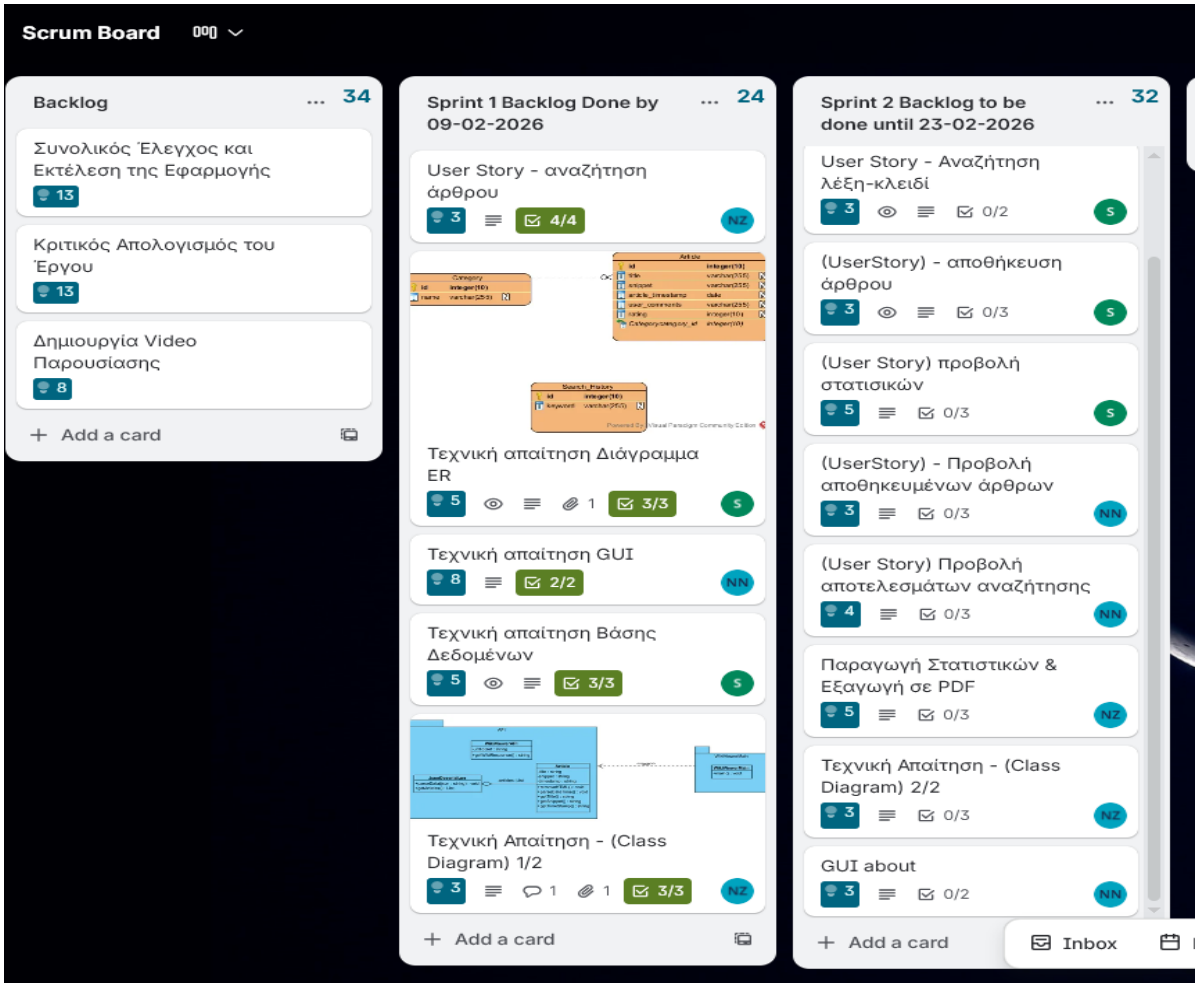
2.6 Χρήση εργαλείου trello

1. Οργάνωση του Agile Board και Λίστες

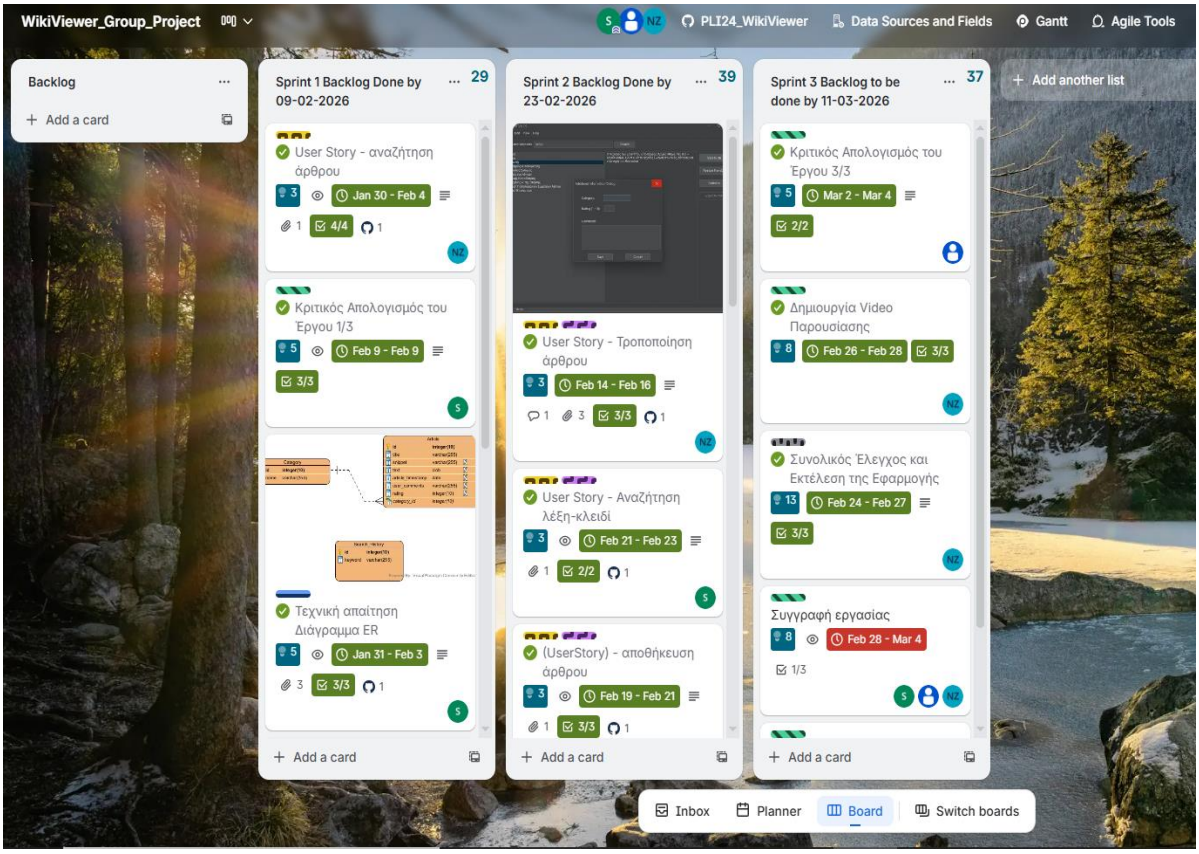
Για την οπτικοποίηση της ροής εργασίας και του προγραμματισμού των παραδοτέων, οργανώσαμε τον πίνακά μας (Agile Board) με βάση τις επαναλήψεις (Sprints). Συγκεκριμένα, δημιουργήσαμε τις παρακάτω λίστες:

- Product Backlog: Η αρχική «δεξαμενή» όπου συγκεντρώθηκαν όλες οι απαιτήσεις του έργου πριν την έναρξη της υλοποίησης.

- Sprint 1, Sprint 2, Sprint 3: Τρεις ξεχωριστές λίστες, μία για κάθε επανάληψη. Μέσα σε αυτές μεταφέρονταν οι κάρτες που είχαν επιλεγεί κατά το Sprint Planning. Η παρακολούθηση της κατάστασης κάθε εργασίας (αν δηλαδή είναι σε αναμονή, υπό επεξεργασία ή ολοκληρωμένη) γινόταν εσωτερικά στην κάθε κάρτα, αξιοποιώντας τα Checklists (Κριτήρια Αποδοχής), τις Ημερομηνίες Λήξης (Due Dates) που πρασίνιζαν όταν ολοκληρώνονταν, και τα αντίστοιχα χρωματικά Labels.



Εικόνα 1: Στιγμιότυπο του Trello Board κατά την έναρξη του 2ου Sprint, όπου διακρίνεται η μεταφορά καρτών από το Backlog.

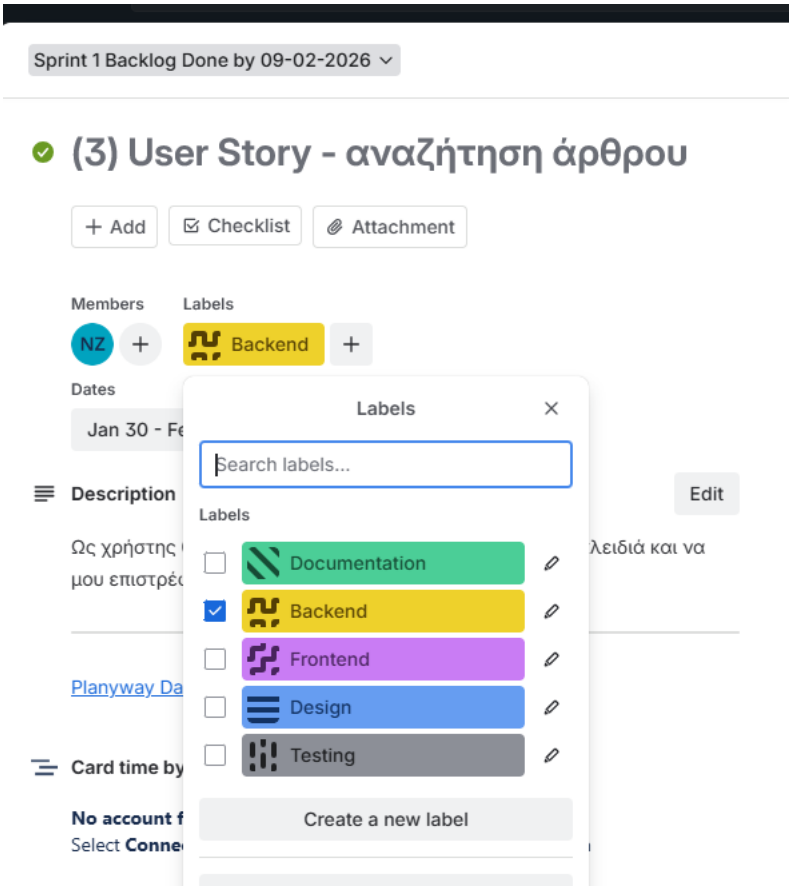


Εικόνα 2: Η τελική μορφή του Trello Board με την ολοκλήρωση του έργου. Το Backlog έχει αδειάσει και όλες οι απαιτήσεις έχουν υλοποιηθεί στα αντίστοιχα Sprints (εκτός από τη συγγραφή της εργασίας που είναι σε εξέλιξη).

2. Ορισμός Κατηγοριών Δραστηριοτήτων

Για τον γρήγορο οπτικό διαχωρισμό των εργασιών, αξιοποιήσαμε τη λειτουργία των χρωματικών ετικετών (Labels) του Trello. Ορίσαμε τις εξής βασικές κατηγορίες:

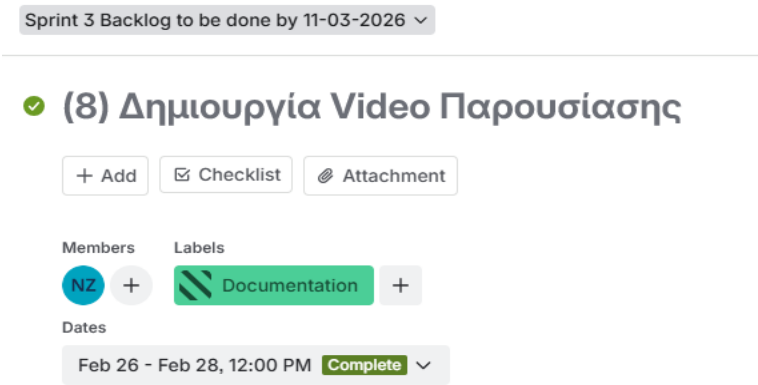
- Backend (Κίτρινο): Εργασίες για τη βάση δεδομένων Derby, το API της Wikipedia και τη λογική της εφαρμογής.
- Frontend (Μωβ): Σχεδιασμός διεπαφής χρήστη (GUI).
- Design (Μπλε): Δημιουργία διαγραμμάτων (ER, Class Diagrams).
- Documentation (Πράσινο): Κριτικός απολογισμός, συγγραφή εργασίας και παρουσίαση.
- Testing (Γκρι): Έλεγχος και διορθώσεις σφαλμάτων κατά την εκτέλεση της εφαρμογής.



3. Καταγραφή Χρόνου Δραστηριοτήτων

Αναφορικά με τον χρόνο (Ημερομηνίες), η ομάδα εφάρμοσε μια εξελικτική προσέγγιση. Κατά τη διάρκεια του Sprint 1 και επειδή δεν είχαμε γνώση όλων των δυνατοτήτων του Trello board, η καταγραφή ημερών γινόταν στο Excel έπειτα από επικοινωνία (Google meets και Viber messages).

Ωστόσο, κατά τον Κριτικό Απολογισμό του 1ου Sprint, η ομάδα διαπίστωσε την ανάγκη για λεπτομερέστερη οπτικοποίηση του χρονοδιαγράμματος (Gantt) καλύτερο συντονισμό αλλά και εξοικονόμηση χρόνου. Έτσι, από την έναρξη του 2ου Sprint και έπειτα, αποφασίστηκε να ενεργοποιηθούν και να χρησιμοποιούνται συστηματικά τα πεδία Start Date και Due Date εντός του Trello για κάθε κάρτα. Η αλλαγή αυτή ήταν το κλειδί για τη δημιουργία αυτοματοποιημένων χρονοδιαγραμμάτων μέσω των Power-Ups στην τελική φάση της τεκμηρίωσης.



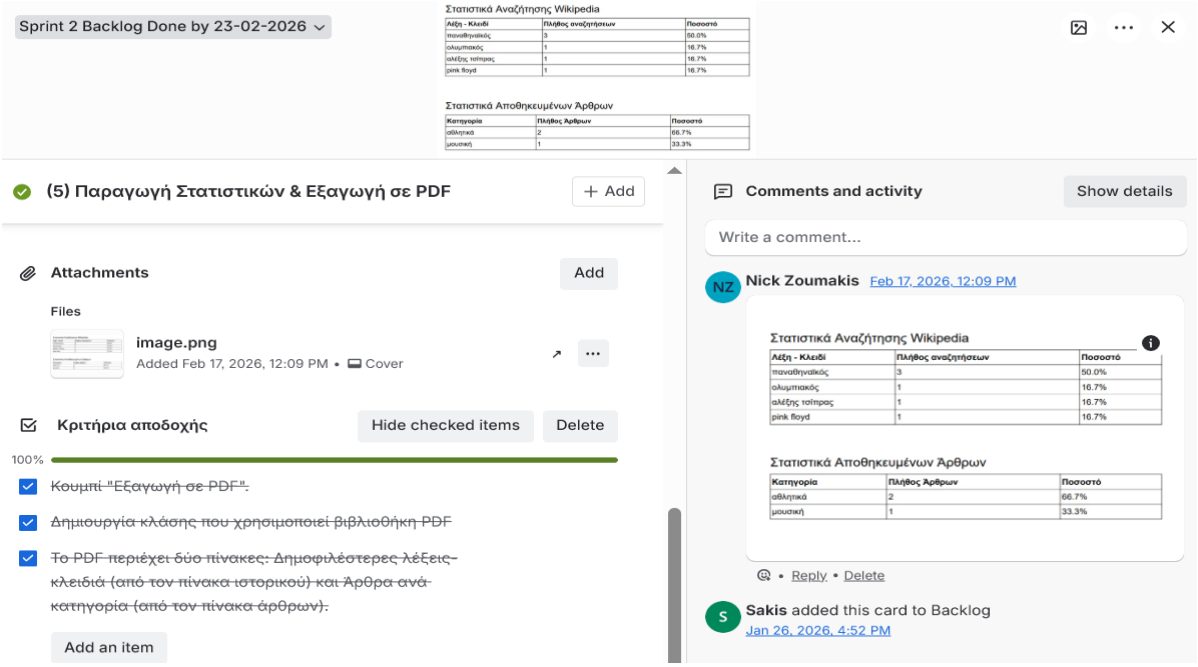
4. Συνεργασία Ομάδας στο Agile Board

Ο πίνακας αποτέλεσε το κεντρικό σημείο αναφοράς για την ομάδα. Η συνεργασία επιτεύχθηκε μέσω:

- Αναθέσεων (Members): Κάθε κάρτα είχε ξεκάθαρα το εικονίδιο του μέλους που την είχε αναλάβει, αποφεύγοντας επικαλύψεις.

- Ασύγχρονης Επικοινωνίας: Χρησιμοποιήσαμε τα σχόλια (Comments) αλλά και τις υπόλοιπες διαθέσιμες λειτουργίες μέσα στις κάρτες για να αφήνουμε διευκρινιστικές σημειώσεις ή κάποιο στιγμιότυπο οθόνης όπου χρειαζόταν.

- Λιστών Ελέγχου (Checklists): Τα Κριτήρια Αποδοχής (Acceptance Criteria) κάθε κάρτας γράφτηκαν σε Checklists. Η κάρτα δεν θεωρούνταν ολοκληρωμένη (Mark complete) αν δεν είχαν τσεκαριστεί όλα τα κουτάκια.



5. Αξιολόγηση Λειτουργιών Trello.

- Το γραφικό περιβάλλον drag-and-drop ήταν εξαιρετικά εύχρηστο για τη μετακίνηση των καρτών. Η προσθήκη Checklists και Labels βοήθησε καθοριστικά στην καλύτερη οργάνωση του έργου. Η γενικότερη απεικόνιση (οπτικά εφέ, χρωματισμοί κτλ) δημιουργεί μια φιλική αίσθηση στον χρήστη σε σύγκριση με ένα απλό φύλλο εργασίας.

- Η μεγαλύτερη αδυναμία της δωρεάν έκδοσης του Trello είναι η απουσία εγγενούς εργαλείου για παραγωγή γραφημάτων (όπως Burndown charts) και οπτικοποίησης χρονοδιαγράμματος (Gantt). Αυτό μας ανάγκασε να δημιουργήσουμε κάποια γραφήματα εξωτερικά στο Excel ή να εκμεταλλευτούμε το χρονικό περιθώριο δοκιμαστικής έκδοσης (Free trial) από τα παρεχόμενα εργαλεία.

6. Συνδέσεις με Τρίτες Εφαρμογές (Power-Ups).

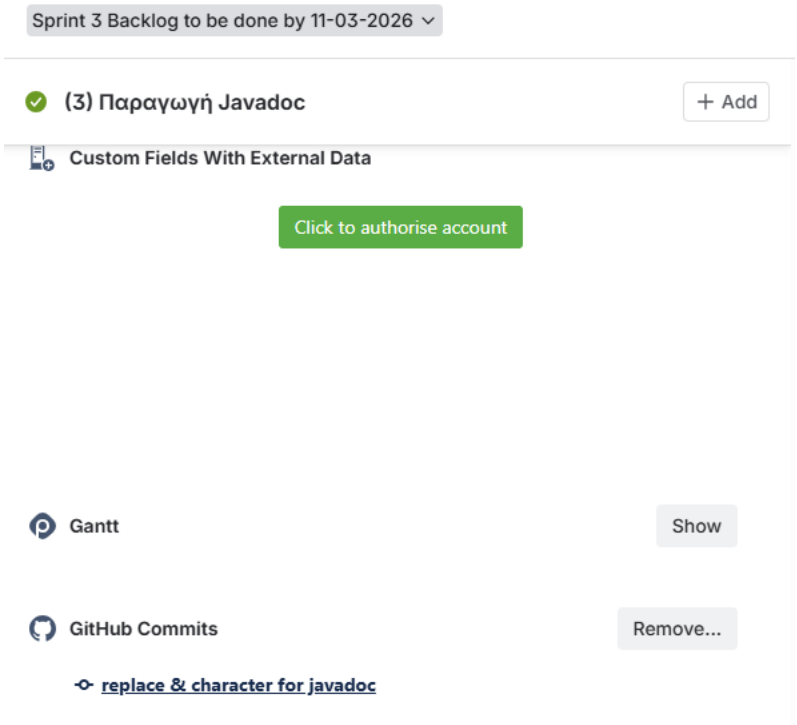
Για να αξιοποιήσουμε περαιτέρω τις δυνατότητες που παρέχονται στην δωρεάν έκδοση αξιοποιήσαμε τα Power-Ups του Trello:

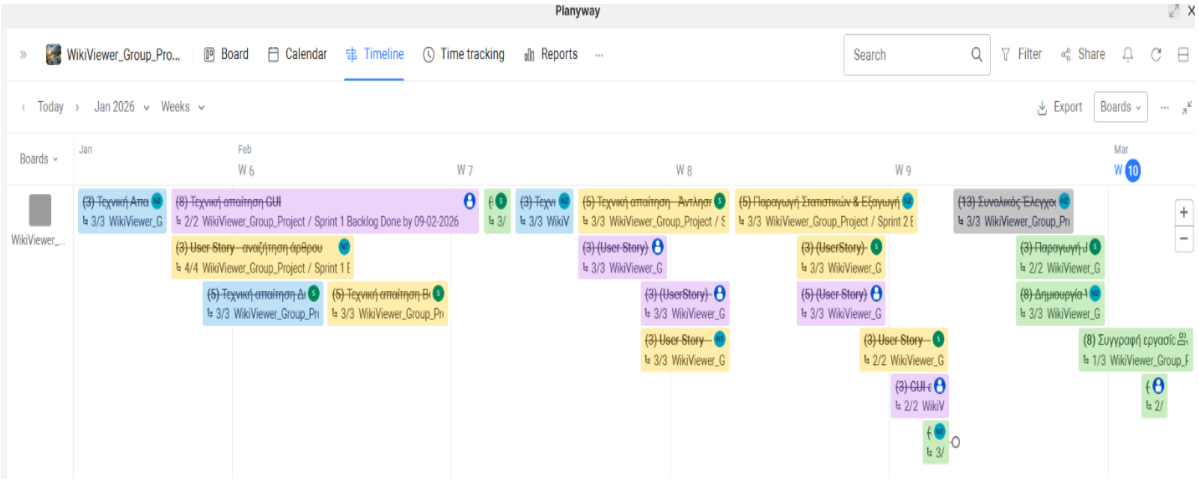
- Scrum for Trello: Η συγκεκριμένη επέκταση περιηγητή (Chrome Extension) αξιοποιήθηκε για την οπτικοποίηση της εκτιμώμενης προσπάθειας (Story Points). Μέσω της επέκτασης αυτής, η εισαγωγή της τιμής σε παρενθέσεις στον τίτλο της κάρτας (π.χ. (5) Δημιουργία Βάσης) μετατρεπόταν αυτόματα σε γραφική ένδειξη (badge) βάρους, ενώ παράλληλα το εργαλείο υπολόγιζε αυτόματα το άθροισμα των Story Points στο πάνω μέρος κάθε λίστας (Sprint), διευκολύνοντας τον έλεγχο της χωρητικότητας (capacity) της ομάδας.

- TrelloExport: Η επέκταση αυτή (Chrome Extension) χρησιμοποιήθηκε για την εξαγωγή των δεδομένων σε φύλλο εργασίας Excel (xlsx).

- GitHub: Συνδέσαμε το Trello board με το αποθετήριο του κώδικά μας στο GitHub. Αυτό μας επέτρεψε να κάνουμε "Attach" συγκεκριμένα Commits μέσα στις κάρτες του Trello. Έτσι, δημιουργήθηκε σύνδεση μεταξύ της απαίτησης της κάρτας του Trello και της τεκμηρίωσης της υλοποίησής της (GitHub Commit).

- Planyway: Χρησιμοποιήσαμε αυτό το Power-Up (στη δωρεάν δοκιμαστική του έκδοση) για να αντλήσουμε τις ημερομηνίες (Start/Due dates) από τις κάρτες μας και να παραγάγουμε αυτόματα το Διάγραμμα Gantt του έργου.


Sprint 3 Backlog to be done by 11-03-2026
(3) Παραγωγή Javadoc
+ Add
Custom Fields With External Data
Click to authorise account
Gantt
Show
GitHub Commits
Remove...
replace & character for javadoc

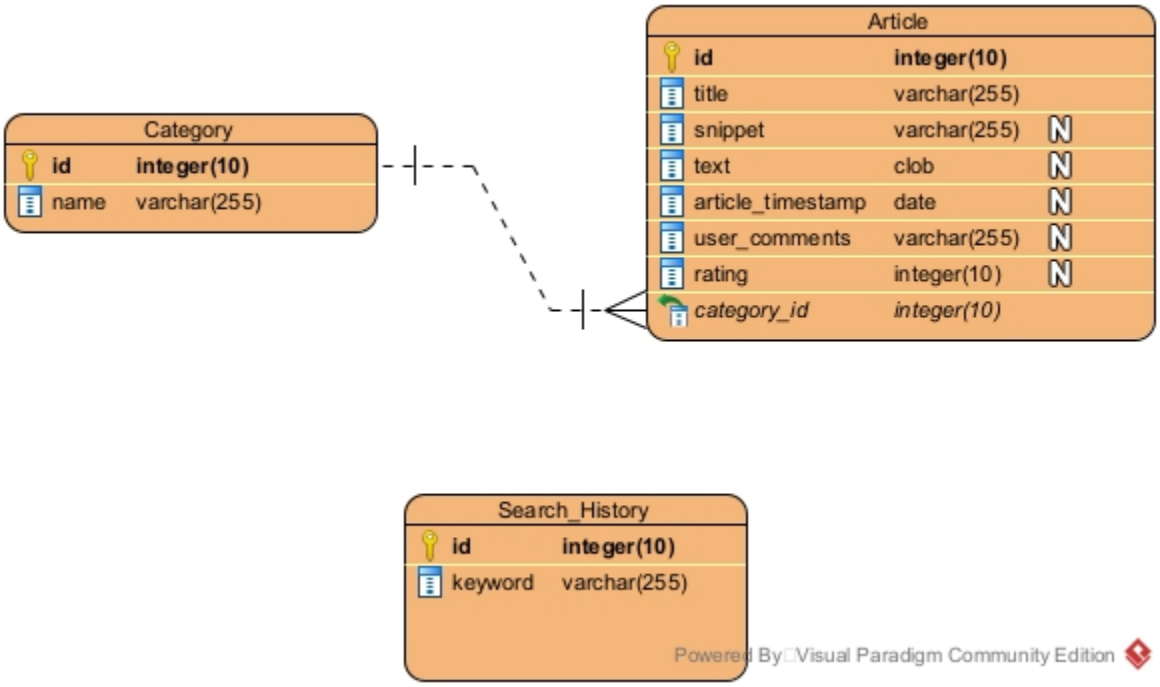

Planyway
WikiViewer_Group_Pro...
Board
Calendar
Timeline
Time tracking
Reports
Search
Filter
Share
Today
Jan 2026
Weeks
Export
Boards
Jan
Feb
Mar
W 6
W 7
W 8
W 9
W 10
(8) Τεχνική απαίτηση GUI
2/2 WikiViewer_Group_Project / Sprint 1 Backlog Done by 09-02-2026
(3) User Story - ανοιξήτηση άρθρου
4/4 WikiViewer_Group_Project / Sprint 1 E
(5) Τεχνική απαίτηση Δι
3/3 WikiViewer_Group_Pr
(5) Τεχνική απαίτηση Βι
3/3 WikiViewer_Group_Pr
(3) Τεχνι
3/3 WikiV
(5) Τεχνική απαίτηση - Αντλησ
3/3 WikiViewer_Group_Project / S
(3) (User Story)
3/3 WikiViewer_G
(3) (UserStory)
3/3 WikiViewer_G
(3) User Story
3/3 WikiViewer_G
(5) Παραγωγή Στατιστικών & Εξαγωγή
3/3 WikiViewer_Group_Project / Sprint 2 E
(3) (UserStory)
3/3 WikiViewer_G
(5) (User Story)
3/3 WikiViewer_G
(3) User Story
2/2 WikiViewer_G
(3) GUI
2/2 WikiV
(13) Συνολικός Έλεγχο
3/3 WikiViewer_Group_Pr
(3) Παραγωγή J
2/2 WikiViewer_G
(8) Δημιουργία
3/3 WikiViewer_G
(8) Συγγραφή εργασίας
1/3 WikiViewer_Group_F

3 ΔΡΑΣΤΗΡΙΟΤΗΤΑ 2 - ΥΛΟΠΟΙΗΣΗ ΕΡΓΟΥ

3.1 Ερώτημα Α– Διάγραμμα Κλάσεων και Υλοποίηση Κλάσεων σε Java

(α)

ER Diagram



Περιγραφή Οντοτήτων και Πεδίων

CATEGORY: Αποθηκεύει τις θεματικές ενότητες που ορίζει ο χρήστης για την οργάνωση των άρθρων.

ID: Μοναδικός κωδικός κατηγορίας (Primary Key).

NAME: Το όνομα της κατηγορίας (π.χ. Επιστήμη).

ARTICLE: Περιλαμβάνει τα δεδομένα των άρθρων που ο χρήστης επέλεξε να αποθηκεύσει από το API, εμπλουτισμένα με δικές του πληροφορίες.

ID: Μοναδικός κωδικός άρθρου (Primary Key).

TITLE, SNIPPET, TIMESTAMP: Βασικές πληροφορίες που ανακτώνται από το Wikipedia API.

TEXT: Το πλήρες κείμενο που ανακτάται από το Wikipedia API.

USER_COMMENTS: Προσωπικές σημειώσεις του χρήστη.

RATING: Αξιολόγηση ενδιαφέροντος από 1 έως 5.

CATEGORY_ID: Ξένο κλειδί (Foreign Key) που συνδέει το άρθρο με μια συγκεκριμένη κατηγορία.

SEARCH_HISTORY: Πίνακας καταγραφής ιστορικού αναζητήσεων.

ID: Μοναδικός κωδικός εγγραφής (Primary Key).

KEYWORD: Η λέξη-κλειδί που πληκτρολόγησε ο χρήστης, ώστε να είναι δυνατή η εξαγωγή στατιστικών συχνότητας.

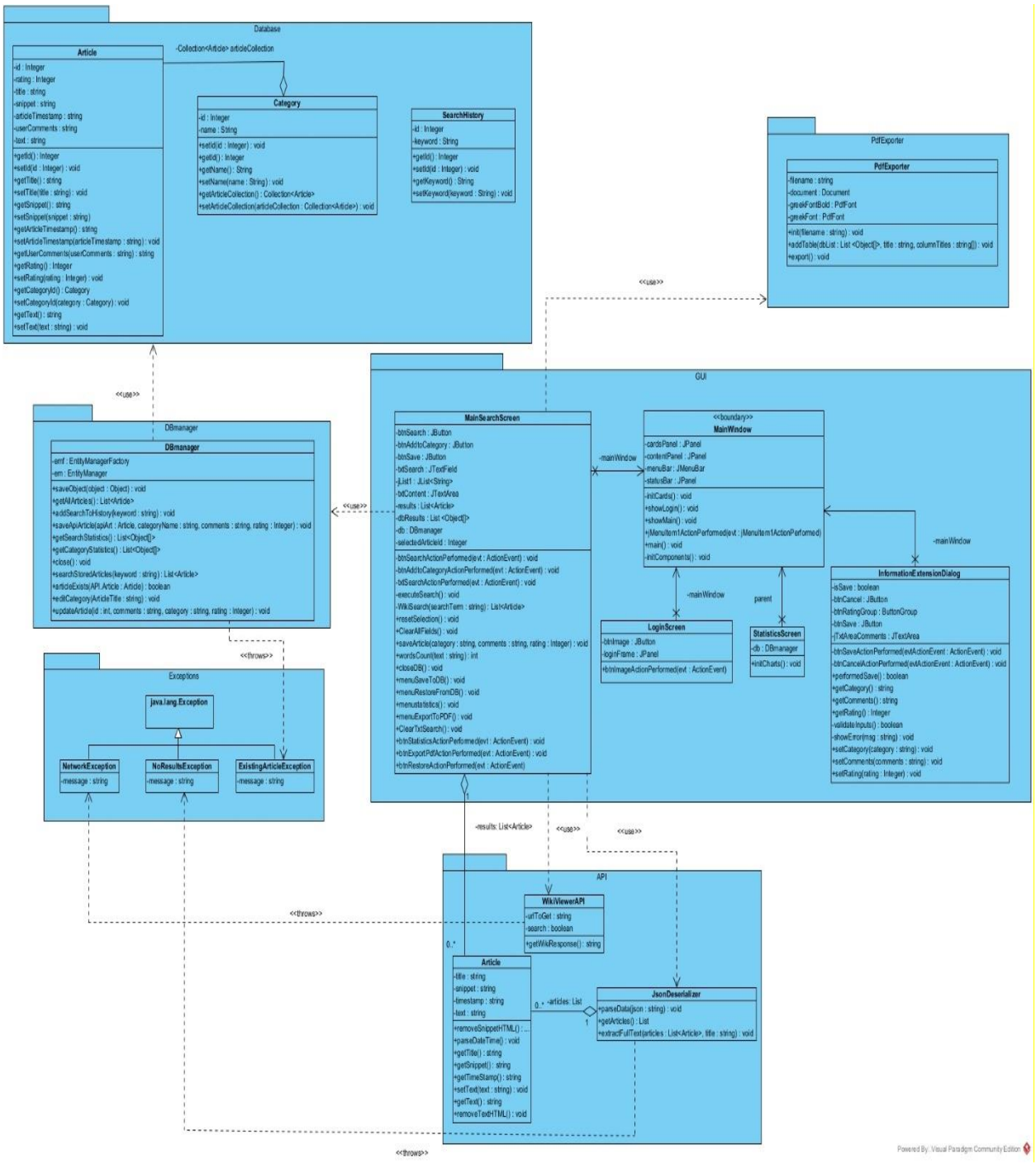
Σχέσεις (Relationships)

CATEGORY - ARTICLE (1:N): Μία κατηγορία μπορεί να περιλαμβάνει πολλά άρθρα, ενώ κάθε άρθρο ανήκει σε ακριβώς μία κατηγορία (σχέση ένα-προς-πολλά).

Παρατίθεται ο κώδικας που υλοποιεί το παραπάνω διάγραμμα (περιλαμβάνεται στο αρχείο "create_db_script.sql")

```
-- Δημιουργία πίνακα Κατηγοριών
CREATE TABLE CATEGORY (
    ID INT NOT NULL GENERATED ALWAYS AS IDENTITY (START WITH 1, INCREMENT BY
1),
    NAME VARCHAR(255) NOT NULL,
    PRIMARY KEY (ID)
);

-- Δημιουργία πίνακα Άρθρων
CREATE TABLE ARTICLE (
    ID INT NOT NULL GENERATED ALWAYS AS IDENTITY (START WITH 1, INCREMENT BY
1),
    TITLE VARCHAR(255) NOT NULL,
    SNIPPET VARCHAR(2000),
    TEXT CLOB(100000),
    ARTICLE_TIMESTAMP VARCHAR(50),
    USER_COMMENTS VARCHAR(1000),
    RATING INT,
    CATEGORY_ID INT,
    PRIMARY KEY (ID),
    CONSTRAINT FK_ARTICLE_CATEGORY FOREIGN KEY (CATEGORY_ID) REFERENCES
CATEGORY(ID)
);

-- Δημιουργία πίνακα για το Ιστορικό Αναζητήσεων
CREATE TABLE SEARCH_HISTORY (
    ID INT NOT NULL GENERATED ALWAYS AS IDENTITY (START WITH 1, INCREMENT BY
1),
    KEYWORD VARCHAR(255) NOT NULL,
    PRIMARY KEY (ID)
);
```

Class Diagram



(β)

Κλάσεις API Package:

Ο κώδικας για τη κλάση Article.java

```
package API;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.joda.time.DateTime;
import org.joda.time.format.DateTimeFormatter;
import org.joda.time.format.DateTimeFormat;
import org.joda.time.format.ISODateTimeFormat;

/**
 * Περιέχει όλα τα πεδία
 * του κάθε array element του search, δηλαδή
 * τα στοιχεία των άρθρων που επέστρεψε η αναζήτηση
 */
public class Article
{
    //πεδία ανα άρθρο στο json (παράδειγμα):
    /*
       {
        "ns": 0,
        "title": "Ελλάδα",
        "pageid": 412,
        "size": 185432,
        "wordcount": 18210,
        "snippet": "Η <span class=\"searchmatch\">Ελλάδα</span> (επίσημα:
Ελληνική Δημοκρατία), είναι χώρα της Νοτιοανατολικής Ευρώπης...",
        "timestamp": "2024-03-15T10:00:00Z"
       }
    */
    /*όπως και στην Articles, δηλώνουμε  μόνο τα πεδία
      που μας ενδιαφέρουν για τον gson builder: title, snippet & timestamp*/
    private String title;
    private String snippet;
    private String timestamp;
    private String text;
    /**
    * Αφαιρεί τα html tags από το κείμενο του snippet
    * και ενημερώνει το πεδίο του class ώστε να επιστρέφεται
    * το καθαρό κείμενο όταν καλειται το getter
    */
    public void removeSnippetHTML()
    {
        Document snip = Jsoup.parse(this.snippet);
        this.snippet = snip.text();
    }

    /**
     * Αφαιρεί HTML tags και μη αναγώσιμα τμήματα
```

```
     * από το πλήρες κείμενο συγκεκριμένου άρθρου
     */
    public void removeTextHTML()
    {
        Document htmlText = Jsoup.parse(this.text);
        htmlText.select(".infobox").remove();
        htmlText.select(".reflist").remove();
        htmlText.select(".thumb").remove();
        htmlText.select(".table").remove();
        htmlText.select(".mw-editsection").remove();
        this.text = htmlText.text();
    }

    /**
     * Μορφοποιεί το timestamp μέσω της jodatime βιβλιοθήκης
     * από ISO 8601 σε dd/MM/yyyy - HH:mm
     */
    public void parseDateTime()
    {
        DateTimeFormatter parser = ISODateTimeFormat.dateTimeParser();

        DateTime dt = parser.parseDateTime(this.timestamp);

        //επιλογή φορματ εμφάνισης του timestamp
        DateTimeFormatter formatter = DateTimeFormat.forPattern("dd/MM/yyyy -
HH:mm");

        this.timestamp = formatter.print(dt);
    }

    /**
     * Setter για τη μεταβλητή text
     * @param text το μη μορφοποιημένο κείμενο με HTML tags
     */
    public void setText(String text)
    {
        this.text = text;
    }

    /*Getter methods για τα πεδία της κλάσης*/
    /**
     * Επιστρέφει τον τίτλο του άρθρου ως String
     * @return title
     */
    public String getTitle()
    {
        return this.title;
    }
    /**
     * Επιστρέφει το snippet που εμφανίζεται το λήμμα αναζήτησης ως String
     * @return snippet
     */
    public String getSnippet()
    {
        return this.snippet;
```

```
    }
    /**
     * Επιστρέφει το μορφοποιημένο timestamp ως String
     * @return timestamp
     */
    public String getTimeStamp()
    {
        return this.timestamp;
    }
    /**
     * Επιστρέφει το πλήρες κείμενο ως String
     * @return text
     */
    public String getText()
    {
        return this.text;
    }
}
```

Ο κώδικας για τη κλάση Articles.java

```
package API;

import java.util.List;
import java.util.ArrayList;

/**
 * Αποτελείται από επιμέρους αντικείμενα
 * Article (με σχέση συναρμολόγησης - aggregation) σε λίστα.
 * Υλοποιεί την αποθήκευση των περιεχομένων του πεδίου search
 */

public class Articles
{
    //περιλαμβάνουμε μόνο το πεδίο search
    //που περιέχει τα αποτελέσματα των άρθρων
    //(τα υπόλοιπα θα αγνοηθούν από το gson builder)

    //δομή πεδίου search στο json:
    /*"search":[
                {<πεδία ανα άρθρο(ns, title, pageid, etc>},
                {...},
                ...
                ]
    -> αποθηκεύεται σε τύπο List με Article objects*/

    private List <Article> search = new ArrayList <>();

    /**
     * Επιστρέφει τη λίστα articles με πεδία όπως υπάρχουν ακριβώς στο
response json
     * @return List Articles με μη διαμορφωμένα πεδία
     */
    public List _getArticles()
    {
        return this.search;
    }
}
```

Ο κώδικας για τη κλάση WikiViewerAPI.java

```
package API;

import java.io.IOException;
import Exceptions.NetworkException;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.Response;

/**
 * Διαμορφώνει το request url για το Wikipedia API
 * και λαμβάνει την απάντηση json σε μορφή String
 */
public class WikiViewerAPI
{
    private String urlToGet;
    //μεταβλητή που καθορίζει τη λειτουργία του class, true για αναζήτηση
άρθρου,
    //false για ανάκτηση πλήρους κειμένου συγκεκριμένου άρθρου
    private boolean search;
    /**
     * Constructor class με ορίσματα
     * @param term ο όρος αναζήτησης του χρήστη ή τίτλος άρθρου για πλήρες
κείμενο
     * @param mode καθορίζει αν ζητείται json αποτελεσμάτων αναζήτησης ή
πλήρους κειμένου
     * @throws IOException εάν ληφθεί κενός (null) όρος
     */
    public WikiViewerAPI(String term, boolean mode) throws IOException
    {
        this.search = mode;
        if (term.isEmpty())
        {
            throw new IOException("Empty input string");
        }
        if (this.search)
        {
            this.urlToGet =

"https://el.wikipedia.org/w/api.php?action=query&list=search&srsearch="
                + term +
                "&format=json";
        }
        else
        {
            this.urlToGet =

"https://el.wikipedia.org/w/api.php?action=parse&formatversion=2&page="
                    + term
                    +"&prop=text&format=json";
        }

    }
```

```
    /**
     * Στέλνει αίτημα στο Wikipedia API και λαμβάνει response
     * @return response body ως String
     * @throws NetworkException εάν το αίτημα δεν είναι επιτυχές
     */
    public String getWikiResponse() throws NetworkException
    {
        OkHttpClient client = new OkHttpClient();

        // build request με user agent header ώστε να γίνει αποδεκτό από το
API
        Request request = new Request.Builder()
            .url(this.urlToGet)
            .header("User-Agent", "MyJavaWikiApp/1.0
(nickzoumakis@gmail.com)")
            .build();
        // επιστροφή της απάντησης με έλεγχο σφάλματος σε κάθε επίπεδο
        try(Response response = client.newCall(request).execute())
        {
            if (!response.isSuccessful())
            {
                throw new NetworkException("No response from API");
            }
            return response.body().string();
        }
        catch (IOException e)
        {
            throw new NetworkException("Network Error: " + e.getMessage());
        }
    }
}
```

Ο κώδικας για τη κλάση jsonDeserializer.java

```
package API;

import Exceptions.NetworkException;
import Exceptions.NoResultsException;
import java.io.IOException;
import com.google.gson.JsonParseException;
import java.util.List;
import com.google.gson.GsonBuilder;
import com.google.gson.Gson;
import com.google.gson.JsonParser;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;

/**
 * Μεταφέρει τα δεδομένα από το response json στα πεδία αντικειμένων Article
 * και τα διαμορφώνει. Ανακτά το πλήρες κείμενο κάποιου άρθρου
 */
public class jsonDeserializer
{
    //λίστα που θα φιλοξενήσει τα τελικά διαμορφωμένα article objects
    private List <Article>  results;

    /**
     * Constructor class
     * @param json η απάντηση του Wikipedia API σε string
     * @throws NoResultsException όταν δεν υπάρχουν αποτελέσματα αναζήτησης
     * @throws JsonParseException σε περίπτωση malformed Json
     */
    public void parseData(String json) throws NoResultsException,
JsonParseException, IOException, NetworkException
    {
        //χρήση JsonParser για απομόνωση του πεδίου "query" από το json
        JsonElement jsonTree = JsonParser.parseString(json);
        JsonObject jsonObject = jsonTree.getAsJsonObject();
        JsonElement queryResults = jsonObject.get("query");

       /*
        * αρχικοποίηση gson builder και παράμετροι:
        * method disableHtmlEscaping επιστρέφει το string χωρίς
        * unicode χαρακτήρες στα HTML tags->έτοιμο για επεξεργασία
        * από Jsoup
        */
        GsonBuilder builder = new GsonBuilder().disableHtmlEscaping();
        Gson gson = builder.create();

        //δημιουργία Java object Articles(ArrayList από Article objects) από
json
        Articles articles = gson.fromJson(queryResults, Articles.class);

        //κλήση της λίστας μη διαμορφωμένων Articles
        this.results = articles._getArticles();
        //έλεγχος αν δεν υπάρχουν αποτελέσματα ( η λίστα είναι άδεια)
        if (this.results == null | this.results.isEmpty())
```

```
        {
            throw new NoResultsException("No search results");
        }
        //loop διαμόρφωσης των πεδίων ανα άρθρο
        for (Article a: this.results)
        {
            //διαμόρφωση πεδίων σε φιλική προς ανάγνωση μορφή*/
            a.removeSnippetHTML();
            a.parseDateTime();
        }
    }

    /**
     * Στέλνει αίτημα για το πλήρες κείμενο του άρθρου article από τη λίστα
με τίτλο title
     * όταν ο χρήστης επιλέξει τον τίτλο από το MainSearchScreen
     * @param articles Λίστα με τα άρθρα που επέστρεψε η αναζήτηση
     * @param title
     * @throws NoResultsException
     * @throws JsonParseException
     * @throws IOException
     * @throws NetworkException
     */
    public void extractFullText(List<Article> articles, String title) throws
NoResultsException, JsonParseException, IOException, NetworkException
    {
        WikiViewerAPI textApi = new WikiViewerAPI(title, false);
        //χρήση JsonParser για απομόνωση του πεδίου "text" από το json
        JsonElement textJsonTree =
JsonParser.parseString(textApi.getWikiResponse());
        JsonObject textJsonObject = textJsonTree.getAsJsonObject();
        JsonElement textResults = textJsonObject.get("parse");
        GsonBuilder textBuilder = new GsonBuilder().disableHtmlEscaping();
        Gson textGson = textBuilder.create();
        //ανάθεση του κειμένου μόνο σε νέο Article object (textArticle)
        // και πίσω στο αρχικό Article object
        Article textArticle = textGson.fromJson(textResults, Article.class);
        for (Article a: articles)
        {
            if (a.getTitle() == title)
            {
                a.setText(textArticle.getText());
                a.removeTextHTML();
            }
        }
    }
    /**
     *
     * @return List Articles με διαμορφωμένα πεδία
     */
    public List getArticles()
    {
        return this.results;
    }
}
```

Κλάσεις DBmanager Package:

Ο κώδικας για τη κλάση DBmanager.java

```
package DBmanager;

import Exceptions.ExistingArticleException;
import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import javax.persistence.Query;
import database.Article;
import database.SearchHistory;

/**
 * Η κλάση DBmanager λειτουργεί ως συνοριακή(boundary class)
 * για την επικοινωνία της εφαρμογής με την ενσωματωμένη βάση δεδομένων.
 */
public class DBmanager {

    // static για να υπάρχει μόνο ένα.
    private static final EntityManagerFactory emf =
Persistence.createEntityManagerFactory("WikiViewerPU");
    private EntityManager em;

    // Constructor για DBManager()
    public DBmanager() {
        em = emf.createEntityManager();
    }

    /**
     * Αποθηκεύει στη βάση (Article, Category, SearchHistory).
     * @param object είναι το αντικείμενο για αποθήκευση
     */
    public void saveObject(Object object) {
        try {
            em.getTransaction().begin();
            em.persist(object);
            em.getTransaction().commit();
        } catch (Exception e) {
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback(); // Ακύρωση αν γίνει λάθος
            }
            e.printStackTrace(); // Εμφάνιση του λάθους στην κονσόλα
        }
    }

    /**
     * Επιστρέφει όλα τα αποθηκευμένα άρθρα.
     * @return Λίστα άρθρων
     */
    public List<Article> getAllArticles() {
        // Επιλέγουμε τα πάντα από την κλάση Article με JPQL query
        Query q = em.createQuery("SELECT a FROM Article a", Article.class);
```

```
        return q.getResultList();
    }

    /**
     * Αναζητά άρθρα στη Βάση Δεδομένων που περιέχουν το keyword
     * @param keyword Η λέξη προς αναζήτηση
     * @return Λίστα με τα άρθρα που βρέθηκαν
     */
    public List<database.Article> searchStoredArticles(String keyword) {
        // Έλεγχος αν το keyword είναι κενό
        if (keyword == null || keyword.trim().isEmpty()) {
            return new java.util.ArrayList<>();
        }

        try {
            // το lower τα κανει όλα μικρα για να μην ειναι case sensitive
            String jpql = "SELECT a FROM Article a WHERE LOWER(a.title) LIKE
:kw OR LOWER(a.snippet) LIKE :kw OR LOWER(a.userComments) LIKE :kw OR
LOWER(a.categoryId.name) LIKE :kw";

            Query q = em.createQuery(jpql, database.Article.class);

            // το % για να βρει τη λέξη οπουδήποτε μέσα στο κείμενο
            String searcheverywhere = "%" + keyword.toLowerCase() + "%";

            q.setParameter("kw", searcheverywhere);

            return q.getResultList();

        } catch (Exception e) {
            e.printStackTrace();
            // Σε περίπτωση σφάλματος επιστρέφουμε άδεια λίστα
            return new java.util.ArrayList<>();
        }
    }
    /**
     * Καταγράφει μια αναζήτηση στο Ιστορικό.
     * @param keyword Η λέξη που αναζητήθηκε
     */
    public void addSearchToHistory(String keyword) {
        SearchHistory sh = new SearchHistory();
        sh.setKeyword(keyword);
        saveObject(sh);
    }

    /**
     * Αποθηκεύει ένα άρθρο μαζί με Κατηγορία, Σχόλια και Βαθμολογία.
     * @param apiArt Το άρθρο που ήρθε από το API
     * @param categoryName Το όνομα της κατηγορίας
     * @param comments Τα σχόλια του χρήστη
     * @param rating Η βαθμολογία του χρήστη
     */
    public void saveApiArticle(API.Article apiArt, String categoryName,
String comments, Integer rating){
        try {
```

```
            em.getTransaction().begin();
            database.Category category;
            // Αναζήτηση αν υπάρχει ήδη η κατηγορία
            try {
                Query q = em.createQuery("SELECT c FROM Category c WHERE
c.name = :name", database.Category.class);
                q.setParameter("name", categoryName);
                category = (database.Category) q.getSingleResult();
            } catch (Exception e) {
                // Αν δεν υπάρχει τη δημιουργούμε
                category = new database.Category();
                category.setName(categoryName);
                em.persist(category);
            }

            // Δημιουργία αντικειμένου άρθρου
            database.Article dbArt = new database.Article();
            dbArt.setTitle(apiArt.getTitle());
            dbArt.setSnippet(apiArt.getSnippet());
            dbArt.setText(apiArt.getText());
            dbArt.setArticleTimestamp(apiArt.getTimeStamp());

            // Προσθήκη των έξτρα στοιχείων
            dbArt.setUserComments(comments);
            dbArt.setRating(rating);
            dbArt.setCategoryId(category);

            em.persist(dbArt);
            // Ολοκλήρωση των αλλαγών
            em.getTransaction().commit();
            System.out.println("Επιτυχής αποθήκευση άρθρου και κατηγορίας!");

        } catch (Exception e) {
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback(); //ακύρωση αλλαγών αν
εμφανιστεί σφάλμα
            }
            e.printStackTrace();// Εμφάνιση του λάθους στην κονσόλα
        }
    }

    /**Ελέγχει την ύπαρξη άρθρου στη database
     * @throws ExistingArticleException αν το άρθρο υπάρχει ήδη στη database
     */
    public boolean articleExists(API.Article apiArt) throws
ExistingArticleException{
        // Έλεγχος αν υπάρχει ήδη το άρθρο στη βάση
        Query checkQuery = em.createQuery("SELECT a FROM Article a WHERE
a.title = :title", database.Article.class);
        checkQuery.setParameter("title", apiArt.getTitle());

        List<database.Article> existingArticles = checkQuery.getResultList();

        if (!existingArticles.isEmpty()) {
```

```
            System.out.println("Το άρθρο '" + apiArt.getTitle() + "' υπάρχει
ήδη στη βάση!");
            //έξοδος απο μεθοδο με exception ώστε να γίνει catch από τη
MainSearchScreen
            throw new ExistingArticleException("Existing Article");
        }
        return false;
    }
    /**
     * Επιστρέφει στατιστικά αναζήτησης.
     * @return λίστα πινάκων [0]=λέξη, [1]=πλήθος, [2]=ποσοστό
     */
    public List<Object[]> getSearchStatistics() {
        Long totalSearches = (Long) em.createQuery("SELECT COUNT(s) FROM
SearchHistory s").getSingleResult();

        if (totalSearches == null || totalSearches == 0) {
            return new java.util.ArrayList<>();
        }

        // Ομαδοποίηση ανά λέξη και καταμέτρηση με δημοφιλέστερα πρώτα (count
DESC)
        String jpql = "SELECT s.keyword, COUNT(s) FROM SearchHistory s GROUP
BY s.keyword ORDER BY COUNT(s) DESC";
        List<Object[]> results = em.createQuery(jpql).getResultList();

        // Υπολογισμος ποσοστού
        List<Object[]> finalResults = new java.util.ArrayList<>();
        for (Object[] row : results) {
            String keyword = (String) row[0];
            Long count = (Long) row[1];
            double percentage = (count.doubleValue() / totalSearches) *
100.0;
            String percentStr = String.format("%.1f%%", percentage);

            // πρόσθεση στοιχείων στη λίστα
            finalResults.add(new Object[]{keyword, count, percentStr});
        }

        return finalResults;
    }

    /**
     * Επιστρέφει το πλήθος των άρθρων ανά κατηγορία.
     * @return λίστα πινάκων με στοιχείο[0] = Κατηγορία [1] = πλήθος άρθρων
και [2]=ποσοστό
     */
    public List<Object[]> getCategoryStatistics() {
        // 1. Βρες το ΣΥΝΟΛΟ των άρθρων
        Long totalArticles = (Long) em.createQuery("SELECT COUNT(a) FROM
Article a").getSingleResult();

        if (totalArticles == null || totalArticles == 0) {
            return new java.util.ArrayList<>();
        }
```

```
        // Ομαδοποίηση ανά κατηγορία και καταμέτρηση με περισσότερα πρώτα
(DESC)
        String jpql = "SELECT c.name, COUNT(a) FROM Article a JOIN
a.categoryId c GROUP BY c.name ORDER BY COUNT(a) DESC";
        List<Object[]> results = em.createQuery(jpql).getResultList();

        // υπολογισμός ποσοστού
        List<Object[]> finalResults = new java.util.ArrayList<>();
        for (Object[] row : results) {
            String catName = (String) row[0];
            Long count = (Long) row[1];
            double percentage = (count.doubleValue() / totalArticles) *
100.0;
            String percentStr = String.format("%.1f%%", percentage);
            // προσθήκη στοιχείνω στην λίστα
            finalResults.add(new Object[]{catName, count, percentStr});
        }

        return finalResults;
    }

    public List<Object[]> getArticleWithCategory(){
        try {
            String txt = "SELECT  a.ID, " +
                         "a.ARTICLE_TIMESTAMP, " +
                         "a.RATING, " +
                         "a.SNIPPET, " +
                         "a.TEXT, " +
                         "a.TITLE, " +
                         "c.NAME AS CATEGORY_NAME, " +
                         "a.USER_COMMENTS " +
                         "FROM ARTICLE a " +
                         "JOIN CATEGORY c ON a.CATEGORY_ID = c.ID";
            Query q = em.createNativeQuery(txt);

            List<Object[]> results = q.getResultList();
            return results;
        }
        catch (Exception e) {
            e.printStackTrace();
            // Σε περίπτωση σφάλματος επιστρέφουμε άδεια λίστα
            return new java.util.ArrayList<>();
        }
    }

    /**
    * Ενημερώνει ένα υπάρχον άρθρο στη βάση δεδομένων.
    * @param id Το μοναδικό ID του άρθρου
    * @param comments Τα νέα σχόλια
    * @param rating Η νέα βαθμολογία
    */
   public void updateArticle(int id, String comments, Integer rating) {
       try {
           em.getTransaction().begin();
```

```
            // Εύρεση του άρθρου από τη βάση μέσω του ID
            database.Article article = em.find(database.Article.class, id);

            if (article != null) {
                article.setUserComments(comments);
                article.setRating(rating);
                //article.setSnippet(txtContent);
                em.getTransaction().commit();
                System.out.println("Επιτυχής ενημέρωση άρθρου ID: " + id);
            } else {
                em.getTransaction().rollback();
            }
        } catch (Exception e) {
            if (em.getTransaction().isActive())
em.getTransaction().rollback();
            e.printStackTrace();
        }
    }

    // Κλείσιμο σύνδεσης με τον τερματισμό
    public void close() {
        em.close();
    }
}
```

Κλάσεις Exceptions Package:

Ο κώδικας για τη κλάση ExistingArticleException.java

```
package Exceptions;

/**
 * Γίνεται throw από τον dbmanager όταν ο χρήστης προσπαθήσει να αποθηκεύσει
 * άρθρο που υπάρχει ήδη στη ΒΔ
 */
public class ExistingArticleException extends Exception
{
    /**
     * Καθορίζει το μήνυμα που εμφανίζεται στην κονσόλα
     * @param message
     */
    public ExistingArticleException(String message)
    {
        super(message);
    }
}
```

Ο κώδικας για τη κλάση NetworkExeption.java

```
//custom exceptions πακέτο
package Exceptions;

/**
 * Γίνεται throw όταν δεν υπάρχει δίκτυο
 * ή δε λαμβάνεται απάντηση από το Wikipedia API
 */
public class NetworkException extends Exception
{
    /**
     * Καθορίζει το μήνυμα που εμφανίζεται στην κονσόλα
     * @param message
     */
    public NetworkException(String message)
    {
        super(message);
    }
}
```

Ο κώδικας για τη κλάση NoResultsException.java

```
package Exceptions;

/**
 * Γίνεται throw όταν δεν έχουν επιστραφεί έγκυρα αποτελέσματα
 * για εμφάνιση
 */
public class NoResultsException extends Exception
{
    /**
     * Καθορίζει το μήνυμα που εμφανίζεται στην κονσόλα
     * @param message
     */
    public NoResultsException(String message)
    {
        super(message);
    }
}
```

(γ)

Οι κλάσεις οντοτήτων (Category, SearchHistory, Article) παρήχθησαν κατά κύριο λόγο αυτόματα μέσω του εργαλείου αυτόματης παραγωγής κώδικα (Entity Classes from Database) του NetBeans. Ως εκ τούτου, ο κώδικας των Getters/Setters, των Constructors, των Named Queries, καθώς και των μεθόδων hashCode(), equals() και toString() παραλείπεται, σύμφωνα με τις οδηγίες της εκφώνησης.

Παρακάτω παρατίθενται αποκλειστικά οι χειροκίνητες παρεμβάσεις και τροποποιήσεις που πραγματοποίησε η ομάδα στην κλάση Article, προκειμένου να καλυφθούν οι τεχνικές απαιτήσεις του έργου.

Κλάση: database.Article

Προστέθηκε ο σχολιασμός (annotation) @Lob στο πεδίο text. Επειδή το κείμενο ενός άρθρου της Wikipedia ξεπερνά το προεπιλεγμένο όριο χαρακτήρων ενός τυπικού πεδίου VARCHAR της βάσης δεδομένων, η χρήση του @Lob δίνει την οδηγία στο JPA να το διαχειριστεί ως CLOB (Character Large Object).

```
@Lob
@Column(name = "TEXT")
private String text;
```

Ο τύπος δεδομένων του πεδίου rating ορίστηκε χειροκίνητα ως αντικείμενο Integer (Wrapper class) αντί για πρωταρχικό τύπο int. Με αυτόν τον τρόπο διασφαλίστηκε ότι κατά την ανάγνωση άρθρων από τη βάση που δεν έχουν αξιολογηθεί ακόμα (null τιμή), η εφαρμογή δεν εγείρει NumberFormatException.

```
@Column(name = "RATING")
private Integer rating;
```

3.2 Ερώτημα Β – Δημιουργία GUI Εφαρμογής

Ο κώδικας για τη κλάση AboutScreen.java

```
package GUI;

/**
 * Παράθυρο πληροφοριών της εφαρμογής WikiViewer.
 * Υλοποιήθηκε στα πλαίσια της 3ης εργασίας της ενότητας ΠΛΗ24 (2025-2026).
 */
public class AboutScreen extends javax.swing.JDialog {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(AboutScreen.class.getName());

    /**
     * Αρχικοποιήση του παραθύρου About.
     * @param parent Το κύριο frame (παράθυρο) της εφαρμογής
     * @param modal true για modal λειτουργία ("κλείδωμα" της εφαρμογής μέχρι
να κλείσει το παράθυρο)
     */
    public AboutScreen(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        // Κεντράρισμα ως προς το γονικό παράθυρο
        // setLocationRelativeTo(parent);
    }

    private void btnCloseActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnCloseActionPerformed
        this.dispose();
    }//GEN-LAST:event_btnCloseActionPerformed
```

Ο κώδικας για τη κλάση InformationExtensionDialog.java

```
package GUI;

import javax.swing.JOptionPane;

/**
 * Ένα Παράθυρο διαλόγου για την επέκταση πληροφοριών ενός άρθρου από τον
χρήστη.
 * Επιτρέπει την προσθήκη κατηγορίας, της βαθμολογίας και των σχολίων του
χρήστη
 * πριν την αποθήκευση.
 */
public class InformationExtensionDialog extends javax.swing.JDialog {
    private boolean isSave;
    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(InformationExtensionDialog.class.getName()
);

    /**
     * Ο κατασκευαστής του παραθύρου για την επέκταση των πληροφοριών
     *
     * @param parent Το κύριο frame (παράθυρο) της εφαρμογής
     * @param modal true για modal λειτουργία ("κλείδωμα" της εφαρμογής μέχρι
να κλείσει το παράθυρο)
     */
    public InformationExtensionDialog(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        this.isSave = false;
        initComponents();
    }

    /**
     * Ελέγχει αν πατήθηκε το κουμπί της αποθήκευσης στη βάση των δεδομένων.
     *
     * @return true αν επιβεβαιώθηκε η αποθήκευση, αλλιώς false.
     */
    public boolean performedSave()    {
        return this.isSave;
    }

    /**
     * Ελέγχει αν δεδομένα εισόδου (κατηγορίς και σχολίων) είναι σύμφωνα
     * με τους περιορισμούς της βάσης δεδομένων.
     *
     * @return true αν τα δεδομένα είναι έγκυρα.
     */
    private boolean validateInputs() {
        // Έλεγχος για το μήκος χαρακτήρων της κατηγορίας
        if (txtFieldCategory.getText().length() > 255)
        {
            showError("Category must be less than 255 characters long");
            return false;
        }
```

```
        // Έλεγχος για το μήκος χαρακτήρων των σχολίων
        if (jTxtAreaComments.getText().length() > 2000)
        {
            showError("Comments must be less than 2000 characters long");
            return false;
        }

        return true;
    }

    private void showError(String msg) {
        JOptionPane.showMessageDialog(this, msg, "Validation Error",
JOptionPane.ERROR_MESSAGE);
    }

    /**
     * Επιστρέφει την κατηγορία του άρθρου.
     * Μετατρέπει τη κατηγορία σε πεζά γράμματα και αφαιρεί τους "κενούς"
χαρακτήρες
     *
     * @return String Τη κατηγορία ή "χωρίς κατηγορία"
     */
    public String getCategory(){
        //έλεγχος του μήκους (varchar 255)
        String category =
this.txtFieldCategory.getText().toLowerCase().strip();
        if (!category.isEmpty())
        {
            return category;
        }
        return "χωρίς κατηγορία";

    }
    /**
     * Λαμβάνει τη βαθμολογία του χρήστη (0-5).
     *
     * @return Integer Τη βαθμολογία του χρήστη ή null
     */
    public Integer getRating() {
        javax.swing.JRadioButton[] buttons = {
            jRadioBtn0, jRadioBtn1, jRadioBtn2, jRadioBtn3, jRadioBtn4,
jRadioBtn5
        };

        for (int i = 0; i < buttons.length; i++) {
            if (buttons[i].isSelected()) {
                return i;
            }
        }
        return null;
    }

    /**
     * Επιστρέφει τα σχόλια του χρήστη ως String
     *
```

```
     * @return String Τα σχόλια του χρήστη
     */
    public String getComments() {
        //έλεγχος του μήκους (varchar 2000)
        String comments = this.jTxtAreaComments.getText();
        return comments;
    }

    /**
     * Ορίζει την κατηγορία που προστεθεί στο άρθρο προς αποθήκευση
     *
     * @param category Η κατηγορία
     */
    public void setCategory(String category) {
        if (category != null) {
            this.txtFieldCategory.setText(category);
        }
    }

    /**
     * Ορίζει το πεδίο των σχολίων για το άρθρο προς αποθήκευση
     *
     * @param comments Το κείμενο των σχολίων
     */
    public void setComments(String comments) {
        if (comments != null) {
            this.jTxtAreaComments.setText(comments);
        }
    }

    /**
     * Ορίζει τη βαθμολογία του χρήστη για το άρθρο προς αποθήκευση
     *
     * @param rating Ακέραιος αριθμός από 0 έως 5
     */
    public void setRating(Integer rating) {
        if (rating == null) {
            return;
        }
        // Επιλογή του σωστού RadioButton βάσει της βαθμολογίας
        switch (rating) {
            case 0: jRadioBtn0.setSelected(true); break;
            case 1: jRadioBtn1.setSelected(true); break;
            case 2: jRadioBtn2.setSelected(true); break;
            case 3: jRadioBtn3.setSelected(true); break;
            case 4: jRadioBtn4.setSelected(true); break;
            case 5: jRadioBtn5.setSelected(true); break;
            default: break;
        }
    }

    /**
     * Χειρισμός event όπου πατιέται το κουμπί Save
     * @param evt
     */
```

```
    private void btnSaveActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnSaveActionPerformed
        // ενημέρωση της μεταβλητής αποθήκευσης
        this.isSave = true;
        //έλεγχος των πεδίων
        if (!validateInputs())
        {
            return; // παύση ροής εκτέλεσης
        }
        //αποδέσμευση παραθύρου από το προσκήνιο
        this.setVisible(false);
    }//GEN-LAST:event_btnSaveActionPerformed

    /**
     * Χειρισμός event όταν πατιέται το κουμπί cancel
     * @param evt
     */
    private void btnCancelActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnCancelActionPerformed
        this.isSave = false;
        //αποδέσμευση παραθύρου από το προσκήνιο
        this.setVisible(false);
    }//GEN-LAST:event_btnCancelActionPerformed
}
```

Ο κώδικας για τη κλάση LoginScreen.java

```
package GUI;

import java.net.URL;
import javax.swing.ImageIcon;

/**
 * Η κλάση αποτελεί την αρχική οθόνη για την είσοδο στην εφαρμογή.
 * Ο χρήστης θα πρέπει να κάνει κλικ στην κεντρική εικόνα για να συνδεθεί
 * στο κύριο περιβάλλον
 */
public class LoginScreen extends javax.swing.JPanel {

    /** Αναφορά στο κύριο παράθυρο για τη διαχείριση των εναλλαγών οθόνης */
    private final MainWindow mainWindow;

    /**
     * Ο κατασκευαστής της LoginscreenLoginScreen
     * Αρχικοποιεί τα elements για το γραγικό περιβάλλον και φορτώνει την
εικόνα.
     *
     * @param main Το αντικείμενο mainWindow.
     */
    public LoginScreen(MainWindow main) {
        this.mainWindow = main;
        initComponents();

        loadLoginImage();

        mainWindow.setStatusMessage("Press the Image to login");
    }

    /**
     * Φορτώνει την εικόνα για το Login σε ένα "κουμπί".
     * Σε περίπτωση που δεν θα μπορεί να εμφανιστεί η εικόνα, θα υπάρχει
     * κατάλληλο μήνυμα κειμένου στο κουμπί.
     */
    private void loadLoginImage(){
        try {
            URL imgURL = getClass().getResource("/images/Login.png");
            if (imgURL != null) {
                btnImage.setIcon(new ImageIcon(imgURL));
                btnImage.setText("");
            }
            else {
                btnImage.setText("The image was not found. Please click here
to continue");
            }
        }
        catch (Exception e){
            btnImage.setText("Error loading the image.");
        }
    }
```

```
    private void btnImageActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnImageActionPerformed
        if (mainWindow != null) {
            mainWindow.showMain("Guest");
        }
    }//GEN-LAST:event_btnImageActionPerformed


    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnImage;
    private javax.swing.JPanel loginFrame;
    // End of variables declaration//GEN-END:variables
}
```

Ο κώδικας για τη κλάση MainSearchScreen.java

```
package GUI;

import API.*;
import Exceptions.*;
import PdfExporter.pdfExporter;
import java.io.IOException;
import java.io.File;
import com.google.gson.JsonParseException;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.DefaultListModel;
import java.awt.datatransfer.StringSelection;
import java.awt.datatransfer.Clipboard;
import java.awt.Toolkit;
import DBmanager.*;

/**
 * Η κλάση MainSearchScreen αποτελεί την κύρια οθόνη για την αναζήτηση
 * και τη διαχείριση άρθρων.
 * Λειτουργίες:
 *     α) Αναζήτηση άρθρων στη Wikipedia μέσω API
 *     β) Προβολή αποτελεσμάτων
 *     γ) Αποθήκευση άρθρων στη βάση δεδομένων
 *     δ) Εξαγωγή στατιστικών σε αρχείο PDF.
 */
public class MainSearchScreen extends javax.swing.JPanel {

    private final MainWindow mainWindow;
    private List <Article> results;     // Αποτελέσματα από το API
    private List <Object[]> dbResults;  // Αποτελέσματα από τη βάση δεδομένων
    private DBmanager db;               // Ο Διαχειριστής της βάσης δεδομένων
    private Integer selectedArticleId;  // Το ID του άρθρου στην βάση
δεδομένων

    /**
     * Ο κατασκευαστής της κλάσης. Αρχικοποιεί τον DBmanager και τις
ρυθμίσεις
     * για το GUI.
     *
     * @param mainWindow Το κεντρικό παράθυρο της εφαρμογής.
     */
    public MainSearchScreen(MainWindow mainWindow) {
        //αρχικοποίηση database manager
        this.db = new DBmanager();
        this.mainWindow = mainWindow;
        initComponents();
        setupTextAreaSplitPane();
        setupListListener();
    }

    /**
     * Ρυθμίσεις για το TextArea και το SplitPane
     */
```

```
    private void setupTextAreaSplitPane(){
        // Ρυθμίσεις του textArea
        txtContent.setLineWrap(true);
        txtContent.setWrapStyleWord(true);
        txtContent.setEditable(true);

        // Προσθήκη για το κεντράρισμα του SplitPane
        javax.swing.SwingUtilities.invokeLater(()-> {
            jSplitPaneMain.setDividerLocation(0.5);
        });
    }

    /**
     * Ρυθμίσεις για τα συμβάντα επιλογής άρθρου από τη λίστα.
     */
    private void setupListListener(){
        jList1.addListSelectionListener(new
javax.swing.event.ListSelectionListener() {
            @Override
            public void valueChanged(javax.swing.event.ListSelectionEvent
evt) {
                if (!evt.getValueIsAdjusting()) {
                    handleListSelection();
                }
            }
        });
    }

    /**
     * Επεξεργάζεται την επιλογή του χρήστη και ενημερώνει
     * τα πεδία προβολής του άρθρου.
     */
    private void handleListSelection() {
        int index = jList1.getSelectedIndex();
        if (index ==-1) return;

        btnSave.setEnabled(true);   //ενεργοποίηση του πλήκτρου αποθήκευσης

        // Α Περίπτωση: Αναζήτηση και προβολή από Search (API)
        if (results != null) {
            if (results.get(index).getText() == null) {
                try {
                    //διαμόρφωση του νέου request για το κείμενο του άρθρου
                    jsonDeserializer deserializer = new jsonDeserializer();
                    deserializer.extractFullText(results,
results.get(index).getTitle());
                    }
                catch (Exception e){
                    System.out.println("Error retrieving the article. " +
e.toString());
                            }
                        }
            updateArticleView(results.get(index).getSnippet(),
                            results.get(index).getText(),
                            results.get(index).getTitle(),
```

```
                                results.get(index).getTimeStamp(),
                                null,
                                null,
                                null);
        }
        // Β Περίπτωση: Αναζήτηση και προβολή από DB
        else if (dbResults !=null){
            Object[] row = dbResults.get(index);
            selectedArticleId = (Integer) row[0];

            String snippet = (String) row[3];
            String fullText = (String) row[4];
            String title = (String) row[5];
            String timestamp = (String) row[1];
            String category = (String) row[6];
            Integer rating = null;
            String comments = (String) row[7];

            if (row[2] != null) {
                rating = (Integer) row[2];
            }

            updateArticleView(snippet,
                                fullText,
                                title,
                                timestamp,
                                category,
                                rating,
                                comments);
        }

        // υπολογισμός των λέξεων του άρθρου
        int plithos = wordsCount(txtContent.getText());
        String txtStatusMessage = "Word count: " + plithos;
        mainWindow.setStatusMessage(txtStatusMessage);
    }

    /**
     * Ενημερώνει τα στοιχεία του GUI με τα δεδομένα του επιλεγμένου άρθρου.
     */
    private void updateArticleView(String snippet, String fullText, String
title, String timestamp,
                                   String category, Integer rating, String
comments) {
        txtContent.setText("Snippet:\n" + snippet + "\n\nFull text:\n" +
fullText);
        txtContent.setCaretPosition(0);
        lblDBTitle.setText(title);
        lblDBDTstamp.setText(timestamp);

        if (category != null){
            lblDBCategory.setText(category);
        }
        else {
            lblDBCategory.setText("");
```

```
        }

        if (rating != null){
            lblDBRating.setText(String.valueOf(rating));
        }
        else {
            lblDBRating.setText("");
        }

        if (comments != null){
            lblDBComments.setText(comments);
        }
        else {
            lblDBComments.setText("");
        }
    }

    /**
     * Μετράει τις λέξεις ενός κειμένου.
     *
     * @param text Το κείμενο προς έλεγχο.
     * @return Το πλήθος των λέξεων.
     */
    public int wordsCount(String text){
        if (text == null || text.trim().isEmpty()){
            return 0;
        }

        String[] words = text.trim().split("\\s+");
        return words.length;
    }

     /**
     * Πραγματοποιεί κλήση της wikiSearch. Χειρισμός των exceptions
     * με κατάλληλο error message.
     * Ενημέρωση της λίστας αποτελεσμάτων.
     */
    private void executeSearch() {
        String searchTerm = this.txtSearch.getText().trim(); //λήψη του
String αναζήτησης που πληκτρολόγησε ο χρήστης
        if (searchTerm.isEmpty()){
            return;
        }

        this.clearAllFields();
        this.txtSearch.setText(searchTerm);

        //κλήση της μεθόδου αναζήτησης wikiSearch
        try {
            //εκχώρηση των αποτελεσμάτων στην results
            this.results = this.wikiSearch(searchTerm, true);
            //γέμισμα του JList με τους τίτλους των αποτελεσμάτων
            DefaultListModel<String> model = new DefaultListModel<>();
            //επαύξηση του model με τους τίτλου, η Jlist ενημερώνεται
αυτόματα
```

```
            for (Article a: results)
            {
                model.addElement(a.getTitle());
            }
            //δυναμική ανάθεση του model στην Jlist
            jList1.setModel(model);

            // Εμφάνιση του πλήθους των αποτελεσμάτων στο status bar
            String txtStatusMessage = "Found " + results.size() + " results";
            mainWindow.setStatusMessage(txtStatusMessage);

            //ενημέρωση της database με τον όρο της αναζήτησης
            if (this.results != null && !this.results.isEmpty())
            {
                this.db.addSearchToHistory(searchTerm.toLowerCase());
            }
        }
        catch (IOException e)
        {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Invalid search term",
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
        catch (NetworkException e)
        {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Network Error",
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
        catch (NoResultsException e)
        {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "No results found",
                    "Information window",
                    JOptionPane.INFORMATION_MESSAGE);
        }
        catch (JsonParseException e)
        {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Data Parsing Error",
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    /**
     * απενεργοποιεί το save button και κάνει εκκαθάριση της επιλογής άρθρου
     */
    public void resetSelection() {
```

```
        this.btnSave.setEnabled(false);
        this.jList1.clearSelection();
    }

    /**
     * Καθαρίζει όλα τα πεδία της οθόνης και επαναφέρει τις αρχικές
ρυθμίσεις.
     */
    public void clearAllFields() {
        this.txtSearch.setText("");
        this.jList1.clearSelection();
        this.jList1.setModel(new DefaultListModel<>());
        this.txtContent.setText("");
        this.lblDBTitle.setText("");
        this.lblDBDTstamp.setText("");
        this.lblDBCategory.setText("");
        this.lblDBRating.setText("");
        this.lblDBComments.setText("");

        this.btnSave.setEnabled(false);
        txtSearch.requestFocus();
    }

    /**
     * Εκτελεί την αναζήτηση μέσω του API
     * @param searchTerm ο όρος αναζήτησης
     * @param mode true για αναζήτηση, false για ανάκτηση από db
     * @throws IOException
     * @throws NetworkException
     * @throws NoResultsException
     * @throws JsonParseException
     * @return Λίστα με τα άρθρα του αποτελέσματος αναζήτησης
     */
    private List <Article> wikiSearch(String searchTerm, boolean mode)throws
IOException, NetworkException, NoResultsException, JsonParseException {
        //δημιουργία του request URL σε mode αναζήτησης
        WikiViewerAPI api = new WikiViewerAPI(searchTerm, mode);
        //λήψη του json response από Wikipedia API
        String jsonResponse = api.getWikiResponse();
        /*deserialize των σχετικών στοιχείων του json response
        σε αντικείμενα Article*/
        jsonDeserializer deserializer = new jsonDeserializer();
        try
        {
            deserializer.parseData(jsonResponse);
        }
        catch (JsonParseException e)
        {
            throw new JsonParseException("Malformed Json");
        }

        List <Article> articlesResults = deserializer.getArticles();
        return articlesResults;
    }
```

```
    /**
     * Αποθηκεύει στη db το επιλεγμένο άρθρο από τη jList μαζί με
     * επιπλέον παραμέτρους που έχει ορίσει ο χρήστης (προαιρετικά)
     * στο InformationExtensionDialog
     * @param category
     * @param comments
     * @param rating
     */
    private void saveArticle(String category, String comments, Integer
rating) {
        int index = jList1.getSelectedIndex();
        if (results != null && index >= 0){
            db.saveApiArticle(this.results.get(index), category, comments,
rating);
        }
    }

    /**
     * Αντιγράφει το κείμενο του txtContent στο πρόχειρο (Clipboard).
     * Αν είναι κενό, εμφανίζει σχετικό μήνυμα στο status bar.
     */
    public void copyContentToClipboard() {
        String textToCopy = txtContent.getText();

        // Έλεγχος αν υπάρχει κείμενο
        if (textToCopy == null || textToCopy.trim().isEmpty()) {
            mainWindow.setStatusMessage("Δεν υπάρχει κείμενο για
αντιγραφή.");
            return;
        }

        // Εστίαση στο txtContent
        txtContent.requestFocusInWindow();
        txtContent.selectAll();

        // Διαδικασία αντιγραφής στο Clipboard της Java
        StringSelection stringSelection = new StringSelection(textToCopy);
        Clipboard clipboard =
Toolkit.getDefaultToolkit().getSystemClipboard();
        clipboard.setContents(stringSelection, null);

        // Εμφάνιση μηνύματος επιτυχίας
        mainWindow.setStatusMessage("Το περιεχόμενο αντιγράφηκε στο
πρόχειρο!");
    }

    /**
     * Κλείνει τη σύνδεση με τη βάση δεδομένων.
     */
    public void closeDB(){
        if (this.db != null) {
            this.db.close();
            System.out.println("Η σύνδεση με τη βάση έκλεισε επιτυχώς.");
        }
    }
```

```
    /**
     * Αποθηκεύει τα δεδομένα στη βάση δεδομένων.
     */
    public void menuSaveToDB(){
        if (this.btnSave.isEnabled()) {
            this.btnSave.doClick(); // Προσομοιώνει το κλικ
        } else {
            // Αν δεν έχει επιλεγεί άρθρο, βγάζει ένα φιλικό μήνυμα
            mainWindow.setStatusMessage("Please choose an article to save.");
        }
    }

    public void menuRestoreFromDB(){
        this.btnRestore.doClick(); // Προσομοιώνει το κλικ
    }

    public void menuStatistics(){
        this.btnStatistics.doClick(); // Προσομοιώνει το κλικ

    }

    public void menuExportToPDF(){
        this.btnExportPdf.doClick(); // Προσομοιώνει το κλικ

    }

    public void clearTxtSearch() {
        txtSearch.setText("");
        txtSearch.requestFocus();
    }

    /**
     * Μέθοδοι που ανταποκρίνονται στην εκτέλεση αναζήτησης
     * είτε με πάτημα του κουμπιού Search είτε με Enter
     * απευθείας στο πεδίο αναζήτησης
     * @param evt
     */
    private void btnSearchActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnSearchActionPerformed
        //κλήση της μεθόδου αναζήτησης executeSearch
        this.executeSearch();
    }//GEN-LAST:event_btnSearchActionPerformed

    private void txtSearchActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_txtSearchActionPerformed
        //κλήση της μεθόδου αναζήτησης executeSearch
        this.executeSearch();
    }//GEN-LAST:event_txtSearchActionPerformed
```

```
    /**
     * Ανοίγει το παράθυρο στατιστικών.
     */
    private void btnStatisticsActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnStatisticsActionPerformed
        // Δημιουργούμε το παράθυρο και του περνάμε τον db manager της κλάσης
μας
        StatisticsScreen stats = new StatisticsScreen(this.mainWindow, true,
this.db);
        stats.setLocationRelativeTo(this); // Κεντράρισμα
        stats.setVisible(true); // Εμφάνιση
    }//GEN-LAST:event_btnStatisticsActionPerformed

    /**
     * Αποθηκεύει ή ενημερώνει ένα άρθρο στη βάση δεδομένων.
     *
     * @param evt
     */
    private void btnSaveActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnSaveActionPerformed
        int index = jList1.getSelectedIndex();
        if (index == -1) return;

        // A Περίπτωση: Νέα αποθήκευση από API
            //έλεγχος αν το επιλεγμένο άρθρο ανήκει ήδη στη λίστα
        if (this.results!=null){
            try {
                if (!(db.articleExists(this.results.get(index)))){
                    //δημιουργία παραθύρου επέκτασης πληροφοριών σε modal
λειτουργία
                    InformationExtensionDialog extensionDialog = new
InformationExtensionDialog(this.mainWindow, true);
                    extensionDialog.setLocationRelativeTo(this);
                    extensionDialog.setVisible(true);

                    //αν ο χρήστης πάτησε το πλήκτρο Save, αποθηκεύονται
                    //το άρθρο και οι πληροφορίες που εισήγαγε στη ΒΔ
                    if (extensionDialog.performedSave()) {
                        this.saveArticle(extensionDialog.getCategory(),
                                extensionDialog.getComments(),
                                extensionDialog.getRating()
                                );
                        mainWindow.setStatusMessage("Article '" +
this.results.get(index).getTitle() + "' saved!");
                    }
                    // αποδέσμευση του παραθύρου από τη μνήμη
                    extensionDialog.dispose();
                }
            }
            catch (ExistingArticleException e) {
                JOptionPane.showMessageDialog(
                        mainWindow,
                        "Article already exists in database",
                        "Information window",
                        JOptionPane.INFORMATION_MESSAGE);
```

```
                }
        }
        // Β Περίπτωση: Ενημέρωση υπάρχοντος άρθρου στη DB
        else if (this.dbResults != null && this.selectedArticleId != null) {
            // Ανοίγουμε τον ίδιο διάλογο για να εισάγει ο χρήστης τις
αλλαγές
            InformationExtensionDialog extensionDialog = new
InformationExtensionDialog(this.mainWindow, true);

            // Γέμισμα του διαλόγου με τα υπάρχοντα δεδομένα
            Object[] row = dbResults.get(index);
            String existingCategory = (String) row[6];
            Integer existingRating = (Integer) row[2];
            String existingComments = (String) row[7];
            //String updContent = this.txtContent.getText();

            extensionDialog.setCategory(existingCategory);
            extensionDialog.setRating(existingRating);
            extensionDialog.setComments(existingComments);


            // Εμφάνιση του παραθύρου του διαλόγου
            extensionDialog.setLocationRelativeTo(this);
            extensionDialog.setVisible(true);

            if (extensionDialog.performedSave()) {
                // Κλήση της νέας μεθόδου Update
                db.updateArticle(this.selectedArticleId,
extensionDialog.getComments(), extensionDialog.getRating());
                mainWindow.setStatusMessage("Changes saved to DB!");

                // Ανανέωση των αποτελεσμάτων με τα νέα δεδομένα
                btnRestore.doClick();
            }
            extensionDialog.dispose();
        }

        //καθαρίζεται η επιλογή από την JList
        //και απενεργοποιείται και πάλι το btnSave μέχρι να επιλεγεί νέο
άρθρο
        this.resetSelection();
    }//GEN-LAST:event_btnSaveActionPerformed

    /**
     * Εξάγει τα στατιστικά της εφαρμογής σε αρχείο PDF.
     *
     * @param evt
     */
    private void btnExportPdfActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnExportPdfActionPerformed
        List<Object[]> searchStats = db.getSearchStatistics();
        List<Object[]> categoryStats = db.getCategoryStatistics();
        // αν η ΒΔ είναι κενή, εμφάνιση σχετικού μηνύματος
        //και επιστροφή στην κανονική ροή προγράμματος
        if (searchStats.isEmpty())
```

```
        {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Database is empty, nothing to export",
                    "EmptyDatabase Error",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        //έλεγχος αν το αρχείο statistics υπάρχει ήδη και απαίτηση
επιβεβαίωσης
        //από το χρήστη για την επανεγγραφή του
        File currentPdf = new File("Statistics.pdf");
        if (currentPdf.exists()) {
            int reply = JOptionPane.showConfirmDialog(
                    null,
                    "Statistics.pdf already exists.\nOverwrite?",
                    "Confirm overwrite",
                    JOptionPane.YES_NO_OPTION);
            if (reply != JOptionPane.YES_OPTION) {
                return;
            }
        }
        //διαδικασία εξαγωγής pdf
        try {
            //αρχικοποίηση του pdfexporter
            pdfExporter pdf = new pdfExporter();
            pdf.init("Statistics");
            String[] searchColumnTitles = {"Λέξη-Κλειδί", "Πλήθος
Αναζητήσεων", "Ποσοστό"};
            //προσθήκη πίνακα στατιστικών αναζήτησης
            pdf.addTable(searchStats, "Στατιστικά Αναζήτησης Wikipedia",
searchColumnTitles);
            //προσθήκη πίνακα στατιστικών κατηγορίας
            //εφόσον υπάρχουν αποθηκευμένα άρθρα
            if (!categoryStats.isEmpty()) {
                String[] catColumnTitles = {"Κατηγορία", "Πλήθος Άρθρων",
"Ποσοστό"};
                pdf.addTable(categoryStats, "Στατιστικά Αποθηκευμένων
Άρθρων", catColumnTitles);
            }
            pdf.export();
            //εμφάνιση μηνύματος επιτυχίας
            System.out.println("Successful PDF export.");
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Pdf export successful, see \"Statistics.pdf\" for
details",
                    "Pdf Export",
                    JOptionPane.INFORMATION_MESSAGE);
        }
        catch (IOException e) {
            JOptionPane.showMessageDialog(
                    mainWindow,
                    "Failed to create pdf file",
```

```
                    e.toString(),
                    JOptionPane.ERROR_MESSAGE);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.printf("%s%n", e.toString());
        }
    }//GEN-LAST:event_btnExportPdfActionPerformed

    /**
     * Ανακτά και εμφανίζει όλα τα αποθηκευμένα άρθρα από τη βάση δεδομένων.
     */
    private void btnRestoreActionPerformed(java.awt.event.ActionEvent evt)
{//GEN-FIRST:event_btnRestoreActionPerformed
        // Clear the fields
        this.clearAllFields();
        this.results = null;
        // Ανάκτηση των δεδομένων από DB
        this.dbResults = this.db.getArticleWithCategory();

        if (this.dbResults.isEmpty()){
            mainWindow.setStatusMessage("Database is empty.");
        }

        //γέμισμα του JList με τους τίτλους των αποτελεσμάτων
        DefaultListModel<String> model = new DefaultListModel<>();

        for (Object[] row : this.dbResults) {
            String title = (String) row[5];
            model.addElement(title);
        }

        //δυναμική ανάθεση του model στην Jlist
        jList1.setModel(model);
        mainWindow.setStatusMessage("Retrieving " + dbResults.size() + "
articles from database.");
    }//GEN-LAST:event_btnRestoreActionPerformed

}
```

Ο κώδικας για τη κλάση MainWindow.java

```
package GUI;

import com.formdev.flatlaf.FlatDarkLaf;
import com.formdev.flatlaf.FlatLightLaf;
import java.awt.CardLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.Timer;


/**
 * Η κύρια οθόνη (Frame) της εφαρμογής WikiViewer.
 * θα κάνει τη διαχείριση στην πλοήγηση μεταξύ των οθονών (Login, Search),
 * το κεντρικό μενού και την εναλλαγή θεμάτων (Dark/Light mode).
 */
public class MainWindow extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(MainWindow.class.getName());
    private Timer statusTimer;  // Timer για την αυτόματη εκκαθάριση
μηνυμάτων στο statusBar

    private static final String CARD_LOGIN = "login";
    private static final String CARD_MAIN  = "main";
    private MainSearchScreen mainSearchScreen;

    /**
     * Ο κατασκευαστής του κεντρικού παραθύρου.
     */
    public MainWindow() {
        // Απενεργοποίηση της ενσωμάτωσης του Menu Bar στον τίτλο
        javax.swing.UIManager.put("TitlePane.menuBarEmbedded", false);

        initComponents();
        this.setLocationRelativeTo(null);   //Κεντράρισμα στην οθόνη

        initCards();
        showLogin();

        // Χειρισμός του κλεισίματος του παραθύρου για ασφαλή τερματισμό της
βάσης
        this.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent windowEvent) {
                handleExit();
            }
        });
    }

    /**
     * Αρχικοποιεί το CardLayout και προσθέτει τις οθόνες (Panels).
```

```
     */
    private void initCards() {
        // Βάζουμε την LoginScreen ως πρώτη κάρτα
        cardsPanel.removeAll();
        cardsPanel.add(new LoginScreen(this), CARD_LOGIN);

        // Προσθήκη κύριας οθόνης αναζήτησης
        this.mainSearchScreen = new MainSearchScreen(this);
        cardsPanel.add(this.mainSearchScreen, CARD_MAIN);

        cardsPanel.revalidate();
        cardsPanel.repaint();
    }

    /**
     * Εμφανίζει την οθόνη σύνδεσης και αποκρύπτει το menubar και statusbar.
     */
    public void showLogin() {
        // Κρύψε menu + status
        menuBar.setVisible(false);
        statusBar.setVisible(false);

        // Δείξε την κάρτα login
        CardLayout cl = (CardLayout) cardsPanel.getLayout();
        cl.show(cardsPanel, CARD_LOGIN);

        revalidate();
        repaint();
    }

    /**
     * Εμφανίζει την κύρια οθόνη αναζήτησης μετά από επιτυχή είσοδο.
     *
     * @param username Το όνομα του χρήστη που συνδέθηκε.
     */
    public void showMain(String username) {
        // Εμφάνισε menu + status
        setJMenuBar(menuBar);
        menuBar.setVisible(true);
        statusBar.setVisible(true);
        StatusLblUsername.setText(username);

        // Προεπιλεγμένη ρύθμιση για το Theme
        ViewThemeDark.setSelected(true);
        ViewThemeLight.setSelected(false);

        // Δείξε την κύρια κάρτα
        CardLayout cl = (CardLayout) cardsPanel.getLayout();
        cl.show(cardsPanel, CARD_MAIN);

        revalidate();
        repaint();
    }

    /**
```

```
     * Αλλάζει το θέμα της εφαρμογής (Dark ή Light).
     *
     * @param isDark true για Dark Mode, false για Light Mode.
     */
    public void changeTheme(boolean isDark){
        try {
            if (isDark) {
                FlatDarkLaf.setup();
                ViewThemeDark.setSelected(true);
                ViewThemeLight.setSelected(false);
            }
            else {
                FlatLightLaf.setup();
                ViewThemeDark.setSelected(false);
                ViewThemeLight.setSelected(true);
            }

            com.formdev.flatlaf.FlatLaf.updateUI();
        }
        catch (Exception ex) {
            logger.log(Level.SEVERE, "Failed to change theme", ex);
        }
    }

    /**
     * Εμφανίζει ένα μήνυμα στο Status Bar το οποίο εξαφανίζεται μετά από 5
δευτερόλεπτα.
     *
     * @param message Το μήνυμα προς εμφάνιση.
     */
    public void setStatusMessage(String message){
        statusGeneralMsg.setText(message);

        // Αν υπάρχει ήδη timer που μετράει, τον σταματάμε
        if (statusTimer != null && statusTimer.isRunning()) {
            statusTimer.stop();
        }

        // Ξεκινάμε νέο χρονόμετρο
        statusTimer = new Timer(5000, e -> statusGeneralMsg.setText(""));
        statusTimer.setRepeats(false); // Θέλουμε να τρέξει μόνο μία φορά
        statusTimer.start();
    }

    /**
     * Διαχειρίζεται την έξοδο από την εφαρμογή, κλείνοντας τη βάση
δεδομένων.
     */
    private void handleExit() {
        if (mainSearchScreen != null) {
            mainSearchScreen.closeDB();
        }
        System.exit(0);
    }
```

```
    /**
     * Κύρια μέθοδος εκκίνησης της εφαρμογής.
     *
     * @param args command line arguments
     */
    public static void main(String args[]) {
        try {
            // Αρχικό Setup του Dark θέματος
            FlatDarkLaf.setup();
        } catch (Exception ex) {
            Logger.getLogger(MainWindow.class.getName()).log(Level.SEVERE,
"FlatLaf Init Failed", ex);
        }

        java.awt.EventQueue.invokeLater(() -> new
MainWindow().setVisible(true));
    }

}
```

Ο κώδικας για τη κλάση StatisticsScreen.java

```
package GUI;

import org.jfree.chart.ChartFactory;
import org.jfree.chart.ChartPanel;
import org.jfree.chart.JFreeChart;
import org.jfree.chart.plot.PlotOrientation;
import org.jfree.data.category.DefaultCategoryDataset;
import DBmanager.DBmanager;
import java.util.List;
import org.jfree.chart.plot.CategoryPlot;
import org.jfree.chart.axis.NumberAxis;
import java.awt.GridLayout;

/**
 * Παράθυρο για την προβολή στατιστικών στοιχείων της εφαρμογής.
 * Χρησιμοποιεί τη βιβλιοθήκη JFreeChart για τη δημιουργία διαγραμμάτων (Bar
Charts)
 * Τα διαγράμματα είναι για τις δημοφιλείς αναζητήσεις και
 * για τις κατηγορίες των αποθηκευμένων άρθρων.
 */
public class StatisticsScreen extends javax.swing.JDialog {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(StatisticsScreen.class.getName());
    private final DBmanager db;

    /**
     * Ο κατασκευαστής του παραθύρου για τα στατιστικά.
     *
     * @param parent Το γονικό παράθυρο.
     * @param modal true για modal λειτουργία ("κλείδωμα" της εφαρμογής μέχρι
να κλείσει το παράθυρο)
     * @param db Ο διαχειριστής της βάσης δεδομένων.
     */
    public StatisticsScreen(java.awt.Frame parent, boolean modal, DBmanager
db) {
        super(parent, modal);
        this.db = db;

        initComponents();

        setTitle("Στατιστικά Δεδομένα");
        setSize(800, 800);

        initCharts();
    }

    /**
     * Αρχικοποιεί και δημιουργεί τα γραφήματα.
     */
    private void initCharts() {
        // Δημιουργία των Datasets
        DefaultCategoryDataset searchDataset = new DefaultCategoryDataset();
```

```
        DefaultCategoryDataset categoryDataset = new
DefaultCategoryDataset();

        // Ανάκτηση δεδομένων από τη βάση
        List<Object[]> searchStats = db.getSearchStatistics();
        List<Object[]> categoryStats = db.getCategoryStatistics();

        //A :Δημιουργία διαγράμματος για τις αναζητήσεις (Search Statistics)
        if (searchStats != null){
            for (Object[] row : searchStats) {
            // row[0] = λέξη, row[1] = πλήθος
            searchDataset.addValue((Number) row[1], "Αναζητήσεις", (String)
row[0]);
            }
        }

        // Δημιουργία του Bar Chart για τις αναζητήσεις
        JFreeChart searchChart = ChartFactory.createBarChart(
                "Δημοφιλείς Αναζητήσεις",   // Τίτλος
                "Λέξη Κλειδί",              // Άξονας X
                "Πλήθος",                   // Άξονας Y
                searchDataset,              // Δεδομένα
                PlotOrientation.VERTICAL,
                true,
                true,
                false);

        formatChartPlot(searchChart);

        //B :Δημιουργία διαγράμματος για τις κατηγορίες (category Statistics)
        if (categoryStats !=null){
             for (Object[] row : categoryStats) {
            // row[0] = λέξη, row[1] = πλήθος
            categoryDataset.addValue((Number) row[1], "Κατηγορίες", (String)
row[0]);
            }
        }

        // Δημιουργία του Bar Chart για τις Κατηγορίες
        JFreeChart categoryChart = ChartFactory.createBarChart(
                "Δημοφιλείς Κατηγορίες",    // Τίτλος
                "Κατηγορία",                // Άξονας X
                "Πλήθος",                   // Άξονας Y
                categoryDataset,            // Δεδομένα
                PlotOrientation.VERTICAL,
                true,
                true,
                false);

        formatChartPlot(categoryChart);

        // Εμφάνιση των γραφημάων στο παράθυρο
        ChartPanel searchPanel = new ChartPanel(searchChart);
        add(searchPanel);
        ChartPanel categoryPanel = new ChartPanel(categoryChart);
```

```
        add(categoryPanel);
    }

    /**
     * Μορφοποιεί το Plot του γραφήματος ώστε οι άξονες να εμφανίζουν μόνο
ακέραιες τιμές.
     *
     * @param chart Το γράφημα προς μορφοποίηση.
     */
    private void formatChartPlot(JFreeChart chart) {
        CategoryPlot plot = chart.getCategoryPlot();
        NumberAxis yAxis = (NumberAxis) plot.getRangeAxis();
        // Εξασφάλιση ότι ο άξονας Y δείχνει μόνο ακέραιους (π.χ. όχι 1.5
αναζήτηση)
        yAxis.setStandardTickUnits(NumberAxis.createIntegerTickUnits());
    }
}
```

3.3 Ερώτημα Γ – Παρουσίαση στατιστικών

Κλάσεις PdfExporter Package:

Ο κώδικας για τη κλάση pdfExporter.java

```
package PdfExporter;

import java.io.IOException;
import java.util.List;
import com.itextpdf.kernel.font.PdfFont;
import com.itextpdf.kernel.font.PdfFontFactory;
import com.itextpdf.kernel.pdf.PdfDocument;
import com.itextpdf.kernel.pdf.PdfWriter;
import com.itextpdf.layout.Document;
import com.itextpdf.layout.element.Paragraph;
import com.itextpdf.layout.element.Table;
import com.itextpdf.layout.element.Cell;
import com.itextpdf.layout.properties.UnitValue;

/**
 * Εξάγει σε pdf τα στατιστικά από τη ΒΔ
 */
public class pdfExporter
{
    //μεταβλητές που χρησιμοποιούνται από όλα τα methods
    private String filename;
    private Document document;
    private PdfFont greekFontBold;
    private PdfFont greekFont;

    /**
     * Constructor που αρχικοποιεί τα fonts που θα χρησιμοποιηθούν
     * (truetype Arial για υποστήριξη ελληνικών) - Απαιτεί το φάκελο Fonts
     * @throws IOException σε περίπτωση αδυναμίας πρόσβασης στα ttf αρχεία
     */
    public pdfExporter() throws IOException
    {
        // λήψη του path για κάθε font ως input stream
        java.io.InputStream boldStream =
getClass().getResourceAsStream("/fonts/arialbd.ttf");
        java.io.InputStream regStream =
getClass().getResourceAsStream("/fonts/arial.ttf");

        if (boldStream == null || regStream == null) {
            throw new IOException("Fonts not found in JAR! Ensure they are in
src/fonts/");
        }

        // ανάγνωση ως byte arrays
        byte[] boldBytes = boldStream.readAllBytes();
        byte[] regBytes = regStream.readAllBytes();

        // δημιουργία των κατάλληλων fonts
```

```
        this.greekFontBold =
com.itextpdf.kernel.font.PdfFontFactory.createFont(
            boldBytes,
            com.itextpdf.io.font.PdfEncodings.IDENTITY_H,

com.itextpdf.kernel.font.PdfFontFactory.EmbeddingStrategy.FORCE_EMBEDDED
        );

        this.greekFont = com.itextpdf.kernel.font.PdfFontFactory.createFont(
            regBytes,
            com.itextpdf.io.font.PdfEncodings.IDENTITY_H,

com.itextpdf.kernel.font.PdfFontFactory.EmbeddingStrategy.FORCE_EMBEDDED
        );

        boldStream.close();
        regStream.close();
    }
    /**
     * Αρχικοποιεί το pdf document με filename και τίτλο
     * @param filename όνομα αρχείου όπως δόθηκε από το χρήστη
     * @throws IOException σε περίπτωση αποτυχίας δημιουργίας του αρχείου
     */
    public void init(String filename) throws IOException
    {
        this.filename = filename;
        //αρχικοποίηση pdfWriter&Document από iText
        //όρισμα του writer το path για το αρχείο
        PdfWriter writer = new PdfWriter(filename + ".pdf");
        PdfDocument pdf = new PdfDocument(writer);
       this.document = new Document(pdf);
    }

    /**
     * Δημιουργεί τους πίνακες στατιστικών και τους τοποθετεί στο pdf
     * @param dbList λίστα Object[] στατιστικών όπως επιστρέφεται από τη ΒΔ
     * @param columnTitles array με τους τίτλους των στηλών
     * @param title ο τίτλος του πίνακα
     * @throws ArrayIndexOutOfBoundsException αν το πλήθος τίτλων στηλών δεν
     * αντιστοιχεί στο ίδιο πλήθος στατιστικών στοιχείων κάθε object της
dbList
     */
    public void addTable(List <Object[]> dbList, String title, String[]
columnTitles) throws ArrayIndexOutOfBoundsException
    {
        //Τίτλος του πίνακα
        this.document.add(new Paragraph (title)
                .setFont(greekFont)
                .setFontSize(18));
        //το πλήθος των στηλών και των δεδομένων κάθε στοιχείου πρέπει να
ταιριάζουν
        if (dbList.get(0).length != columnTitles.length)
        {
            throw new ArrayIndexOutOfBoundsException("Non matching lengths of
dblist and column titles");
```

```
        }
        //δημιουργία columnTitles στηλών ίσου μήκους
        int columnCount = columnTitles.length;
        float[] columnWidths = new float[columnTitles.length];
        for (int i = 0; i < columnCount; i++)
        {
            columnWidths[i] = 1;
        }
        Table table = new Table(UnitValue.createPointArray(columnWidths));
        table.setWidth(UnitValue.createPercentValue(100));

        //προσθήκη τίτλων στηλών
        for (int i = 0; i < columnCount; i++)
        {
            table.addHeaderCell(new Cell().add(new
Paragraph(columnTitles[i]).setFont(this.greekFontBold)));
        }
        //για κάθε index του κάθε Object[] της λίστας δημιουργούνται κελιά
        dbList.stream()
                .limit(10)
                .forEach(o ->
        {
            for (int i = 0; i < columnCount; i++)
            {
                String data = o[i].toString();
               table.addCell(new Cell().add(new
Paragraph(data).setFont(greekFont)));
            }
        });
        //προσθήκη του πίνακα στο pdf document
        this.document.add(table);
        //μικρό κενό μεταξύ των πινάκων
        for (int i = 0; i < 5; i++)
        {
            this.document.add(new Paragraph(" "));
        }
    }

    /**
     * Ολκληρώνει τη δημιουργία του pdf και εμφανίζει σχετικό μήνυμα στο
console
     */
    public void export()
    {
        this.document.close();
        System.out.println("Pdf succesfuly created: " + filename + ".pdf");
    }
}
```

Ο κώδικας για τη κλάση StatisticsScreen.java

```
package GUI;

import org.jfree.chart.ChartFactory;
```

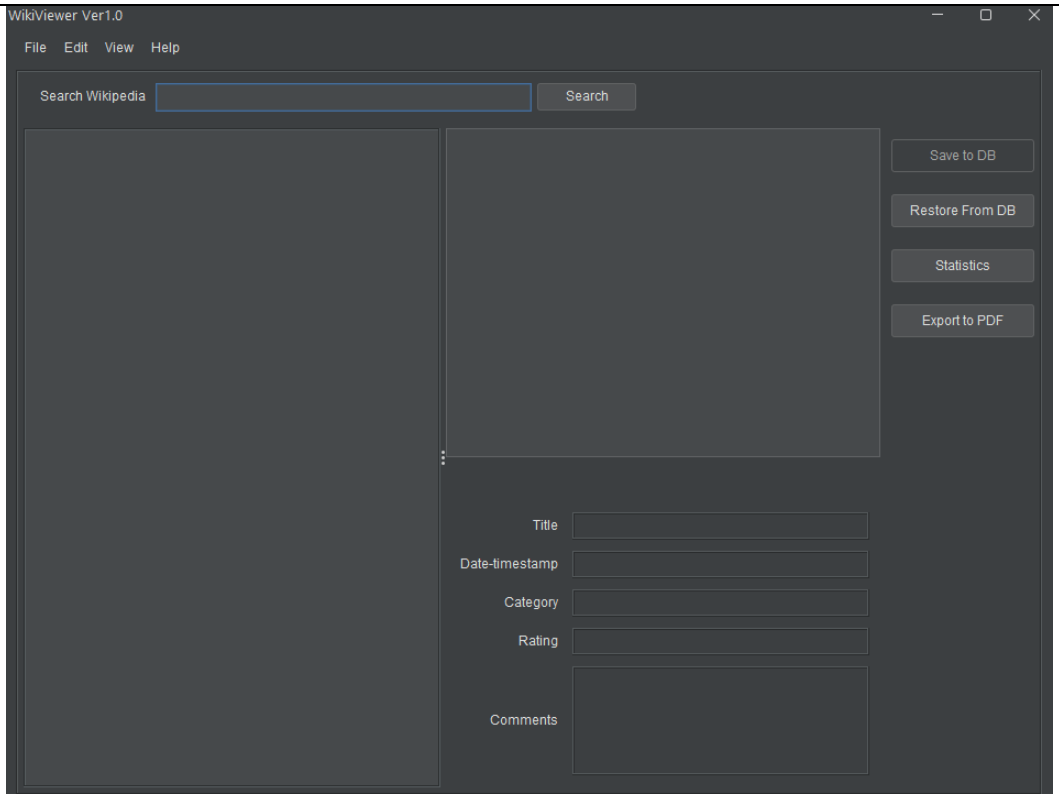
```
import org.jfree.chart.ChartPanel;
import org.jfree.chart.JFreeChart;
import org.jfree.chart.plot.PlotOrientation;
import org.jfree.data.category.DefaultCategoryDataset;
import DBmanager.DBmanager;
import java.util.List;
import org.jfree.chart.plot.CategoryPlot;
import org.jfree.chart.axis.NumberAxis;
import java.awt.GridLayout;

/**
 * Παράθυρο για την προβολή στατιστικών στοιχείων της εφαρμογής.
 * Χρησιμοποιεί τη βιβλιοθήκη JFreeChart για τη δημιουργία διαγραμμάτων (Bar
Charts)
 * Τα διαγράμματα είναι για τις δημοφιλείς αναζητήσεις και
 * για τις κατηγορίες των αποθηκευμένων άρθρων.
 */
public class StatisticsScreen extends javax.swing.JDialog {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(StatisticsScreen.class.getName());
    private final DBmanager db;

    /**
     * Ο κατασκευαστής του παραθύρου για τα στατιστικά.
     *
     * @param parent Το γονικό παράθυρο.
     * @param modal true για modal λειτουργία ("κλείδωμα" της εφαρμογής μέχρι
να κλείσει το παράθυρο)
     * @param db Ο διαχειριστής της βάσης δεδομένων.
     */
    public StatisticsScreen(java.awt.Frame parent, boolean modal, DBmanager
db) {
        super(parent, modal);
        this.db = db;

        initComponents();

        setTitle("Στατιστικά Δεδομένα");
        setSize(800, 800);

        initCharts();
    }

    /**
     * Αρχικοποιεί και δημιουργεί τα γραφήματα.
     */
    private void initCharts() {
        // Δημιουργία των Datasets
        DefaultCategoryDataset searchDataset = new DefaultCategoryDataset();
        DefaultCategoryDataset categoryDataset = new
DefaultCategoryDataset();

        // Ανάκτηση δεδομένων από τη βάση
        List<Object[]> searchStats = db.getSearchStatistics();
```

```
        List<Object[]> categoryStats = db.getCategoryStatistics();

        //A :Δημιουργία διαγράμματος για τις αναζητήσεις (Search Statistics)
        if (searchStats != null){
            for (Object[] row : searchStats) {
            // row[0] = λέξη, row[1] = πλήθος
            searchDataset.addValue((Number) row[1], "Αναζητήσεις", (String)
row[0]);
            }
        }

        // Δημιουργία του Bar Chart για τις αναζητήσεις
        JFreeChart searchChart = ChartFactory.createBarChart(
                "Δημοφιλείς Αναζητήσεις",   // Τίτλος
                "Λέξη Κλειδί",              // Άξονας X
                "Πλήθος",                   // Άξονας Y
                searchDataset,              // Δεδομένα
                PlotOrientation.VERTICAL,
                true,
                true,
                false);

        formatChartPlot(searchChart);

        //B :Δημιουργία διαγράμματος για τις κατηγορίες (category Statistics)
        if (categoryStats !=null){
             for (Object[] row : categoryStats) {
            // row[0] = λέξη, row[1] = πλήθος
            categoryDataset.addValue((Number) row[1], "Κατηγορίες", (String)
row[0]);
            }
        }

        // Δημιουργία του Bar Chart για τις Κατηγορίες
        JFreeChart categoryChart = ChartFactory.createBarChart(
                "Δημοφιλείς Κατηγορίες",    // Τίτλος
                "Κατηγορία",                // Άξονας X
                "Πλήθος",                   // Άξονας Y
                categoryDataset,            // Δεδομένα
                PlotOrientation.VERTICAL,
                true,
                true,
                false);

        formatChartPlot(categoryChart);

        // Εμφάνιση των γραφημάων στο παράθυρο
        ChartPanel searchPanel = new ChartPanel(searchChart);
        add(searchPanel);
        ChartPanel categoryPanel = new ChartPanel(categoryChart);
        add(categoryPanel);
    }

    /**
```

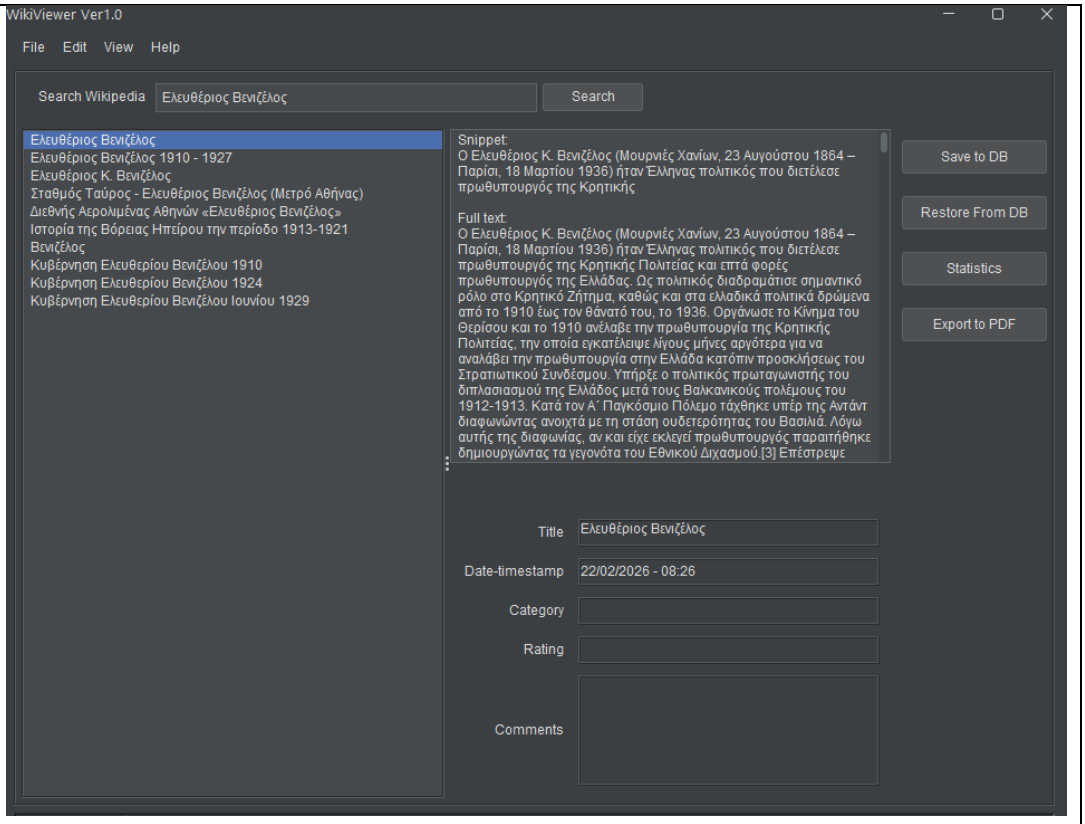
```
     * Μορφοποιεί το Plot του γραφήματος ώστε οι άξονες να εμφανίζουν μόνο
ακέραιες τιμές.
     *
     * @param chart Το γράφημα προς μορφοποίηση.
     */
    private void formatChartPlot(JFreeChart chart) {
        CategoryPlot plot = chart.getCategoryPlot();
        NumberAxis yAxis = (NumberAxis) plot.getRangeAxis();
        // Εξασφάλιση ότι ο άξονας Y δείχνει μόνο ακέραιους (π.χ. όχι 1.5
αναζήτηση)
        yAxis.setStandardTickUnits(NumberAxis.createIntegerTickUnits());
    }

    /**
     * This method is called from within the constructor to initialize the
form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {


setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        getContentPane().setLayout(new java.awt.GridLayout(2, 1, 0, 10));

        pack();
    }// </editor-fold>

    // Variables declaration - do not modify
    // End of variables declaration
}
```

3.4 Ερώτημα Δ – Συνολικός Έλεγχος και Εκτέλεση της Εφαρμογής

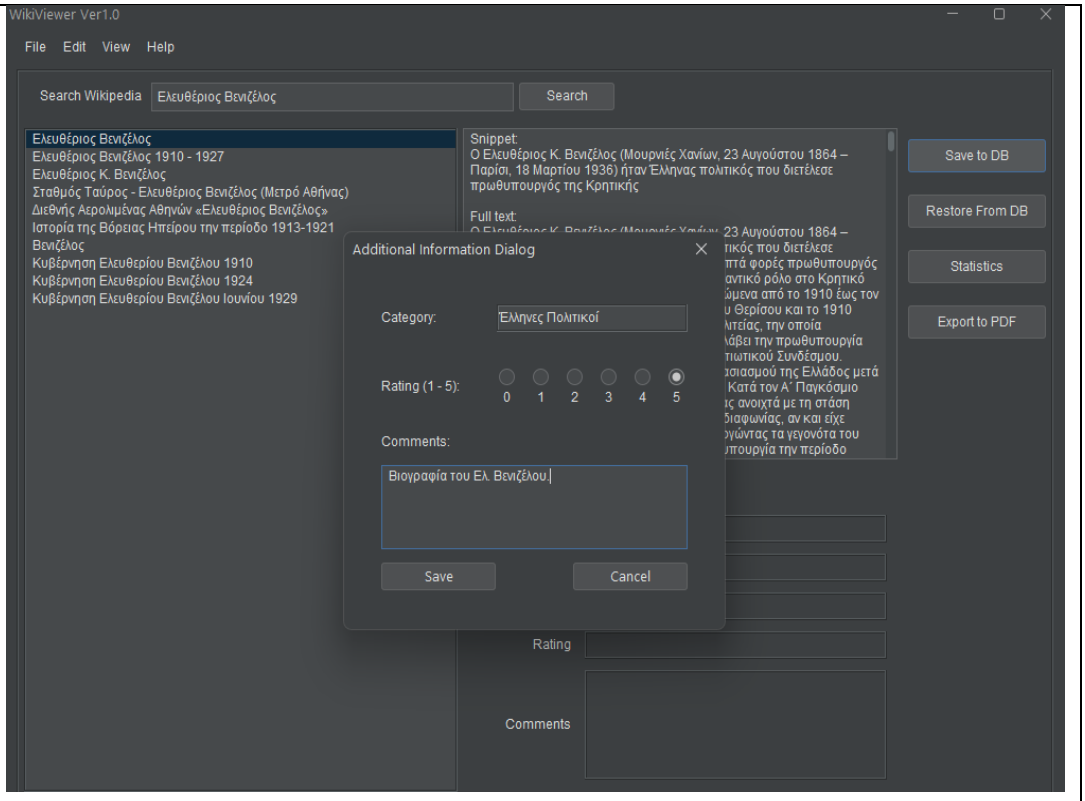
Α)

Απαίτηση	**Εικόνα**
R1	 *Στο βασικό μενού που εμφανίζεται με την εκκίνηση της εφαρμογής, διατίθενται στο χρήστη οι εξής επιλογές:* • *Αναζήτηση άρθρου (Live από το API της ελληνικής Wikipedia) βάσει λέξης-κλειδιού που εισάγεται στη φόρμα **Search Wikipedia.*** • *Προβολή των τοπικά αποθηκευμένων άρθρων από το χρήστη κάνοντας κλικ στο κουμπί **Restore From DB.*** • *Αποθήκευση επιλεγμένου άρθρου μετά από αναζήτηση στο API, κάνοντας κλικ στο κουμπί **Save to DB.** Ο χρήστης έχει τη δυνατότητα εμπλουτισμού των στοιχείων του άρθρου που αποθηκεύεται.* • *Προβολή στατιστικών αναζητήσεων με βάση τη λέξη-κλειδί, αλλά και των αποθηκευμένων άρθρων ανα κατηγορία με κλικ στο κουμπί **Statistics**. Τα στοιχεία παρατίθενται και σε pdf με την επιλογή **Export to PDF.***

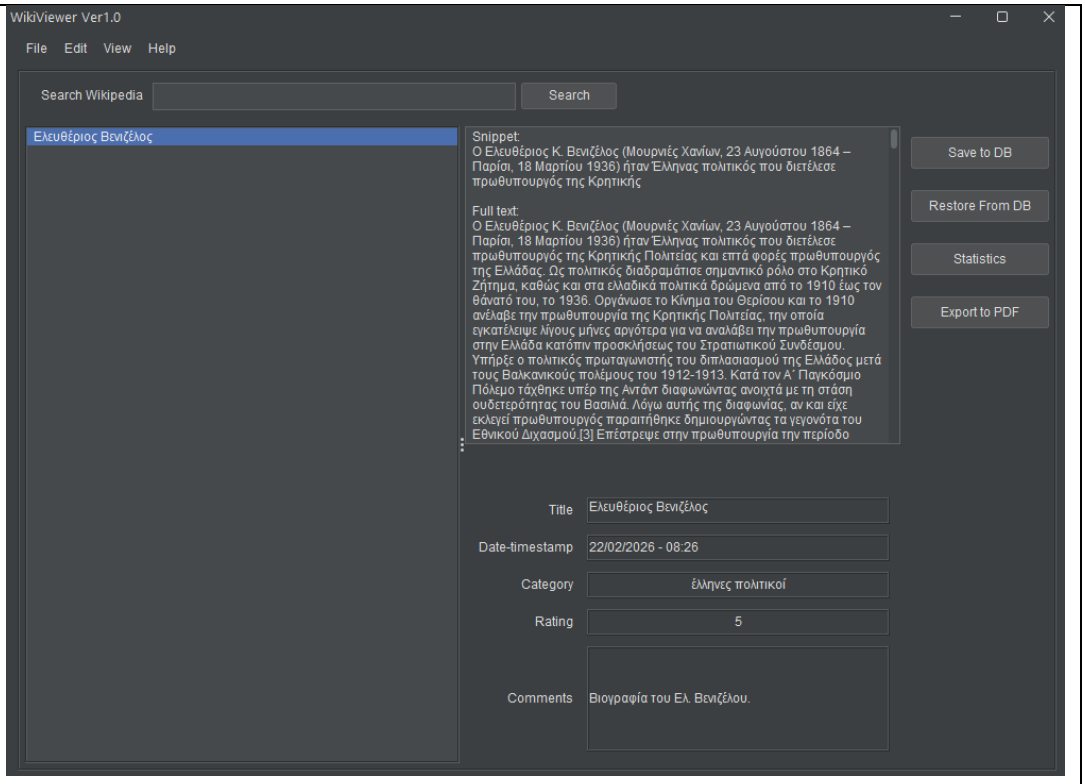
R2



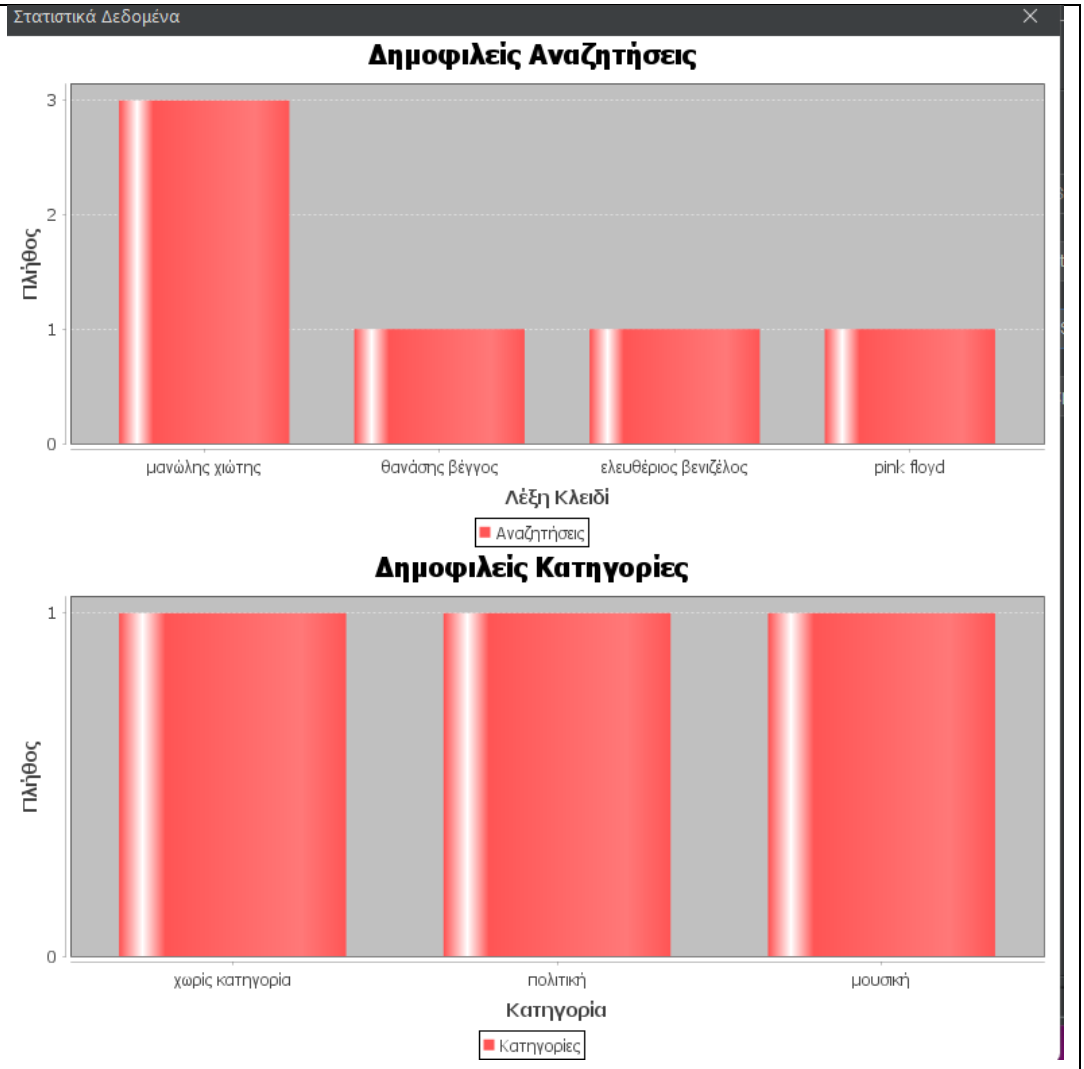
*Κατά την αναζήτηση άρθρων βάσει μίας λέξης – κλειδιού από το API, ο χρήστης πληκτρολογεί τον όρο αναζήτησης στο πεδίο **Search Wikipedia** και έπειτα πατάει το κουμπί **Search,** είτε απλά **Enter.** Τα αποτελέσματα εμφανίζονται σε λίστα κάτω από το πεδίο αναζήτησης. Με την επιλογή ενός άρθρου, εμφανίζονται στα πλαίσια της εφαρμογής οι πληροφορίες του. Συγκεκριμένα, ο χρήστης μπορεί να δει το απόσπασμα κειμένου (**Snippet**) όπου εμφανίζεται η λέξη – κλειδί που αναζήτησε, καθώς και το πλήρες κείμενο (**Full Text**) του άρθρου. Στα πεδία **Title** και **Date – Timestamp** παρουσιάζονται ο τίτλος και η ημερομηνία τελευταίας καταχώρησης στο άρθρο.*

R3	 *Πατώντας το πλήκτρο **Save to DB** με επιλεγμένο κάποιο άρθρο από τη λίστα αποτελεσμάτων αναζήτησης στο API, το άρθρο μπορεί να αποθηκευτεί στην τοπική βάση δεδομένων της εφαρμογής. Προαιρετικά, ο χρήστης μπορεί να προσθέσει πεδία όπως **Κατηγορία (Category), Βαθμολογία ενδιαφέροντος (Rating), Σχόλια (Comments).***

R4



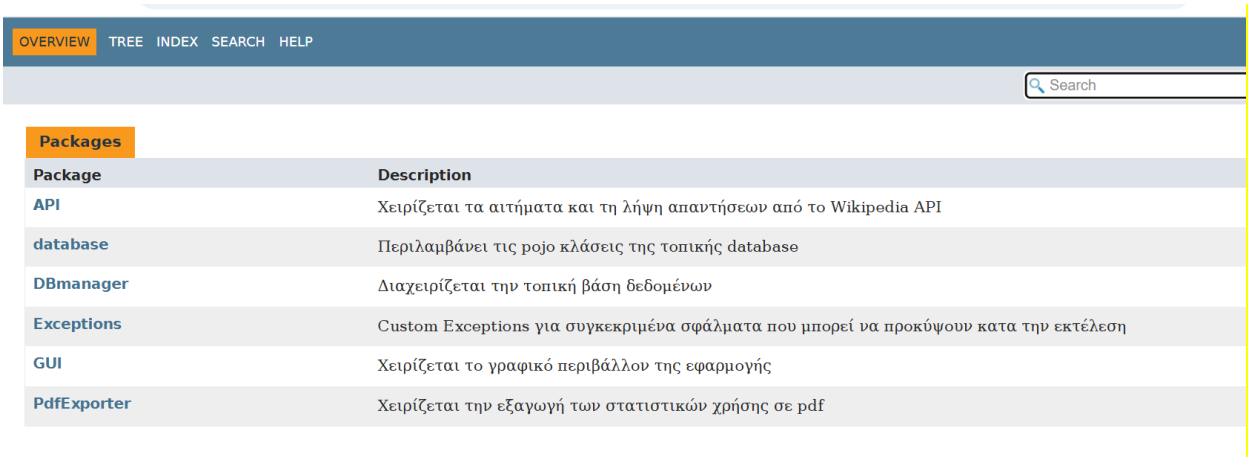
*Ένα αποθηκευμένο στην τοπική βάση δεδομένων άρθρο ανακτάται πατώντας το κουμπί **Restore From DB,** ενώ εμφανίζεται με τα εμπλουτισμένα δεδομένα που ενδεχομένως έχει προσθέσει ο χρήστης. Ο χρήστης μπορεί να επαναλάβει τη διαδικασία αποθήκευσης εκ νέου πατώντας και πάλι **Save to DB** προκειμένου να τροποποιήσει τα δεδομένα αυτά.*

R5	 *Πατώντας το κουμπί **Statistics** εμφανίζονται σε νέο παράθυρο:* • *Οι πιο συχνές λέξεις – κλειδιά που έχει αναζητήσει ο χρήστης* • *Πόσα άρθρα έχουν αποθηκευτεί ανα κατηγορία που έχει οριστεί στο R3* *Παρέχεται η δυνατότητα εξαγωγής των στατιστικών αυτών σε πινακοποιημένη μορφή σε αρχείο μορφής pdf, κάνοντας κλικ στο κουμπί **Export to PDF.***

B)

Διαδρομή:

/WikiViewer/dist/javadoc/index.html



Package	Description
API	Χειρίζεται τα αιτήματα και τη λήψη απαντήσεων από το Wikipedia API
database	Περιλαμβάνει τις pojo κλάσεις της τοπικής database
DBmanager	Διαχειρίζεται την τοπική βάση δεδομένων
Exceptions	Custom Exceptions για συγκεκριμένα σφάλματα που μπορεί να προκύψουν κατα την εκτέλεση
GUI	Χειρίζεται το γραφικό περιβάλλον της εφαρμογής
PdfExporter	Χειρίζεται την εξαγωγή των στατιστικών χρήσης σε pdf

4 ΚΡΙΤΙΚΟΣ ΑΠΟΛΟΓΙΣΜΟΣ ΤΟΥ ΕΡΓΟΥ

4.1 Προβλήματα που αντιμετωπίσαμε και τι θα αλλάζαμε.

Κατά την υλοποίηση του έργου, αντιμετωπίσαμε αρκετές τεχνικές και σχεδιαστικές προκλήσεις σε τρία βασικά επίπεδα:

- **Βάση Δεδομένων.**
  Αρχικά, η βάση δημιουργήθηκε ως αυτόνομος server. Στην πορεία διαπιστώσαμε ότι η αρχιτεκτονική client-server απαιτούσε χειροκίνητη εκκίνηση, δημιουργώντας προβλήματα φορητότητας. Έτσι, πήραμε τη στρατηγική απόφαση να τη μετατρέψουμε σε Embedded (ενσωματωμένη). Παράλληλα, αντιμετωπίσαμε προβλήματα με τον χειρισμό των κενών (null) τιμών από τη βάση (π.χ. απουσία βαθμολογίας), τα οποία προκαλούσαν NumberFormatException. Η λύση δόθηκε με refactoring της μεταβλητής rating στα Entities από πρωταρχικό τύπο int σε αντικείμενο Integer.

- **Διασύνδεση με το API.**
  Η υλοποίηση αποστολής αιτημάτων στο Wiki API μέσω της βιβλιοθήκης OkHttp συνάντησε σοβαρές δυσκολίες λόγω των πολλαπλών της εξαρτήσεων (Okio, Kotlin-stdlib). Η εύρεση συμβατών .jar αρχείων και η χειροκίνητη ενσωμάτωσή τους στο Apache Ant ήταν δυσχερής λόγω μη καταγεγραμμένων ασυμβατοτήτων, απαιτώντας σημαντικό χρόνο για τον συντονισμό και συγχρονισμό του project μεταξύ των μελών της ομάδας.
  Η προσθήκη (αργότερα) λειτουργικότητας για την εμφάνιση του πλήρους κειμένου δημιούργησε μια νέα, μη λειτουργική απαίτηση επιδόσεων. Τα πολλαπλά ταυτόχρονα αιτήματα προκαλούσαν καθυστερήσεις, καθιστώντας δυσχερή τη χρήση σε πραγματικό χρόνο και απαιτώντας πιο αποδοτικές λύσεις κλήσεων. Επίσης, κατά τις δοκιμές, διαπιστώσαμε την επιστροφή malformed json από το API. Αυτό το runtime exception δεν μπορούσε να προβλεφθεί πριν από τη διαδικασία του deserialize, και ο χειρισμός του απαίτησε τη διεξοδική διερεύνηση του class tree της βιβλιοθήκης Gson.

- **Γραφικό Περιβάλλον (GUI).**
  Η μετάβαση από απλά layouts σε πιο σύνθετες διεπαφές αποτέλεσε σημαντική τεχνική πρόκληση. Αρχικά, στο Status Bar χρησιμοποιήθηκε ο FlowLayout, ο οποίος δεν επέτρεπε τη σωστή κατακόρυφη στοίχιση, πρόβλημα που επιλύθηκε με τη μετάβαση σε GridBagLayout. Παράλληλα, για τον ακριβή διαμοιρασμό (50-50) της οθόνης μέσω του JSplitPane, διαπιστώσαμε ότι η απλή κλήση μεθόδων δεν αρκούσε. Χρειάστηκε να παρέμβουμε προγραμματιστικά μέσω της SwingUtilities.invokeLater, ώστε να διασφαλιστεί ότι το GUI έχει φορτώσει πλήρως πριν γίνει ο διαμοιρασμός. Τέλος,

κληθήκαμε να επιλύσουμε σχεδιαστικά εμπόδια μορφοποίησης: στο JTextArea το κείμενο εμφανιζόταν σε μία συνεχή γραμμή (λύθηκε με τις ιδιότητες setLineWrap και setWrapStyleWord), η στοίχιση του κειμένου στα radio buttons χρειάστηκε παραμετροποίηση (verticalTextPosition), ενώ για τη σωστή εμφάνιση περιθωρίων στο status bar απαιτήθηκε η σύνθεση πλαισίων μέσω ενός CompoundBorder.Αν είχαμε τη δυνατότητα να αλλάξουμε κάτι στο τρέχον έργο, θα αφιερώναμε περισσότερο χρόνο στο Sprint 1 για τον αυστηρό σχεδιασμό των τύπων δεδομένων και τον εξαρχής ορισμό της Embedded αρχιτεκτονικής, ώστε να αποφύγουμε την απώλεια χρόνου στο debugging και το refactoring.

4.2 Αποκλίσεις στις επαναλήψεις (Sprints) σε σχέση με τα προβλεπόμενα.

Κατά το πρώτο Sprint, ο χρόνος που χρειάστηκε για την εκμάθηση και το στήσιμο της βάσης Derby, η εξοικείωση με το API της Wikipedia αλλά και με τη βιβλιοθήκη Swing είχε ως αποτέλεσμα η ταχύτητα της ομάδας (Velocity) να είναι ελαφρώς χαμηλότερη στην αρχή. Βάσει αυτού, επιβεβαιώσαμε την ορθότητα της απόφασής μας να εντάξουμε την πλήρη ανάπτυξη των στατιστικών λειτουργιών και την εξαγωγή σε PDF στο Sprint 2. Μέσω της ευελιξίας της μεθοδολογίας Scrum, αναπροσαρμόσαμε τον φόρτο εργασίας των μελών και καταφέραμε να παραδώσουμε τα παραδοτέα εντός του συνολικού χρονοδιαγράμματος, κλείνοντας όλα τα Sprints χωρίς καμία απόκλιση από τα Backlogs τους.

4.3 Χρόνος που απαιτήθηκε σε σχέση με τον προϋπολογισμένο.

Η παρακολούθηση του χρόνου ανέδειξε ενδιαφέροντα συμπεράσματα, τα οποία καταγράφηκαν και στα διαγράμματα κατανάλωσης προσπάθειας (Burndown Charts). Στο Sprint 1, παρατηρήθηκαν υποεκτιμήσεις του χρόνου: για παράδειγμα, η «Αναζήτηση Άρθρου» και το «Αρχικό GUI» απαίτησαν 6 και 10 ανθρωποημέρες αντίστοιχα, ξεπερνώντας το αρχικό πλάνο, λόγω της αρχικής δυσκολίας ενσωμάτωσης των εργαλείων. Αντίθετα, στο Sprint 2, η ομάδα βρήκε τον ρυθμό της και ο πραγματικός χρόνος (σε ημέρες) ταυτίστηκε εντυπωσιακά με την εκτιμώμενη δυσκολία (Story Points) των απαιτήσεων. Το πιο θετικό στοιχείο καταγράφηκε στο Sprint 3, όπου ο απαιτούμενος χρόνος για τον «Συνολικό Έλεγχο» (Testing) υπερεκτιμήθηκε (ολοκληρώθηκε σε 4 ημέρες αντί για τις αναμενόμενες 13 μονάδες προσπάθειας). Αυτό συνέβη διότι η συνεχής ενοποίηση (Github commits) και ο έλεγχος του κώδικα κατά τα προηγούμενα Sprints, μας παρέδωσε ένα ήδη σταθερό σύστημα.

Αξίζει να σημειωθεί ότι ο αρχικός μας προγραμματισμός (Sprints) αποδείχθηκε εξαιρετικά ακριβής. Ως αποτέλεσμα, η παράταση των 7 ημερών που δόθηκε για την παράδοση του έργου

δεν ήταν αυστηρά απαραίτητη για την υλοποίηση των παραδοτέων. Ωστόσο, η ομάδα αξιοποίησε τον επιπλέον χρόνο στρατηγικά, ως «περιθώριο ασφαλείας». Αυτό μας επέτρεψε να ολοκληρώσουμε τη συγγραφή της αναφοράς και τον τελικό ποιοτικό έλεγχο χωρίς πίεση και άγχος, βελτιώνοντας το τελικό αποτέλεσμα και διατηρώντας την ευημερία της ομάδας, μια βασική αρχή των Ευέλικτων Μεθοδολογιών.

4.4 Αξιολόγηση εργαλείου συνεργασίας Trello.

Ο ρόλος του Scrum Master, αλλά και ο συντονισμός όλης της ομάδας, διευκολύνθηκε καθοριστικά από το Trello. Η διαμόρφωση του Agile Board σε κάθετες λίστες με βάση τις επαναλήψεις (Product Backlog, Sprint 1, Sprint 2, Sprint 3) μας επέτρεψε να έχουμε πλήρη εποπτεία του χρονοδιαγράμματος. Η χρήση των checklists με τα Κριτήρια Αποδοχής για κάθε κάρτα ήταν καθοριστική για την ομαλή εξέλιξη του έργου. Στην καθημερινή μας εργασία, μας βοήθησε ιδιαίτερα η επέκταση περιηγητή «Scrum for Trello», η οποία υπολόγιζε αυτόματα τα Story Points, γλιτώνοντάς μας από χειροκίνητους υπολογισμούς. Επίσης, η διασύνδεση (Power-Up) του Trello με το GitHub αποδείχθηκε η πιο κρίσιμη, καθώς κάθε ολοκληρωμένη κάρτα συνδεόταν αυτόματα με το αντίστοιχο commit του κώδικα, ελαχιστοποιώντας τα προβλήματα επικοινωνίας.

4.5 Κίνδυνοι που πραγματώθηκαν και τρόπος απόκρισης.

Όπως αναφέρθηκε, η ενσωμάτωση λειτουργικότητας για την εμφάνιση του πλήρους κειμένου των άρθρων δημιούργησε καθυστερήσεις λόγω πολλαπλών αιτημάτων προς το API. Αποκριθήκαμε διερευνώντας εναλλακτικές, πιο αποδοτικές λύσεις κλήσεων ώστε να μην «παγώνει» η εφαρμογή.

Σε οργανωτικό επίπεδο, αντιμετωπίσαμε κάποια θέματα με το Git, καθώς το παραγόμενο αρχείο derby.log της ενσωματωμένης βάσης παρακολουθούνταν από το αποθετήριο. Το λύσαμε άμεσα με τη χρήση της εντολής git rm --cached και την προσθήκη του αρχείου στο .gitignore για να μην «παρακολουθείται» πλέον από το repository.

4.6 Τι θα αλλάζαμε σε ένα επόμενο αντίστοιχο έργο.

Σε μια μελλοντική υλοποίηση αντίστοιχου έργου, θα εστιάζαμε στη βελτίωση των εσωτερικών μας διαδικασιών και θα προχωρούσαμε στις κάτωθι αλλαγές:

- **Αυστηρότερη στρατηγική Git.**
  Για να ελαχιστοποιήσουμε τα merge conflicts που συναντήσαμε (όπως με το κλείδωμα των αρχείων της βάσης), θα υιοθετούσαμε μια πιο επαγγελματική ροή στο GitHub. Συγκεκριμένα, κάθε νέα απαίτηση θα αναπτυσσόταν σε ξεχωριστό κλάδο (Branch) και

θα ενσωματωνόταν στον κύριο κώδικα μέσω Pull Request (PR), αφού πρώτα γινόταν ένας γρήγορος έλεγχος κώδικα (Code Review) από ένα δεύτερο μέλος της ομάδας.

- **Ορθότερη εκτίμηση του μη προγραμματιστικού φόρτου.**
  Στο τρέχον έργο, η διαδικασία του Planning Poker και η απόδοση Story Points επικεντρώθηκε σχεδόν αποκλειστικά στις καθαρά προγραμματιστικές απαιτήσεις. Ωστόσο, στην πράξη διαπιστώσαμε ότι ο συντονισμός της ομάδας, η διαχείριση του Trello, η προετοιμασία των Sprints και η επίλυση των git conflicts (καθήκοντα κυρίως του Scrum Master) απαίτησαν σημαντικό χρόνο που δεν είχε αποτυπωθεί στο Backlog. Σε ένα επόμενο έργο, θα εντάσσαμε αυτές τις οργανωτικές εργασίες ως διακριτές κάρτες.

- **Πιο ισορροπημένη κατανομή της προσπάθειας.**
  Λαμβάνοντας υπόψη την τεχνική εξοικείωση που πλέον διαθέτουμε, θα αναδιαμορφώναμε την κατανομή των Story Points ανά επανάληψη. Στο παρόν έργο, το Sprint 2 και το Sprint 3 επωμίστηκαν τον μεγαλύτερο όγκο (39 και 37 Story Points αντίστοιχα), καθώς το Sprint 1 αναλώθηκε αναπόφευκτα στην καμπύλη εκμάθησης (API, DB, GUI). Έχοντας πλέον κατακτήσει την τεχνογνωσία, σε ένα νέο έργο θα επιδιώκαμε μια πιο ομοιόμορφη κατανομή της πολυπλοκότητας από την πρώτη κιόλας εβδομάδα, αποφεύγοντας έτσι την απόκλιση στον φόρτο που επωμίζεται η ομάδα σε κάθε Sprint.

4.7 Θετικά σημεία που αποκομίσαμε από την εργασία.

Η εμπειρία από την υλοποίηση του «WikiViewer» ήταν εξαιρετικά ωφέλιμη. Μάθαμε στην πράξη την αξία του διαχωρισμού των επιπέδων. Η δημιουργία της κλάσης DBmanager ως boundary class προσέφερε επεκτασιμότητα, αφού το GUI δεν ήρθε ποτέ σε άμεση επαφή με τα πολύπλοκα JPQL ερωτήματα.

Η τριβή με εργαλεία ελέγχου εκδόσεων (GitHub), πλατφόρμες Agile (Trello) και REST APIs, μας προσέφερε δεξιότητες που είναι απολύτως απαραίτητες στο σύγχρονο επαγγελματικό περιβάλλον.

Η ανάγκη διαχείρισης απρόβλεπτων εξαιρέσεων μας οδήγησε να μελετήσουμε σε βάθος την αρχιτεκτονική τρίτων βιβλιοθηκών (όπως το class tree του Gson). Επιπλέον, τα ζητήματα καθυστερήσεων λόγω πολλαπλών API requests μας εισήγαγαν πρακτικά στην έννοια της βελτιστοποίησης απόδοσης (performance optimization) και της διαχείρισης πόρων σε εφαρμογές πραγματικού χρόνου.

Αποκομίσαμε πολύτιμη εμπειρία στον σχεδιασμό σύγχρονων και φιλικών διεπαφών χρήστη (User Experience) στη Java. Μάθαμε πώς να ενσωματώνουμε εξωτερικά, μοντέρνα θέματα

εμφάνισης (όπως η βιβλιοθήκη FlatLaf) και πώς να δημιουργούμε καθαρές ροές εργασίας χρησιμοποιώντας τον CardLayout για την εναλλαγή οθονών, αποφεύγοντας το άνοιγμα πολλαπλών παραθύρων. Επιπλέον, εξοικειωθήκαμε με τη χρήση του javax.swing.Timer για την εμφάνιση προσωρινών μηνυμάτων στο status bar, προσφέροντας μια πιο διαδραστική εμπειρία στον τελικό χρήστη.

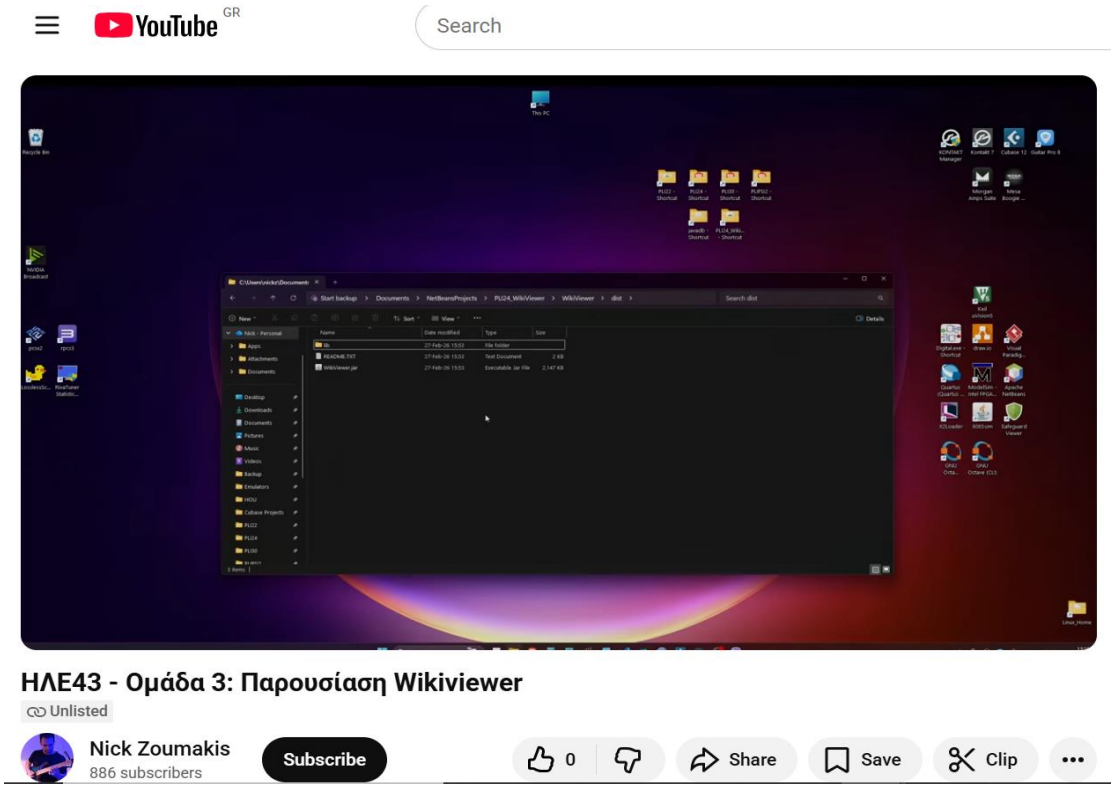
Αποδείξαμε ότι ως ομάδα μπορούμε να αυτο-οργανωθούμε, να ξεπεράσουμε τα τεχνικά εμπόδια του integration και να παραδώσουμε ένα πλήρως λειτουργικό λογισμικό εντός των προκαθορισμένων χρονικών ορίων.

5 VIDEO

Μία σύντομη παρουσίαση της λειτουργικότητας βρίσκεται στον σύνδεσμο:

https://tinyurl.com/3rmupfbp

Εναλλακτικά μπορεί να σαρωθεί το QR code:



6 ΑΝΑΦΟΡΕΣ

Apache Software Foundation. (χ.χ.). Apache Derby tutorial. https://db.apache.org/derby/papers/DerbyTut/index.html

Deitel, P. J., & Deitel, H. M. (2014). Java: How to program. Prentice Hall.

iText. (2016). iText: Jump-Start Tutorial for Java. https://kb.itextpdf.com/itext/itext-jump-start-tutorial-for-java

Jarvis, A. [Ava Jarvis]. (2015, 15 Νοεμβρίου). Agile SCRUM for Trello board demo [Βίντεο]. YouTube. https://www.youtube.com/watch?v=3lSp-T4Fk5U

Jenkov, J. (2016). GSON - Gson. Jenkov Tutorials. https://jenkov.com/tutorials/java-json/gson.html

Joda-Time. (χ.χ.). Home. https://www.joda.org/joda-time/

JSON Viewer. (χ.χ.). http://jsonviewer.stack.hu

MediaWiki. (χ.χ.). API:Main page. Ανακτήθηκε 26 Ιανουαρίου 2026, από https://www.mediawiki.org/wiki/API:Main_page

Panos Fitsilis. (χ.χ.). ΕΑΠ - ΠΛΗ24 [Λίστα αναπαραγωγής]. YouTube. https://www.youtube.com/playlist?list=PL8Prx4IjOaZZyCJakQutExaMKg9g99Ton

Panos Fitsilis. (χ.χ.). ΕΑΠ-ΠΛΗ24-JAVA [Λίστα αναπαραγωγής]. YouTube. https://www.youtube.com/playlist?list=PL8Prx4IjOaZZ_g8LBjuWaNTWz3Hj2UNnh

Panos Fitsilis. (χ.χ.). ΕΑΠ-ΠΛΗ24-UML-FITSILIS [Λίστα αναπαραγωγής]. YouTube. https://www.youtube.com/playlist?list=PL8Prx4IjOaZbOfKkJrTSn02WSzARG3fJ7

Square, Inc. (2019). OkHttp. https://square.github.io/okhttp/

Stack Overflow. (2024). Stack Overflow - Where Developers Learn, Share, & Build Careers. https://stackoverflow.com/

Trello. (χ.χ.). Trello Agile Series: The Sprint Board [Βίντεο]. YouTube. https://www.youtube.com/watch?v=pFbVxXOvQ8I

Wikipedia. (χ.χ.). Special:ApiSandbox. Ανακτήθηκε 26 Ιανουαρίου 2026, από https://el.wikipedia.org/wiki/Special:ApiSandbox

Φιτσιλής, Π. (2023). Ευέλικτες μέθοδοι διοίκησης και διαχείρισης έργων [Προπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις.